

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Ingeniería de Software

Sistema de planificación de cargas para empresas de transporte terrestre de mercancías

Haritz Gómez Sarasola

Dirección

Felipe Ibáñez Anfurrutia

8 de septiembre de 2026

Agradecimientos

En primer lugar, quiero agradecer a Felipe, el tutor del proyecto, por proponerme la idea de este trabajo y guiarme a lo largo de él.

También quiero agradecer a mi familia, y especialmente a Irati, por su apoyo a lo largo de mis estudios y durante la realización del trabajo.

Resumen

Este Trabajo de Fin de Grado describe el desarrollo de una aplicación multiplataforma accesible desde web y dispositivos móviles para la gestión operativa de empresas de transporte terrestre de mercancías. El desarrollo incluye el proceso completo de un proyecto de Ingeniería del Software, desde la planificación y el análisis de requisitos funcionales y no funcionales, hasta las pruebas y el despliegue final, pasando por el diseño y la implementación del sistema.

La funcionalidad principal permite realizar la planificación de los pedidos de transporte y dar de alta a los propios conductores de la empresa, para que cada uno tenga su hoja de ruta con sus tareas asignadas. Además de esta funcionalidad, el sistema desarrollado cuenta con una funcionalidad de generación de cartas de porte digitales, respetando la normativa vigente para esta documentación. La aplicación es *multi-tenant*, está diseñada para ser utilizada por más de una empresa de transporte y cuenta con la opción de que cada empresa invite a sus colaboradores para gestionar los procesos que requieren a más de un usuario de la cadena de suministro.

Para el desarrollo, se ha seguido una arquitectura cliente-servidor integrada con servicios en la nube. El cliente se ha implementado en el framework multiplataforma **Flutter** y el servidor es un monolito implementado en el framework **FastAPI**. Junto a estas tecnologías se ha utilizado el Backend as a Service (BaaS) **Firebase** para la autenticación, la base de datos en la nube, el servicio de notificaciones (FCM) y el repositorio de archivos para almacenar las cartas de porte.

Finalmente, se ha validado el código utilizando pruebas unitarias y de integración en el cliente y el servidor. Se ha seguido un proceso de integración continua (CI) con las pruebas implementadas y utilizando la herramienta de análisis estático de código **SonarCloud**. Tras la verificación, el sistema se ha desplegado en Google Cloud, dentro de los límites gratuitos tanto de **Firebase Hosting** para el cliente como de **Cloud Run** para el servidor.

Objetivos de Desarrollo Sostenible

Este Trabajo de Fin de Grado contribuye a distintos Objetivos de Desarrollo Sostenible (ODS) definidos por la Organización de las Naciones Unidas (ONU). Principalmente, el ODS con el que más relacionado está es el **ODS 9: Industria, innovación e infraestructura**, ya que se trata de una aplicación de gestión de transporte terrestre que ayuda con la transformación digital del sector. Dentro de este ODS, tiene impacto en la **meta 9.4**, centrada en modernizar la infraestructura de las industrias, ya que el sistema ofrece varias funcionalidades de gestión que hoy en día aún se realizan manualmente en empresas tradicionales y con bajos recursos destinados a la transformación digital, como la planificación de cargas utilizando hojas de cálculo o la comunicación por teléfono.

Otro de los ODS con los que se alinea es el **ODS 12: Producción y consumo responsables**, concretamente con la **meta 12.5**, relacionada con la reducción de desechos. A raíz de los cambios legislativos en este sector, a partir de octubre de 2026, parte de la documentación que hasta ahora se gestionaba en papel se debe gestionar de forma digital. La aplicación tiene en cuenta esta nueva norma y permite generar y almacenar esa documentación digitalmente, eliminando el impacto climático y la generación de residuos del uso diario de papel.



Figura 1: ODS relacionados con el proyecto

Índice de contenidos

Índice de contenidos	VII
Índice de figuras	XI
Índice de tablas	XIII
Índice de listados	XV
1 Introducción	1
1.1. Contexto	1
1.2. Organización del documento	1
2 Planificación	3
2.1. Alcance	3
2.1.1. Objetivos concretos	3
2.1.2. Objetivos fuera del alcance del proyecto	4
2.1.3. Fases del proyecto	5
2.1.4. Descomposición de tareas	5
2.2. Periodos y planificación de las tareas	9
2.2.1. Dependencias entre tareas	9
2.2.2. Planificación de las tareas	10
2.2.3. Planificación de las iteraciones	10
2.2.4. Estimación de dedicación a las tareas	11
2.2.5. Hitos	12
2.3. Gestión del riesgo	12
2.4. Gestión de la calidad	14
2.5. Gestión de comunicaciones e información	14
2.5.1. Sistemas de almacenamiento	14
2.5.2. Sistemas de comunicación	15
2.6. Gestión de los interesados	15
3 Análisis de requisitos	17
3.1. Descripción del producto	17
3.1.1. Ciclo de vida de las cargas	20

3.2.	Requisitos funcionales	21
3.2.1.	Requisitos de autenticación	21
3.2.2.	Requisitos del módulo de gestión de tráfico	22
3.2.3.	Requisitos del módulo de operaciones	23
3.2.4.	Requisitos del módulo de comunicación	23
3.3.	Requisitos no funcionales	23
3.3.1.	Requisitos de usabilidad	23
3.3.2.	Requisitos de portabilidad	24
3.3.3.	Requisitos de eficiencia	24
3.3.4.	Requisitos de fiabilidad	24
3.3.5.	Requisitos de seguridad	24
3.3.6.	Requisitos de mantenibilidad	24
3.4.	Casos de uso	25
3.4.1.	Clases de usuario	25
3.4.2.	Funcionalidades del producto	25
3.5.	Especificación de los casos de uso	30
4	Selección de tecnologías	41
4.1.	Frontend - Flutter	41
4.2.	API - FastAPI	42
4.3.	Base de datos y servicios - Firebase	43
4.4.	Tecnologías de testing	43
4.5.	Despliegue - Firebase Hosting y Google Cloud Run	44
5	Diseño	45
5.1.	Diseño de la arquitectura general	45
5.1.1.	Lado del cliente (Frontend)	48
5.1.2.	Lado del servidor (Backend)	49
5.2.	Diseño de la base de datos	50
5.3.	Diseño de la seguridad	52
5.3.1.	Autenticación	52
5.3.2.	Autorización	52
5.3.3.	Otros aspectos de seguridad	52
5.4.	Diseño de la API	53
5.5.	Diseño de las interfaces de usuario multiplataforma	54
5.5.1.	Componentes principales	54
5.5.2.	Diseño responsive	54
5.5.3.	Principios de diseño	56
5.6.	Diagramas de secuencia	57
6	Implementación	63
6.1.	Configuración inicial del proyecto	63
6.1.1.	Estructura del proyecto	63
6.1.2.	Flavors de Flutter	64

6.2.	Implementación de la capa de infraestructura y datos	64
6.2.1.	Firebase Authentication	64
6.2.2.	Firestore	66
6.3.	Implementación del frontend	67
6.3.1.	Implementación del sistema MultiProvider	67
6.3.2.	Implementación responsiva de interfaces de usuario	68
6.4.	Implementación del módulo de gestión de tráfico	69
6.5.	Implementación del módulo de notificaciones	73
6.6.	Implementación del módulo de operaciones	74
6.7.	Despliegue	75
6.7.1.	Infraestructura necesaria	75
6.7.2.	Proceso de despliegue	76
7	Pruebas	77
7.1.	Análisis estático de código	77
7.2.	Pruebas implementadas	78
7.2.1.	Pruebas unitarias	78
7.2.2.	Pruebas de integración	81
7.3.	Integración Continua (CI)	82
8	Seguimiento y control	83
8.1.	Evolución del alcance	83
8.2.	Análisis de riesgos materializados	84
8.3.	Desviaciones temporales	84
9	Conclusiones	87
9.1.	Objetivos cumplidos	87
9.2.	Competencias logradas	88
9.3.	Trabajo futuro	88
10	Declaración del uso de IA generativa	91
A	Interfaces de usuario secundarias	93
B	Reglas de seguridad de Firebase	97
C	MultiProvider completo	99
D	Pipeline CI completo	101
E	Documentación técnica	105
E.1.	Configuración del entorno de desarrollo	105
E.1.1.	Configuración del Backend (FastAPI)	105
E.1.2.	Configuración del Frontend (Flutter) y Flavors	106
E.2.	Documentación de la API	107

E.3. Manual de despliegue	107
E.3.1. Despliegue del Backend (Google Cloud Run)	107
E.3.2. Despliegue del Frontend (Firebase Hosting)	108
Bibliografía	109

Índice de figuras

1.	ODS relacionados con el proyecto	v
2.1.	Diagrama EDT	6
2.2.	Diagrama de dependencias entre las tareas	9
2.3.	Diagrama de Gantt	10
3.1.	Modelo conceptual del dominio de contrato de transporte	18
3.2.	Ejemplo de carta de porte generada por la aplicación	19
3.3.	Diagrama de máquina de estados UML para el ciclo de vida de una Carga.	21
3.4.	Diagrama de casos de uso de autenticación y gestión de usuarios	27
3.5.	Diagrama de casos de uso del módulo principal	28
3.6.	Diagrama de casos de uso del usuario chófer	29
3.7.	Interfaz de dar de alta a usuario	31
3.8.	Interfaz de Realizar Planificación, numerada	34
3.9.	Interfaz de Visualizar Planificación	35
3.10.	Formulario de selección de carga.	36
3.11.	Desglose de cargas añadidas.	36
3.12.	Flujo de inserción de cargas dentro del nuevo pedido.	36
5.1.	Diagrama de la arquitectura general por capas	47
5.2.	Diagrama de componentes UML del sistema desarrollado	47
5.3.	Diagrama de clases UML para la gestión de estado usando el patrón Provider	49
5.4.	Modelo lógico de la Base de datos, mediante diagrama de clases UML	51
5.5.	Ejemplo de interfaz responsive	55
5.6.	Ejemplo de conversión de tabla a lista	56
5.7.	Diagrama de secuencia de inicio de sesión	57
5.8.	Diagrama de secuencia de registro de usuario	58
5.9.	Diagrama de secuencia de dar de alta a un usuario	59
5.10.	Diagrama de secuencia de dar de baja a un usuario	60
5.11.	Diagrama de secuencia de creación de pedido	61
5.12.	Diagrama de secuencia de generación de carta de porte	62
6.1.	Diagrama de despliegue del sistema	75
7.1.	Resumen de SonarCloud	78

7.2. Ejecución automática de jobs en GitHub Actions	82
A.1. Flujo de registro del usuario y de su empresa.	93
A.2. Interfaz para registrar un nuevo tipo de carga.	94
A.3. Interfaces de consulta y gestión de conductores y vehículos.	94
A.4. Interfaces móviles del módulo de operaciones del conductor.	95
A.5. Diálogos de confirmación para operaciones de eliminación.	95
A.6. Aviso mostrado cuando la aplicación pierde la conexión a Internet.	96

Índice de tablas

2.1. Estimación de tiempos y desglose de tareas del proyecto.	11
2.2. Hitos principales del proyecto.	12
2.3. Resumen de riesgos identificados	12
3.1. Resumen de los casos de uso especificados.	30
3.2. Caso de uso Dar de alta	30
3.3. Caso de uso Dar de baja	31
3.4. Caso de uso Asignar vehículo	32
3.5. Caso de uso Asignar carga a chófer	32
3.6. Caso de uso Ceder carga a subcontratado	33
3.7. Caso de uso Realizar Planificación	33
3.8. Caso de uso Visualizar Planificación	34
3.9. Caso de uso Insertar Pedido	35
3.10. Caso de uso Seleccionar tipo de carga	36
3.11. Caso de uso Crear tipo de carga	37
3.12. Caso de uso Generar carta de porte	37
3.13. Caso de uso Visualizar cargas asignadas	38
3.14. Caso de uso Consultar detalles de carga	38
3.15. Caso de uso Reportar incidencia	38
3.16. Caso de uso Visualizar incidencias	39
6.1. Resumen de funcionalidades implementadas en el módulo de gestión de tráfico	70
6.2. Detalle de archivos - Creación del pedido	70
6.3. Detalle de archivos - Realizar Planificación	72
6.4. Detalle de archivos - Módulo de operaciones	74
7.1. Resumen de las pruebas implementadas	78
7.2. Pruebas unitarias del frontend (Flutter/Dart)	79
7.3. Pruebas unitarias del backend (Python/FastAPI)	79
7.4. Pruebas de integración	81
8.1. Comparación de horas planificadas y reales por tarea.	85
8.2. Comparación de fechas planificadas frente a reales por hito.	86

Índice de Listados

1.	Suscripción al estado de autenticación	65
2.	Función que asigna los custom_claims al usuario encargado	65
3.	Implementación de autorización por rol	66
4.	Actualización atómica utilizando Batches de Firestore	67
5.	Implementación de MultiProvider	68
6.	Ejemplo de uso de <i>responsive breakpoints</i> en Flutter	69
7.	Ejemplo de llamada al servicio de notificaciones	74
8.	Ejemplo de dart_defines.json para configurar dependencias del frontend	76

Introducción

1.1. Contexto

El sector de transporte terrestre de mercancías es un sector vital en la cadena de suministro que, como muchos otros sectores, cada vez se digitaliza más. Sin embargo, mientras que los grandes operadores logísticos cuentan con infraestructuras de software robustas, muchas empresas pequeñas y autónomos siguen utilizando sistemas de gestión manuales con herramientas como Excel, llamadas telefónicas y documentación en papel. Recientemente, la entrada en vigor de normativas que exigen la digitalización de la documentación mercantil, como las cartas de porte [1], ha aumentado la necesidad de un software que permita gestionar el equipo y generar documentación digital de manera unificada.

En este contexto, este TFG se centra en desarrollar una primera versión de una aplicación multiplataforma de planificación de transporte, accesible desde la web y desde dispositivos móviles Android e iOS. El proyecto se caracteriza por su simplicidad y por ser capaz de dar servicio de manera gratuita, siempre que se mantenga bajo los límites de las capas gratuitas de Google. No tiene como objetivo competir contra otros sistemas de gestión de transporte (TMS, por sus siglas en inglés) más generales o complejos. Las funcionalidades que incluye se centran en el flujo diario de trabajo: la realización de planificaciones de pedidos, la comunicación de incidencias en tiempo real entre el planificador y los conductores, cierta colaboración con otros usuarios de la cadena de transporte y, por último, la generación y almacenamiento de las cartas de porte digitales, que se han mencionado antes.

1.2. Organización del documento

El documento se divide en varios capítulos donde se describen distintos aspectos del proyecto desarrollado.

1. INTRODUCCIÓN

El capítulo de [Introducción](#) presenta el contexto y la motivación del proyecto. El capítulo de [Planificación](#) detalla los objetivos concretos, el alcance y la planificación del TFG. En el capítulo de [Análisis de requisitos](#) se recogen los requisitos funcionales y no funcionales del sistema, mientras que el capítulo de [Selección de tecnologías](#) justifica las tecnologías usadas para su desarrollo. El [Diseño](#) describe las decisiones de arquitectura y diseño tomadas, y el capítulo de [Implementación](#) explica en detalle cómo se han llevado a la práctica. El capítulo de [Pruebas](#) recoge la estrategia de pruebas seguida para verificar el correcto funcionamiento del sistema. El capítulo de [Seguimiento y control](#) analiza la evolución del proyecto respecto a lo planificado inicialmente, y, por último, las [Conclusiones](#) recogen las conclusiones del trabajo, las competencias adquiridas y las líneas de trabajo futuro propuestas.

Planificación

En este capítulo se detallarán los aspectos relevantes de la planificación del TFG. Se describirán el alcance, los objetivos concretos y las fases del proyecto, la descomposición de las tareas en distintos paquetes de trabajo y otros aspectos de interés, como la gestión del tiempo y de los riesgos del proyecto. El objetivo del capítulo es tener una referencia sobre la cual ir desarrollando el trabajo y poder verificar que se están cumpliendo las necesidades del proyecto.

2.1. Alcance

El alcance del proyecto incluye el trabajo necesario para el diseño e implementación de una aplicación multiplataforma para agilizar la gestión de tráfico de transportistas profesionales, cuyo objetivo principal es facilitar la comunicación y la gestión de cargas durante la jornada laboral.

2.1.1. Objetivos concretos

El objetivo principal del proyecto es la creación de una aplicación multiplataforma, accesible desde Android, iOS y web, que facilite la gestión de tráfico de una empresa de transporte terrestre y la comunicación entre los encargados de logística y los chóferes durante su jornada laboral.

La aplicación también debe proporcionar una plataforma de comunicación eficiente y centralizada entre los encargados y los conductores, permitiendo el intercambio de información en tiempo real y mejorando la coordinación de las operaciones logísticas.

Para la consecución del objetivo principal del TFG, se han definido los siguientes objetivos concretos que deben cumplirse en el proyecto:

- Diseñar una **arquitectura adecuada y escalable** que permita incluir la aplicación, el backend, la base de datos y los servicios necesarios para el funcionamiento de la aplicación. La aplicación debe estar lista para poder añadir nuevas funcionalidades cuando sea necesario, por lo que definir una estructura escalable desde el inicio es fundamental. También tendrá que exponer una API REST para ofrecer la capacidad de integración futura con sistemas externos.
- Crear un **sistema de gestión de cargas y pedidos** que permita al gestor de tráfico de la empresa de transporte organizar los pedidos y las cargas haciendo planificaciones semanales, asignaciones de vehículos y conductores.
- Permitir que colaboradores y empresas externas que actúan en la cadena de transporte interactúen con el sistema para que puedan hacer el seguimiento y realizar actualizaciones de los pedidos.
- Implementar un sistema de notificaciones push que informe a los usuarios de los cambios que suceden en la planificación y de las asignaciones que ocurren.
- La aplicación debe generar documentación de control (cartas de porte) que cumpla con la ley de contratos de transporte de mercancías, incluyendo el contenido que exige la nueva normativa y permitiendo su almacenamiento y consulta en formato digital. [2]
- Crear una **interfaz de usuario sencilla de usar** que pueda utilizarse con escasos conocimientos técnicos en informática y con una curva de aprendizaje pequeña.
- La aplicación desarrollada deberá tener la calidad suficiente para que sea un producto listo para el despliegue. La aplicación web deberá desplegarse.
- Realizar **pruebas de funcionamiento** en la aplicación para verificar que las distintas funcionalidades implementadas cumplen con lo esperado.

2.1.2. Objetivos fuera del alcance del proyecto

Hay ciertos objetivos que quedan fuera del alcance debido a su complejidad o al tiempo necesario para realizarlos.

- **Facturación y sistema de pagos:** Queda fuera del alcance ya que el sistema se centra únicamente en la planificación de transporte, no en la gestión económica.
- **Despliegue de las aplicaciones móviles en Play Store y App Store:** Aunque la aplicación web sí se desplegará, desplegar las aplicaciones móviles en las tiendas oficiales de Google y Apple queda fuera del alcance debido al coste económico y el tiempo que supondría pasar los procesos de revisión obligatorios.
- **Optimización de rutas y asignaciones:** La planificación y las asignaciones se realizarán manualmente por los usuarios de la aplicación. Realizar planificaciones optimizadas requeriría solucionar el problema **Vehicle Routing Problem (VRP)**, un problema complejo de combinatoria cuya resolución queda fuera del alcance del proyecto.

2.1.3. Fases del proyecto

El proyecto cuenta con distintas fases que ayudarán al seguimiento de la consecución de los objetivos. Siendo la primera vez que se desarrolla un proyecto de estas características y la primera vez que se utilizan varias de las tecnologías del proyecto, se seguirá un ciclo de vida iterativo. En cada una de las iteraciones se analizará la viabilidad de las funcionalidades a implementar, teniendo en cuenta las limitaciones de tiempo y la complejidad de estas.

Las fases con las que cuenta cada iteración del TFG son:

- **Fase de análisis de requisitos:** En esta primera fase se analizarán los requisitos funcionales de los casos de uso y los requisitos no funcionales de la aplicación, tales como seguridad, eficiencia, etc. Es una de las fases más importantes del proyecto, ya que una incorrecta captura de requisitos afectará negativamente a la calidad de las funcionalidades y de la aplicación móvil. Esto generaría retrabajo al tener que repetir la captura de requisitos y consumiría tiempo de las siguientes fases.
- **Fase de diseño:** En esta fase se diseñará la arquitectura principal de la aplicación, teniendo en cuenta las funcionalidades a implementar y las arquitecturas utilizadas en la industria para este tipo de aplicaciones móviles. Ya que se va a utilizar un ciclo de vida iterativo, es fundamental que la arquitectura sea escalable y permita implementar nuevos casos de uso sin afectar a las funcionalidades ya implementadas.
- **Fase de implementación:** En esta fase se desarrollarán los componentes del sistema definidos durante la fase de diseño. La implementación se realizará de manera incremental, siguiendo el ciclo de vida iterativo planteado para el proyecto, permitiendo desarrollar funcionalidades de forma progresiva y evaluar su funcionamiento antes de continuar con nuevas iteraciones.
- **Fase de pruebas:** La fase de pruebas es la fase en la que se probarán los componentes desarrollados hasta esa iteración. Es importante que las nuevas funcionalidades que se van desarrollando no rompan lo implementado anteriormente en iteraciones previas. Por ello, para las pruebas se utilizará un pipeline de integración continua (CI/CD) en GitHub Actions que permitirá ejecutar pruebas automáticas cada vez que se introduzcan cambios en el código del proyecto.

Para conocer en detalle lo que se realizará en cada iteración, véase la [sección 2.2.3: Planificación de las iteraciones](#).

2.1.4. Descomposición de tareas

La estructura de descomposición de trabajo (EDT) recoge los apartados en los que se descompone el proyecto. En la Figura 2.1 se pueden apreciar estos apartados: Investigación, Aplicación, Trabajo Académico, Gestión.

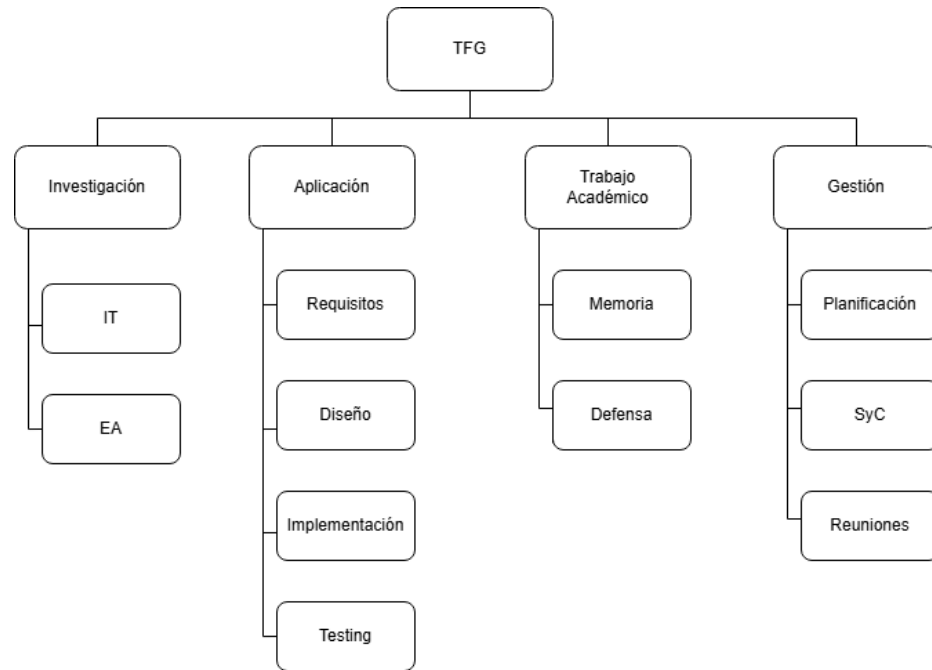


Figura 2.1: Diagrama EDT

2.1.4.1. Apartado de Investigación

Este apartado incluye los paquetes y tareas necesarias para el estudio inicial de la aplicación a desarrollar y las tecnologías que se utilizarán. Se utilizarán tecnologías que no se conocen en profundidad, por lo que hará falta una inversión de tiempo en aprender sobre ellas y comprender sus ventajas y limitaciones. También se analizarán aplicaciones existentes en el mercado para identificar posibles mejoras que se puedan añadir a la aplicación.

El apartado se descompone en los siguientes subpaquetes de trabajo:

- **Investigación de tecnologías (IT):** Incluye las tareas necesarias para comprender las tecnologías de las que se hará uso en la aplicación.
 - **IT.T1:** Estudio de tecnologías frontend multiplataforma para aplicaciones móviles, tales como Flutter o React Native.
 - **IT.T2:** Estudio de las herramientas de Firebase.
 - **IT.T3:** Estudio de Firestore.
 - **IT.T4:** Estudio de Android Studio y comparación con Visual Studio Code.
- **Estudio de aplicaciones (EA):** Incluye las tareas necesarias para el análisis de aplicaciones sobre gestión de tráfico para transportistas existentes en el mercado.
 - **EA.T1:** Búsqueda y análisis de aplicaciones existentes en el sector del transporte.

- **EA.T2:** Análisis y decisión de las funcionalidades concretas a incluir en la aplicación.

2.1.4.2. Apartado de Aplicación

Este apartado incluye los paquetes y tareas necesarias para desarrollar la aplicación móvil en su totalidad. Se incluyen los paquetes de trabajo necesarios para el análisis de requisitos, el diseño, la implementación y el testing de la aplicación. Cada uno de estos paquetes de trabajo incluye sus tareas específicas.

- **Análisis de Requisitos (AR):** Incluye las tareas necesarias para realizar la captura de los requisitos de la aplicación completa.
 - **AR.T1:** Análisis de requisitos funcionales de los casos de uso del encargado y del chófer.
 - **AR.T2:** Análisis de requisitos no funcionales
 - **AR.T3:** Análisis de requisitos del sistema
- **Diseño (D):** Incluye las tareas necesarias para diseñar la arquitectura de la aplicación y conseguir un diseño escalable a largo plazo. También se diseñarán los diagramas UML correspondientes a cada caso de uso.
 - **D.T1:** Diseño de la arquitectura general.
 - **D.T2:** Diseño de las interfaces de usuario.
 - **D.T3:** Diseño de la API: definición de endpoints, parámetros y gestión de la autenticación de la API.
 - **D.T4:** Diagramas UML: Se definirán los diagramas de casos de uso y diagramas de secuencia en UML para especificar el comportamiento del sistema. Incluye los diagramas UML del encargado (**D.T4.1**) y de los usuarios secundarios (**D.T4.2**).
 - **D.T5:** Diseño del modelo de datos: Creación del diagrama MER y definición de las colecciones en Firestore.
- **Implementación (I):** Incluye las tareas para implementar los distintos apartados y funcionalidades de la aplicación. La aplicación se implementará siguiendo el diseño definido en el apartado anterior.
 - **I.T1:** Configuración inicial del entorno frontend y de los servicios de Firebase (Firestore, Auth, Firebase Cloud Messaging).
 - **I.T2:** Creación de colecciones necesarias en Firestore e implementación de la comunicación API-Firebase.
 - **I.T3:** Desarrollo del backend en FastAPI e implementación de la lógica de negocio. Incluye el sistema CRUD base y la lógica más avanzada.

- **I.T4:** Implementar la funcionalidad de cartas de porte.
 - **I.T5:** Implementar el sistema de notificaciones.
 - **I.T6:** Implementación de las interfaces de usuario siguiendo el diseño definido.
 - **I.T7:** Implementación del frontend para la gestión del estado y el consumo de la API.
 - **I.T8:** Despliegue y redacción de documentación técnica: archivo README.md, documentación de la API implementada y creación del manual de despliegue.
- **Testing (T):** Incluye las tareas para probar los distintos apartados y funcionalidades de la aplicación. La aplicación se probará a medida que se implementen los distintos módulos, con el objetivo de tener margen de reacción en caso de errores.
 - **T.T1:** Configuración del pipeline de CI en GitHub Actions.
 - **T.T2:** Implementación de los tests unitarios y los tests de integración.

2.1.4.3. Apartado de Trabajo Académico

Este apartado recoge los paquetes de trabajo relacionados con la elaboración del trabajo académico del TFG.

- **Memoria:** Elaboración de la memoria del proyecto durante la realización del mismo.
- **Defensa:** Preparación de la presentación y realización de la defensa tras completar el TFG.

2.1.4.4. Apartado de Gestión

Este apartado recoge los paquetes de trabajo relacionados con la gestión del proyecto y la verificación del cumplimiento de los objetivos.

- **Planificación:** Elaboración de una planificación en la que se recogerán el alcance, los objetivos, riesgos y la dedicación estimada de cada tarea.
- **Seguimiento y control (SyC):** Seguimiento y control del cumplimiento de los objetivos definidos en la planificación inicial y, en caso necesario, realización de replanificaciones.
- **Reuniones:** Reuniones con el tutor para evaluar el progreso del proyecto. Incluye la reunión inicial donde se fijan los objetivos concretos del TFG. Se realizarán aproximadamente cada dos semanas.

2.2. Periodos y planificación de las tareas

En esta sección se detallan los aspectos relacionados con la planificación de las tareas comentadas anteriormente. En primer lugar, se detectan las dependencias que hay entre ellas, se planifican mediante un diagrama de Gantt y se definen los principales hitos del proyecto. Por último, se realiza una estimación del tiempo que se va a dedicar a cada tarea.

2.2.1. Dependencias entre tareas

Para garantizar el flujo de trabajo y establecer el orden entre las tareas, se han identificado las dependencias críticas entre las tareas del proyecto.

El proyecto inicia con la planificación (P). Tras la planificación, se estudiarán las tecnologías necesarias para el proyecto (IT) y se analizarán las aplicaciones existentes en el mercado (EA). Después, se empezará con las tres iteraciones planificadas. Cada una de estas iteraciones tiene su fase de requisitos (AR), diseño (D), implementación (I) y testing (T), y no se podrá comenzar una fase hasta que se haya terminado la anterior. Una vez finalizada una iteración, antes de empezar con la siguiente, habrá que hacer una sesión de seguimiento para determinar si hace falta replanificar la siguiente iteración.

En la Figura 2.2 se puede observar la dependencia entre estas tareas.

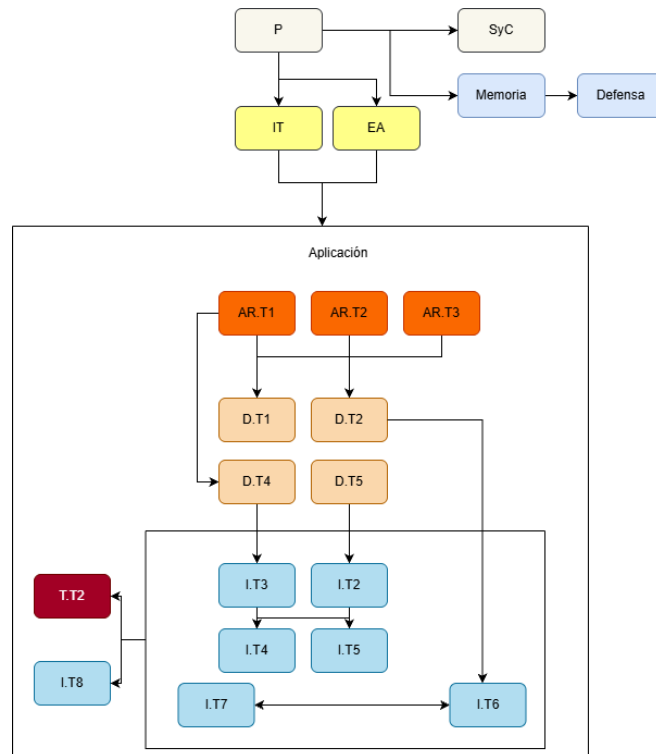


Figura 2.2: Diagrama de dependencias entre las tareas

2.2.2. Planificación de las tareas

En la Figura 2.3 se puede observar el diagrama de Gantt del proyecto. En él se puede ver en qué paquetes se trabajará a lo largo de las semanas que dura el proyecto. Tras realizar la planificación la primera semana de marzo, el inicio del proyecto se dará la segunda semana de marzo y acabará el día de la defensa (primera o segunda semana de julio).

Se puede observar el ciclo de vida iterativo que se seguirá para la realización de la aplicación. En cada iteración, se realizará el análisis de requisitos, el diseño, la implementación y las pruebas de los casos de uso y funcionalidades desarrolladas.

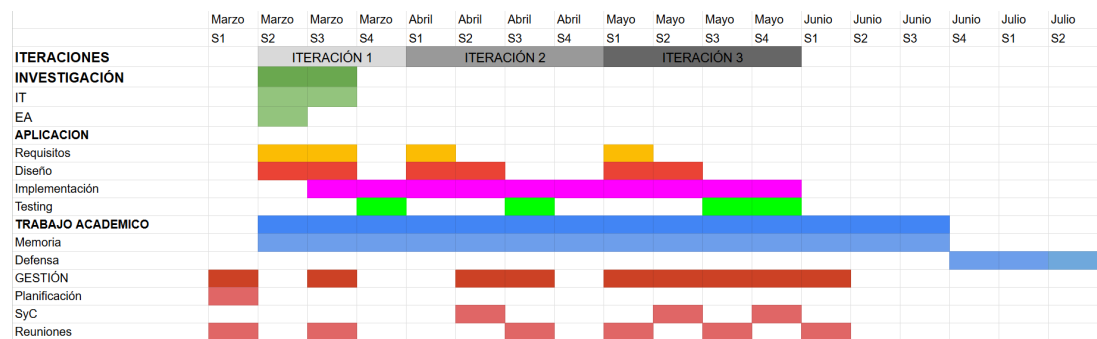


Figura 2.3: Diagrama de Gantt

2.2.3. Planificación de las iteraciones

El proyecto se divide en tres iteraciones principales. En cada una se capturarán los requisitos, se realizarán los diagramas UML y se realizará la implementación de las funcionalidades que corresponden a esa iteración.

Primera iteración:

En esta iteración se llevará a cabo la investigación de las tecnologías necesarias y se analizarán las aplicaciones existentes para entender qué funcionalidades debería incluir mi aplicación y cómo implementarlas. En cuanto a la aplicación, se configurará el entorno de trabajo y se desarrollarán los módulos base de los que dependerán las siguientes iteraciones.

Segunda iteración:

Esta es la iteración principal del proyecto. En esta fase, se analizarán, diseñarán y desarrollarán los casos de uso principales de la aplicación. Al finalizar la iteración, los casos de uso del módulo principal correspondientes a los usuarios «Encargado», «Subcontratado» y «Cargador» (véase sección 3.4.1: *tipos de usuario*) deberían estar completados y la calidad del código debería ser razonablemente buena.

Tercera iteración:

En esta iteración, se desarrollarán los casos de uso del usuario «Chófer» y la funcionalidad de

visualizar en tiempo real la ubicación de los vehículos. Además, se harán las pruebas finales, se documentará el proyecto y se desplegará la aplicación web.

2.2.4. Estimación de dedicación a las tareas

En la Tabla 2.1 se puede ver la estimación de dedicación a cada una de las tareas definidas.

Apartado	Paquete de Trabajo	Tarea	Dedicación (h)
Investigación	Investigación Tecnologías (IT)	IT.T1	2
		IT.T2	2
		IT.T3	0.5
		IT.T4	0.5
	Estudio de aplicaciones (EA)	EA.T1	1
		EA.T2	2
	Subtotal Investigación		8
Aplicación	Análisis de Requisitos (AR)	AR.T1	10
		AR.T2	5
		AR.T3	2
	Diseño (D)	D.T1	5
		D.T2	5
		D.T3	2
		D.T4.1	15
		D.T4.2	10
		D.T5	3
	Implementación (I)	I.T1	15
		I.T2	4
		I.T3	55
		I.T4	10
		I.T5	5
		I.T6	20
		I.T7	15
		I.T8	10
	Testing (T)	T.T1	5
		T.T2	10
	Subtotal Aplicación		206
Trabajo Académico	Memoria		60
	Defensa		10
	Subtotal Trabajo Académico		70
Gestión	Planificación		10
	Seguimiento y Control (SyC)		5
	Reuniones		5
	Subtotal Gestión		20
	TOTAL		304

Tabla 2.1: Estimación de tiempos y desglose de tareas del proyecto.

2.2.5. Hitos

En la Tabla 2.2 se encuentran disponibles los hitos principales de este proyecto. Tras cada iteración, se hará un seguimiento y control para comprobar si es necesario replanificar las siguientes iteraciones.

Hito	Fecha
Reunión de inicio	5/03/2026
Inicio del proyecto	09/03/2026
Configuración del entorno	11/03/2026
Fin de iteración 1	31/03/2026
SyC 1	08/04/2026
Fin de iteración 2	24/04/2026
SyC 2	27/04/2026
Fin de iteración 3	29/05/2026
SyC 3	01/06/2026
Entrega de memoria	21/06/2026
Defensa	29/06/2026 - 15/07/2026

Tabla 2.2: Hitos principales del proyecto.

2.3. Gestión del riesgo

A continuación, se describen los riesgos que podrían ocurrir durante el desarrollo del proyecto. Para cada uno, se detallan la descripción, la probabilidad de que ocurra, el impacto y el plan de acción en caso de que ocurran. La Tabla 2.3 resume cada riesgo indicando su identificador, título, probabilidad e impacto.

Tabla 2.3: Resumen de riesgos identificados

ID	Título	Probabilidad	Impacto
R1	Tecnologías desconocidas	Baja	Alto
R2	Compaginación de asignaturas	Media	Medio
R3	Planificación incorrecta	Baja	Medio
R4	Análisis de requisitos incorrecto	Baja	Alto
R5	Riesgos en los servicios externos	Baja	Alto

R1 - Tecnologías desconocidas

- **Descripción:** Varias tecnologías que se usarán durante el proyecto no se han utilizado anteriormente en la carrera. No se conoce bien la curva de aprendizaje de estas tecnologías, lo que podría requerir dedicar más tiempo del esperado para aprenderlas.

- **Prevención:** Se escogerán tecnologías que cuenten con documentación extensa y que sean similares a las tecnologías utilizadas anteriormente en la carrera.
- **Plan de acción:** Si el aprendizaje requiere más tiempo del previsto, se reducirá temporalmente el alcance de funcionalidades no esenciales y se replantearán las iteraciones para mantener los hitos principales.

R2 - Compaginación de asignaturas

- **Descripción:** El TFG se llevará a cabo a lo largo del segundo cuatrimestre, junto con otras dos asignaturas. Una de las asignaturas requiere la realización de un proyecto extenso, lo que podría derivar en una saturación de tareas.
- **Prevención:** Realizar una planificación semanal con bloques de trabajo fijos para el TFG, mientras se mantienen al día el resto de las asignaturas.
- **Plan de acción:** En caso de que la carga de trabajo sea demasiado grande y vaya a afectar a la calidad del TFG, habrá que reorganizar las tareas, posponiendo las tareas secundarias y priorizando la calidad en los entregables principales del TFG.

R3 - Planificación incorrecta

- **Descripción:** Es la primera vez que se lleva a cabo una planificación de un proyecto tan extenso. Una estimación inicial inexacta puede provocar desviaciones en los tiempos y el esfuerzo.
- **Prevención:** Además de la planificación inicial, realizar planificaciones semanales y hacer un seguimiento y control estricto del proyecto.
- **Plan de acción:** Replanificar el proyecto para seguir cumpliendo los objetivos principales del TFG.

R4 - Análisis de requisitos incorrecto

- **Descripción:** Una captura incompleta o ambigua de requisitos puede generar implementaciones que no resuelvan las necesidades reales del encargado y del chófer.
- **Prevención:** Validar los requisitos con el tutor mediante revisiones tempranas y mantener los casos de uso claros antes de implementar. También se utilizará un ciclo de vida iterativo que permita validar y corregir los requisitos con suficiente antelación.
- **Plan de acción:** Corregir los requisitos priorizando los más críticos y planificar refactorizaciones pequeñas y controladas para incluir los nuevos requisitos.

R5 - Riesgos en los servicios externos

- **Descripción:** Los servicios en la nube pueden cambiar sus condiciones de uso. El riesgo más importante aquí es que Firebase deje de tener un plan gratuito o que sea muy limitado.
- **Prevención:** Diseñar una buena arquitectura que facilite los cambios y las refactorizaciones en caso de que se tengan que realizar.
- **Plan de acción:** Buscar servicios alternativos que reemplacen los servicios utilizados de Firebase.

2.4. Gestión de la calidad

Para asegurar que el TFG completo cumpla con los requerimientos de calidad necesarios, se garantizará que el proceso y el producto cumplan con los siguientes estándares y requisitos:

- **Calidad en los requisitos:** Se debe mantener la trazabilidad de los requisitos y comprobar que cada requisito ha sido implementado y probado.
- **Calidad en el diseño:** Se debe tener un diagrama UML para cada caso de uso que refleje correctamente la interacción de los usuarios con el sistema.
- **Calidad en la implementación:** El proyecto se implementará siguiendo las buenas prácticas de ingeniería de software y respetando los requisitos y el diseño definidos.
- **Calidad del producto:** El producto debe funcionar de manera fluida y sin errores que afecten negativamente a la experiencia del usuario.
- **Calidad de testing:** Todos los casos de uso principales se deben probar y los errores principales se deberán gestionar de forma correcta.
- **Calidad de la documentación:** La documentación debe tener una calidad de nivel académico tanto en el contenido como en el formato. La memoria no debería contener ningún error ortográfico ni incoherencias en el contenido.

2.5. Gestión de comunicaciones e información

2.5.1. Sistemas de almacenamiento

El almacenamiento del código fuente de la aplicación y de la documentación del TFG se llevará a cabo utilizando las siguientes herramientas:

- **Google Drive:** Se creará una carpeta en la nube que contendrá toda la información y los componentes necesarios para la realización de la memoria, tales como tablas, diagramas, gráficos, etc. Las actas de reunión con el director también se guardarán en esta carpeta. Habrá una copia de seguridad local que se mantendrá sincronizada en todo

momento para prevenir la pérdida de información ante posibles fallos de conectividad y/o problemas en el acceso a la cuenta.

- **Overleaf:** La memoria del proyecto se desarrollará en la plataforma Overleaf. Esta plataforma permite visualizar las distintas versiones del documento y mantener una buena trazabilidad de los cambios realizados.
- **GitHub:** El código fuente del proyecto y la documentación técnica se guardarán en un repositorio de GitHub¹.

2.5.2. Sistemas de comunicación

La comunicación entre el alumno y el director es importante para mantener el seguimiento del cumplimiento de los objetivos del TFG. Para ello, se plantea el siguiente sistema de comunicación:

- **Correo electrónico:** Se utilizará para resolver dudas sencillas y para concretar reuniones con el tutor del TFG.
- **Reuniones:** Las reuniones serán presenciales en el despacho del director y se llevarán a cabo aproximadamente cada dos semanas. Estas reuniones servirán para resolver dudas más complejas y verificar que se cumplen los objetivos según lo planificado.

2.6. Gestión de los interesados

Hay cuatro interesados principales en el proyecto:

- **Tutor del proyecto:** Felipe Ibáñez. Hará un seguimiento continuo del proyecto para verificar que se cumplan todos los objetivos.
- **Cliente:** Felipe Ibáñez. Simulará ser el cliente, dando *feedback* y proponiendo mejoras a las funcionalidades implementadas.
- **Alumno:** Haritz Gómez. Se asegurará de cumplir con todos los objetivos del TFG.
- **Tribunal:** Se encargará de evaluar la memoria y la defensa pública.

¹https://github.com/haritz99/TFG_Gestion_Transporte

CAPÍTULO 3

Análisis de requisitos

En este capítulo se realizará el análisis de requisitos de la aplicación a desarrollar. Se analizarán los distintos tipos de usuario de la aplicación y las funcionalidades de cada rol dentro del sistema, se hará la captura de requisitos funcionales de cada módulo y se analizarán los requisitos no funcionales del sistema siguiendo el modelo de calidad definido por la norma ISO 25010 [3].

El capítulo toma como referencia las convenciones estándar IEEE para la especificación de requisitos software. A lo largo del capítulo se describen los requisitos funcionales y no funcionales de la aplicación, así como los casos de uso y su correspondiente especificación.

3.1. Descripción del producto

La aplicación es un producto diseñado e implementado para facilitar y optimizar la gestión de los pedidos y la colaboración entre los distintos usuarios de la cadena de transporte, además de la comunicación entre los encargados y el equipo de chóferes. Para ayudar al entendimiento del dominio y del vocabulario que se utilizará en el resto del documento, en la Figura 3.1 se presenta un diagrama conceptual para mostrar el dominio de la aplicación. El diagrama no incluye todos los detalles de implementación ni la estructura de la base de datos, esto se detallará en la [sección de Diseño](#).

3. ANÁLISIS DE REQUISITOS

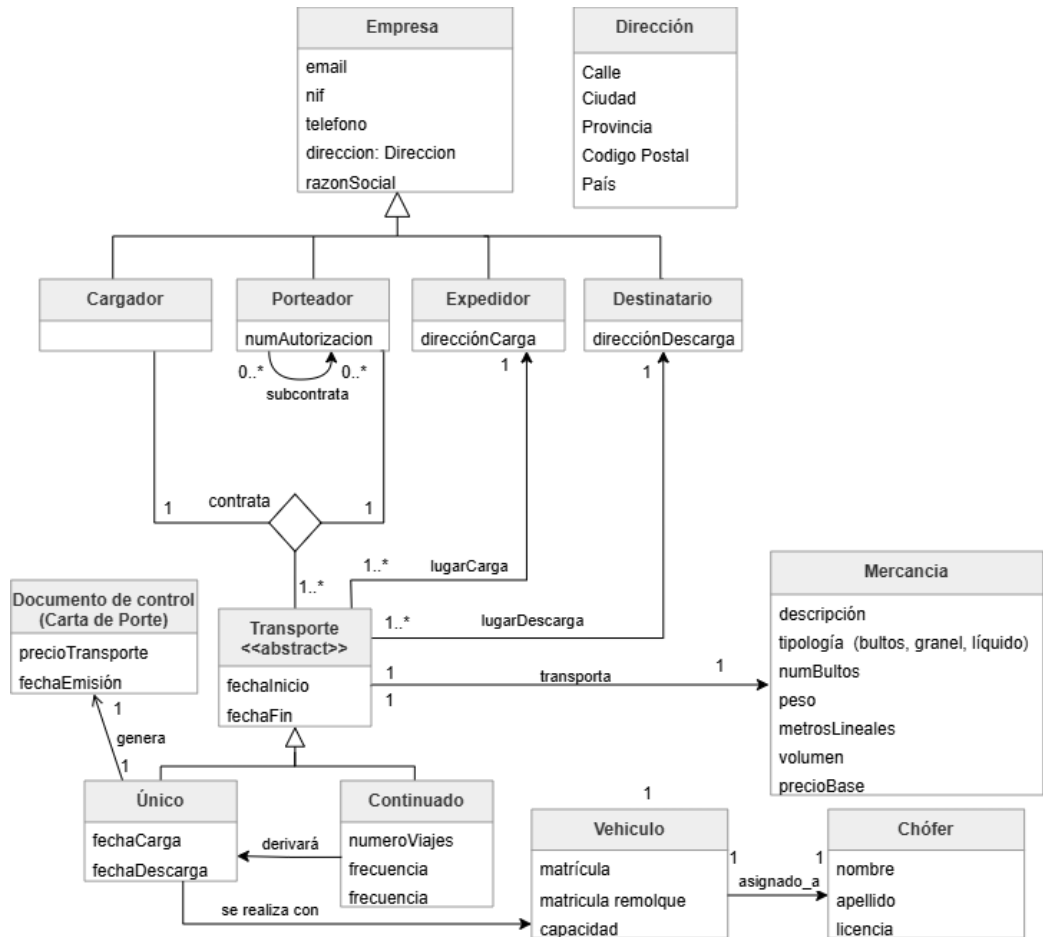


Figura 3.1: Modelo conceptual del dominio de contrato de transporte

Para entender el dominio también se recomienda acceder a la Ley 15/2009 [4], que regula el contrato de transporte terrestre de mercancías en España. Un elemento de gran importancia del dominio es la **carta de porte**, este es el documento mercantil que formaliza el contrato de transporte terrestre de mercancías entre el cargador, el transportista y el destinatario. Recoge los datos esenciales del envío: las partes implicadas, el origen y destino y los aspectos mercantiles y económicos del transporte. En la Figura 3.2 se puede observar un ejemplo de una carta de porte generada por la aplicación (los datos son ilustrativos).

A diferencia del documento tradicional en papel, la carta de porte digital debe incorporar un **código QR** que permita acceder al documento original en la nube, por lo que cada carta de porte debe contar con una URL única, tal y como se exige en la normativa vigente [2]. La ley no exige que los documentos tengan que ir firmados digitalmente, pero el documento incluye un apartado destinado a las firmas, por si la empresa quiere firmar el documento. Los requisitos concretos del contenido de las cartas de porte se detallan más tarde en los requisitos de la sección 3.2.2.

3.1. Descripción del producto

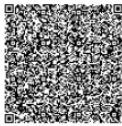
CARTA DE PORTE DIGITAL		
Nº DOCUMENTO CRG-085	FECHA DE EMISIÓN 18/06/2026 11:21	REFERENCIA PEDIDO PED-023
		
1. SUJETOS INTERVINIENTES		
CARGADOR / EXPEDIDOR Haritz Cargador, S.L.		NIF / CIF CARGADOR B12345678
DIRECCIÓN DEL CARGADOR Av. Navarra, 4, Beasain (Gipuzkoa)		
DESTINATARIO Empresa Destinataria, S.L.		NIF / CIF DESTINATARIO J5434567
DIRECCIÓN DESTINATARIO Calle, 24504 Ciudad (Gipuzkoa)		
2. OPERADORES DE TRANSPORTE DEL CONTRATO		
PORTEADOR CONTRACTUAL Logística del Sur, S.L.		NIF PORTEADOR CONTRACTUAL A12345678
DIRECCIÓN PORTEADOR CONTRACTUAL Avenida de la Libertad, 15, 20000 Donostia (Gipuzkoa)		Nº AUTORIZACIÓN TRANSPORTE MDP-28-0012345-6
TRANSPORTISTA EFECTIVO (RAZÓN SOCIAL) Mismo que Porteador Contractual		NIF / CIF TRANSPORTISTA EFECTIVO —
DIRECCIÓN TRANSPORTISTA EFECTIVO —		Nº AUTORIZACIÓN TRANSPORTISTA EFECTIVO —
3. DATOS DE LA RUTA, FECHAS Y VEHÍCULO		
PUNTO DE CARGA Paseo Ubarburu, 20014 Donostia / San Sebastián (Euskadi)		FECHA / HORA PREVISTA CARGA 18/06/2026 11:00
PUNTO DE ENTREGA Gran vía Don Diego López de Haro / On Diego Lopez Haroko kale nagusia 45, 48011 Bilbao (Euskadi)		FECHA / HORA PREVISTA ENTREGA 18/06/2026 16:06
CONDUCTOR ASIGNADO Ana Ruiz	MATRÍCULA TRACTORA 5555ABC	MATRÍCULA SEMIRREMOLQUE 1234ANC
4. ESPECIFICACIÓN DE LA MERCANCÍA		
NATURALEZA DE LOS BIENES Automoción / Estructuras metálicas		TIPO DE EMBALAJE —
NÚMERO DE BULTOS 6	PESO BRUTO (KG) 480.0	VOLUMEN / DIMENSIONES ESTIMADAS 1.2x0.8x1.7 m
5. CONDICIONES ECONÓMICAS DEL CONTRATO		
PRECIO DEL TRANSPORTE (BASE IMPONIBLE) 45.0 €		TOTAL NETO DERIVADO —

Figura 3.2: Ejemplo de carta de porte generada por la aplicación

Desde una perspectiva funcional, existen varios módulos en los que se puede clasificar el producto:

- **Módulo de gestión de tráfico/logística**

Este módulo es el núcleo del sistema y está pensado para que sea gestionado exclusivamente por el **perfil del encargado de tráfico**. Desde aquí se organiza el trabajo diario de los chóferes del sistema: se registran los camiones y conductores disponibles, se anotan los pedidos y se decide quién llevará cada carga. Además, la funcionalidad de generar las cartas de porte con los datos de los miembros de la cadena de transporte pertenece a este módulo.

Cuando se habla de **pedido**, se refiere a una petición (normalmente semanal) de transporte que hace el cargador a la empresa de transporte, durante el acuerdo de transporte continuado (mencionado en la Figura 3.1), el cargador irá realizando varios pedidos.

Del mismo modo, un pedido tendrá varios transportes únicos o **cargas**, que son la unidad mínima de trabajo, y es lo que se acaba planificando y asignando a un vehículo y conductor.

Será accesible a través de una aplicación web y también a través del dispositivo móvil del encargado, por lo que todas las interfaces deben ser responsivas.

■ Módulo de operaciones

Este módulo está diseñado **para el chófer**. Su función es mostrar al conductor su hoja de ruta diaria, junto con los detalles del transporte que tenga que realizar. El módulo será accesible desde el dispositivo móvil del conductor de manera simple y estará sincronizado con el módulo de gestión de tráfico.

■ Módulo de comunicación y notificaciones

Es el canal de conexión en tiempo real entre los distintos usuarios. Permite que el chófer informe de cualquier imprevisto (atascos, averías, retrasos) a través de las incidencias y que el encargado reciba la información en su panel de control para tomar decisiones. Otro mecanismo de comunicación de la aplicación son las **notificaciones push**, que garantizan que la información importante llegue de manera inmediata a los dispositivos de los usuarios incluso cuando la aplicación no está abierta en primer plano.

- **Módulo de usuarios y administración** Este módulo es el encargado de dar los privilegios necesarios que necesita cada usuario para interactuar con su módulo correspondiente. Se ocupa de gestionar el registro de nuevos perfiles, la seguridad de las contraseñas y la autenticación y autorización en el sistema.

3.1.1. Ciclo de vida de las cargas

Cada una de las cargas tiene un ciclo de vida independiente que pasa por distintos estados. Las acciones que se pueden realizar sobre la carga (como generar su carta de porte o replanificar la fecha) varían dependiendo del estado en el que está cada carga. La Figura 3.3 muestra el ciclo de vida de una carga y las restricciones de cada estado.

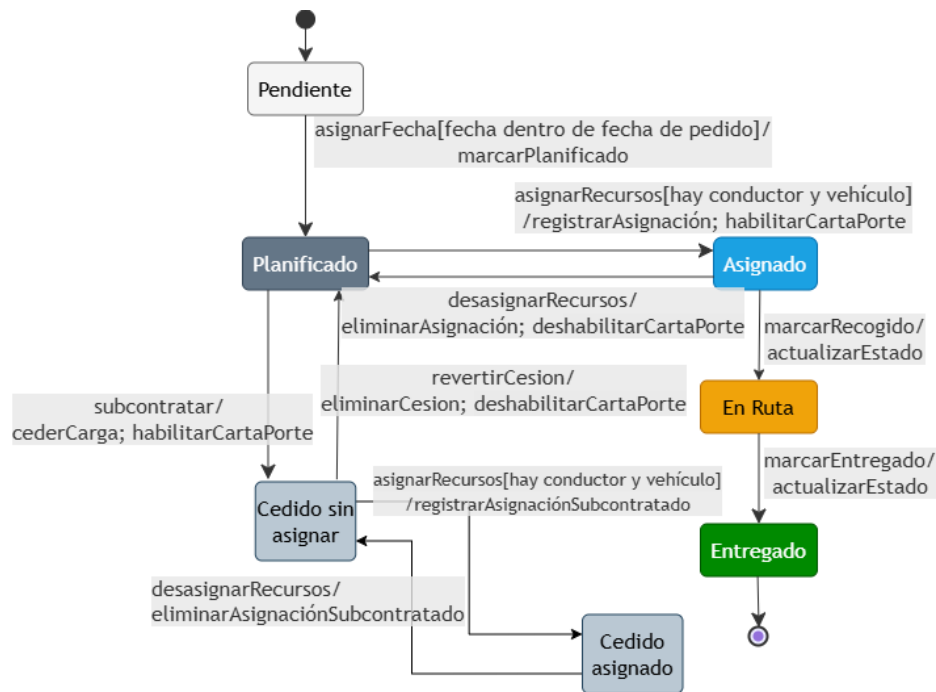


Figura 3.3: Diagrama de máquina de estados UML para el ciclo de vida de una Carga.

3.2. Requisitos funcionales

A continuación, se detallan los requisitos funcionales con los que debe cumplir la aplicación. Estos requisitos se diferencian por módulos para tener una mayor trazabilidad y poder verificarlos en el apartado de pruebas. Los requisitos funcionales se identifican mediante la nomenclatura **REQ-[MOD]-xx**, donde **[MOD]** indica el módulo al que pertenece el requisito.

3.2.1. Requisitos de autenticación

- **REQ-AUT-01:** El sistema implementará un sistema de gestión de identidades basado en Firebase Auth, utilizando tokens de acceso JWT para validar la identidad del usuario en cada petición. Este token se usará para validar las llamadas a la API siguiendo el esquema Bearer Token.
- **REQ-AUT-02:** Los permisos de cada usuario se deben limitar a realizar operaciones dentro de su empresa. Los usuarios no podrán acceder a recursos de otras empresas que utilicen la aplicación.
- **REQ-AUT-03:** En cada llamada a la API, se debe verificar que el token de autenticación no ha sido alterado, esto se hará comprobando la firma criptográfica y la vigencia del token.

- **REQ-AUT-04:** En cada llamada a la API, se debe verificar que el usuario tiene los permisos para realizar la acción que se quiere ejecutar, validando el usuario según su rol dentro del sistema.
- **REQ-AUT-05:** Las peticiones al backend se deben realizar siguiendo el protocolo HTTPS.
- **REQ-AUT-06:** El usuario podrá cerrar su sesión y el sistema deberá borrar los datos locales, revocar el acceso al servidor e invalidar la navegación hacia atrás.

3.2.2. Requisitos del módulo de gestión de tráfico

- **REQ-GT-01:** El sistema debe permitir al encargado realizar la gestión de las flotas, permitiendo dar de alta, editar y dar de baja vehículos y conductores.
- **REQ-GT-02:** El sistema debe permitir el registro de pedidos, vinculándolos a un cargador, fechas de compromiso y un conjunto de una o más cargas.
- **REQ-GT-03:** El sistema debe validar la disponibilidad horaria de los conductores y vehículos para poder asignarlos a una carga. También deberá validar que el peso de la carga no supere la capacidad del camión.
- **REQ-GT-04:** El sistema debe permitir dar de baja a cargadores y subcontratados que estaban dados de alta.
- **REQ-GT-05:** El sistema permitirá al encargado realizar una planificación inicial semanal e ir cambiándola a medida que avanza la semana y surgen cambios en los pedidos y las asignaciones de las cargas. En caso de que queden cargas por asignar de la semana actual para la siguiente, el sistema debe indicarlo y permitir hacer la asignación pendiente de la semana anterior en la semana actual.
- **REQ-GT-06:** El sistema debe mostrar un panel de control con el estado en tiempo real de los pedidos, junto con las incidencias abiertas que pueda haber en los distintos pedidos.
- **REQ-GT-07:** El sistema debe permitir buscar las cargas planificadas por fecha para facilitar la planificación del encargado. Todos los detalles relevantes deberán poder verse.
- **REQ-GT-08:** El sistema debe permitir generar una carta de porte en formato PDF con todos los datos reales de la carga y legalmente requeridos para una carta de porte digital. Debe incluir los datos del destinatario, cargador, empresa de transporte y en caso de que la haya, los de la empresa subcontratada. [5]
- **REQ-GT-09:** Las cartas de porte se deberán conservar en la nube durante un periodo mínimo de un año, que es lo legalmente requerido. Además, el documento debe tener un código QR que apunte al documento original en la nube. Además, contará con un apartado de firmas de las partes intervinientes (cargador, porteador y destinatario).

3.2.3. Requisitos del módulo de operaciones

- **REQ-OP-01:** El chófer podrá ver las tareas que tiene asignadas a lo largo de la semana y conocer los detalles de cada una de sus cargas. El sistema debe priorizar visualmente la entrega cronológica más cercana para facilitar la toma de decisiones.
- **REQ-OP-02:** El sistema debe permitir reportar una incidencia sobre la carga actual.
- **REQ-OP-03:** El sistema debe permitir visualizar las incidencias registradas para sus cargas.
- **REQ-OP-04:** El sistema permitirá al chófer redirigirse a Google Maps para visualizar la ruta con el origen y destino de la carga.

3.2.4. Requisitos del módulo de comunicación

- **REQ-COM-01:** El sistema debe enviar una notificación push al conductor asignado cuando se le asocie una nueva carga.
- **REQ-COM-02:** El sistema debe enviar una notificación push al cargador y al subcontratado cuando se genere la carta de porte de una de sus cargas.
- **REQ-COM-03:** El sistema debe enviar un correo electrónico al usuario cuando este sea registrado en el sistema, incluyendo sus credenciales temporales de acceso y un enlace para establecer una contraseña propia.

3.3. Requisitos no funcionales

El análisis de requisitos no funcionales se ha llevado a cabo siguiendo el modelo de calidad que definen las [normas ISO 25010](#). Debido al alcance definido en el contexto del TFG y la falta de usuarios reales con los que se pueda verificar el cumplimiento de los requisitos, no se han incluido todos los requisitos de la lista. Es importante que los requisitos sean medibles, así que se han incluido aquellos que se cree que se pueden cumplir. Entre paréntesis, se muestra con qué celda del modelo de calidad se corresponde el requisito. Los requisitos no funcionales se identifican mediante la nomenclatura **REQ-NF-xx**.

3.3.1. Requisitos de usabilidad

- **REQ-NF-01:** La aplicación debe ofrecer una interfaz sencilla que permita a los usuarios realizar las operaciones sin necesidad de conocimientos técnicos avanzados. (**Reconocibilidad de la adecuación**)
- **REQ-NF-02:** Todos los mensajes de error del sistema deben estar redactados en lenguaje natural e indicar al usuario qué acción debe tomar para resolverlos. (**Asistencia al usuario**)

- **REQ-NF-03:** La aplicación debe mostrar mensajes con información adecuada que ayuden a utilizar la aplicación al realizar las tareas. (**Autodescriptividad**)

3.3.2. Requisitos de portabilidad

- **REQ-NF-04:** La aplicación debe ejecutarse correctamente en dispositivos Android con versión 8.0 o superior e iOS con versión 14.0 o superior.

3.3.3. Requisitos de eficiencia

- **REQ-NF-05:** El tiempo de respuesta de los endpoints de la API no debe producir esperas perceptibles que resulten molestas. (**Comportamiento temporal**)
- **REQ-NF-06:** La aplicación (tanto web como móvil) deberá funcionar correctamente con un renderizado mínimo de 60 FPS. (**Capacidad**)

3.3.4. Requisitos de fiabilidad

- **REQ-NF-07:** El sistema deberá recuperarse correctamente de errores controlados mostrando un mensaje al usuario y evitando el cierre inesperado de la aplicación. (**Ausencia de fallos**)
- **REQ-NF-08:** Si hay una pérdida de conexión durante el uso de la aplicación, el sistema debe informarlo de manera correcta, sin que la aplicación se cierre inesperadamente. (**Disponibilidad, Tolerancia a fallos**)

3.3.5. Requisitos de seguridad

- **REQ-NF-09:** El sistema debe garantizar que cada usuario solo pueda acceder a las funcionalidades correspondientes a su rol. (**Confidencialidad**)
- **REQ-NF-10:** El sistema debe implementar autenticación y autorización y verificar el token de autenticación en cada petición a la API. (**Integridad**)

3.3.6. Requisitos de mantenibilidad

- **REQ-NF-11:** La aplicación debe seguir una arquitectura modular y escalable. Debe seguir una estructura feature-first en el frontend y una arquitectura de tres capas en la aplicación completa. (**Modularidad, Reusabilidad, Capacidad para ser modificado**)
- **REQ-NF-12:** La cobertura de tests unitarios sobre la lógica de negocio debe ser superior al 60 %. (**Capacidad para ser probado**)

3.4. Casos de uso

3.4.1. Clases de usuario

Los distintos usuarios que interactúan con el sistema se distinguen en función de su experiencia técnica, sus niveles de privilegio dentro del sistema y las funcionalidades a las que tienen acceso.

- **Gestor/Encargado de tráfico:** Es el usuario responsable de la gestión logística y administrativa de las operaciones. Pertenece a la empresa porteadora principal, la responsable de las cargas frente al cargador. Su interacción con el sistema se realiza mayoritariamente a través de una interfaz gráfica, por lo que se le presupone una experiencia técnica media. Tiene permisos de administración para modificar estados de entrega, realizar asignaciones y generar documentación legal, como las cartas de porte.
- **Chófer:** Su interacción con el sistema ocurre principalmente durante la jornada en los puntos de carga y descarga a través del dispositivo móvil. Su experiencia técnica puede ser variada y, debido al entorno en el que trabaja, se le debe ofrecer una interfaz de usuario muy simplificada. Tiene permisos limitados para las acciones relacionadas con sus tareas e incidencias que le afectan directamente.
- **Cargador:** Es el perfil que inicia la cadena logística y provee las cargas a la empresa de transporte. Su función en la aplicación es insertar nuevos pedidos, especificando los detalles y seleccionando algún tipo de carga que ya tenga prefijado con la empresa de transporte. El acceso al sistema se gestiona a través del alta que le da el encargado y sus permisos están limitados al apartado de insertar y editar pedidos. También puede ver las cartas de porte generadas para sus cargas.
- **Subcontratado:** Es la empresa externa a la que se le delegan las cargas que la empresa de transporte principal no puede realizar con medios propios. Es la empresa porteadora efectiva, responsable frente a la empresa porteadora principal. Este usuario solo puede visualizar las cargas que se le han cedido y tiene acceso a insertar los datos de sus propios vehículos y chóferes para realizar los viajes de las cargas cedidas y poder generar la carta de porte. No tiene permisos para modificar la planificación o interactuar con los datos de la empresa propietaria de las cargas.

3.4.2. Funcionalidades del producto

A continuación, se muestran las principales funcionalidades del sistema:

Para el encargado de tráfico:

- **Dar de alta a un usuario:** El encargado será la persona responsable de registrar a los transportistas subcontratados, a los cargadores y a los chóferes de su empresa.

- **Dar de baja a un usuario:** Cuando la relación con el usuario termine, podrá dar de baja al usuario para que no tenga acceso al sistema.
- **Enviar correo con credenciales:** Una vez que haya dado de alta a un usuario, el encargado podrá enviar un correo con una contraseña temporal y una dirección URL para restablecer su contraseña.
- **Gestionar flota:**
 - **Insertar vehículo en el sistema:** Podrá insertar un vehículo de la empresa en el sistema, introduciendo los datos relevantes. Existen vehículos internos y externos.
 - **Actualizar detalles de vehículo:** Podrá actualizar los detalles de un vehículo de la empresa.
 - **Eliminar vehículo:** Podrá eliminar un vehículo del sistema.
- **Realizar planificación:** Podrá realizar la planificación semanal asignando los vehículos y las cargas a los chóferes.
 - **Asignar vehículo:** Podrá asignar un vehículo a un chófer para que realice el viaje.
 - **Asignar carga:** Podrá asignar una carga a un chófer.
 - **Ceder cargas a subcontratado:** Podrá ceder varias cargas de un pedido a la empresa subcontratada.
- **Enviar carta de porte:** Podrá generar y enviar la carta de porte que le corresponde a cada carga del pedido. La carta de porte se envía tanto al cargador como al chófer y en su caso, a la empresa subcontratada.
- **Visualizar planificación general:** El encargado dispondrá de un calendario para monitorizar qué chóferes están en ruta, cuáles están libres y el estado de las entregas en tiempo real.
- **Gestionar Incidencias:** Además de consultarlas, el encargado podrá marcar incidencias como “Resueltas”, adjuntar notas sobre la solución y, si es necesario, reasignar la carga a otro chófer.

La Figura 3.4 muestra los casos de uso de relacionados con la autenticación y la gestión de perfiles.

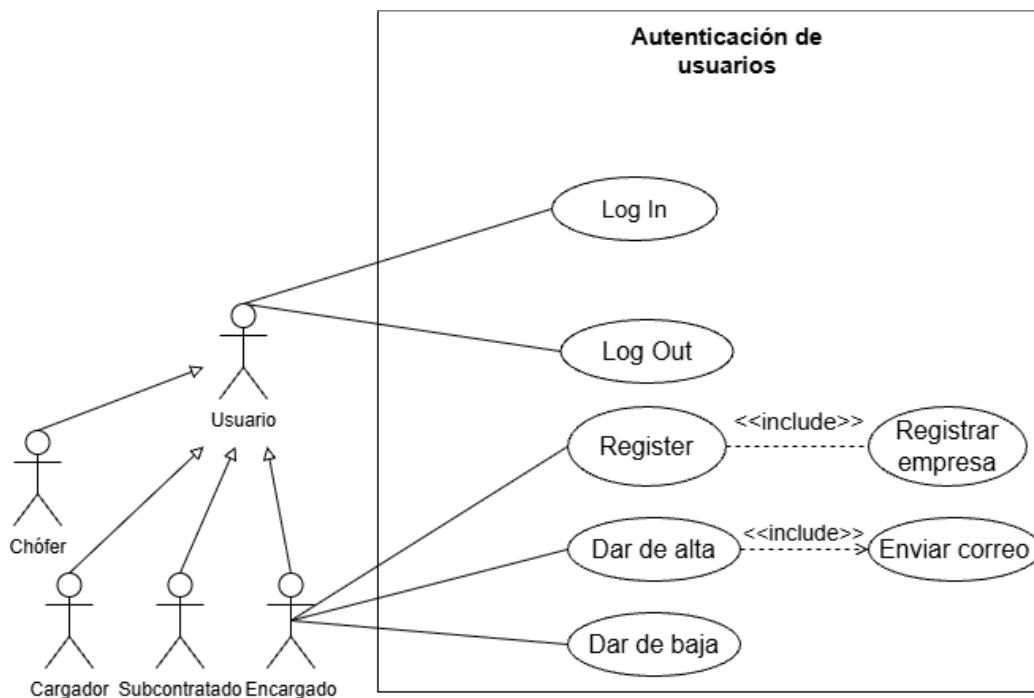


Figura 3.4: Diagrama de casos de uso de autenticación y gestión de usuarios

Funcionalidades comunes (Encargado y Cargador):

■ Gestionar pedidos y cargas:

La gestión de los pedidos podrá hacerla tanto el cargador como el encargado, una vez que se le haya dado de alta en el sistema.

- **Insertar pedido:** Podrá añadir un pedido, seleccionando el tipo de carga que tengan prefijado.
- **Editar pedido:** Podrá actualizar los detalles de un pedido.
- **Eliminar pedido:** Podrá eliminar un pedido y las cargas contenidas en el pedido.

Funcionalidades comunes (Encargado y Subcontratado):

- **Asignar vehículo a carga:** Podrá asignar un vehículo a una carga.
- **Asignar chófer a carga:** Podrá asignar un chófer de su empresa a una carga.
- **Visualizar cargas:** Podrán visualizar los detalles de las cargas. El encargado verá todas y el subcontratado solo las que se le han cedido.

3. ANÁLISIS DE REQUISITOS

La Figura 3.5 muestra los casos de uso relacionados con la gestión de pedidos y cargas y la planificación del transporte.

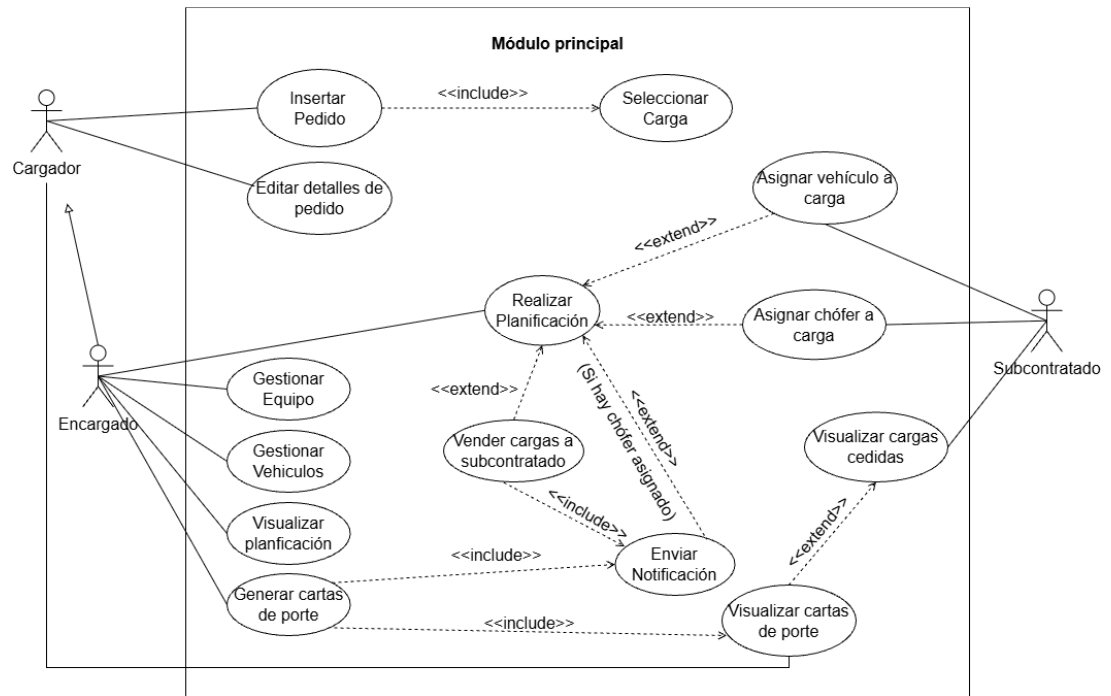


Figura 3.5: Diagrama de casos de uso del módulo principal

Para los chóferes:

- **Visualizar cargas asignadas:** Podrá visualizar las cargas que tiene asignadas en un calendario.
- **Consultar detalles de una carga:** Podrá solicitar los detalles de sus cargas asignadas.
- **Confirmar recogida:** Podrá confirmar la recogida de una carga en un origen concreto.
- **Confirmar entrega:** Podrá confirmar la entrega de una carga en un destino concreto.
- **Reportar incidencia:** Podrá reportar una incidencia con los detalles de la misma.
- **Consultar incidencias:** Podrá consultar las incidencias que haya creado o las que ha creado el encargado y le afecten.
- **Visualizar ruta actual:** La navegación será delegada a Google Maps con el origen y el destino seleccionados para que vea la ruta en caso de que lo necesite.

La Figura 3.6 muestra los casos de uso descritos en la lista de arriba.

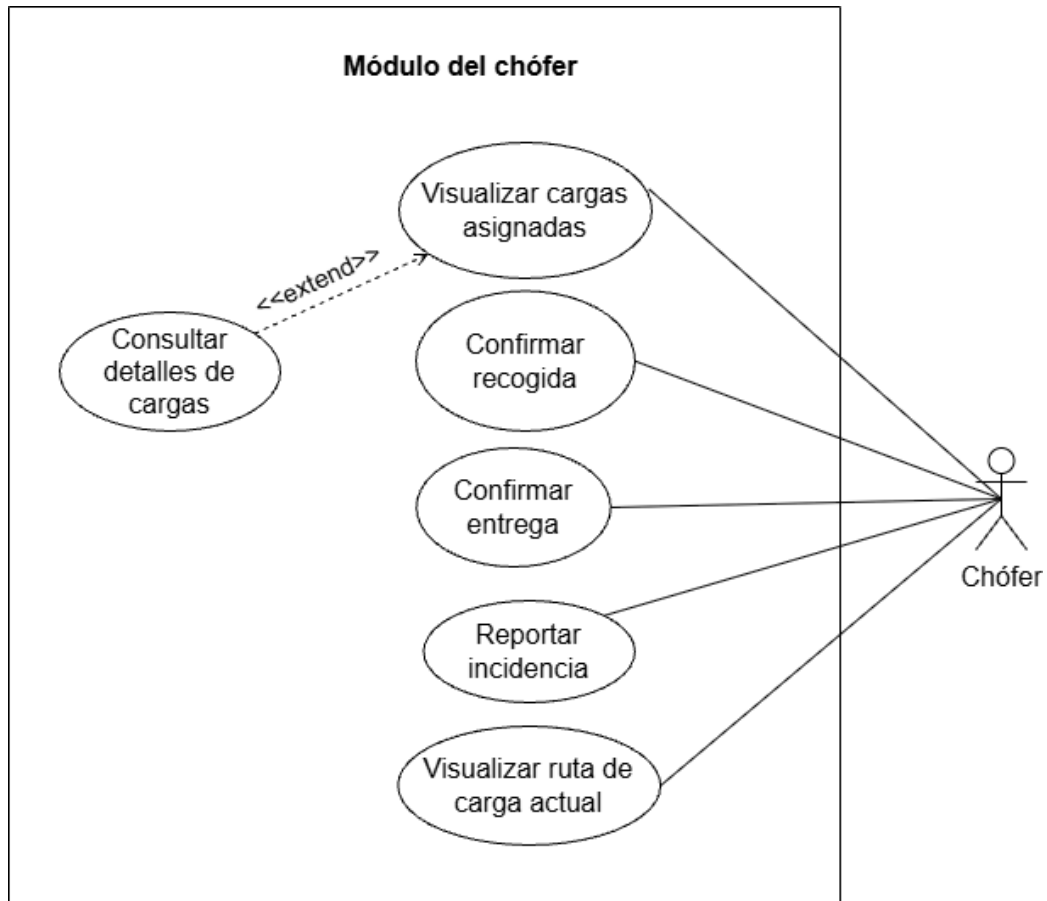


Figura 3.6: Diagrama de casos de uso del usuario chófer

3.5. Especificación de los casos de uso

A continuación, se muestra la especificación de los casos de uso más importantes de los definidos en la Figura 3.4, la Figura 3.5 y la Figura 3.6. Para cada uno, se muestran el actor, las precondiciones, el flujo normal, el flujo alternativo y las postcondiciones. La Tabla 3.1 permite localizar la especificación de cada caso de uso y, cuando corresponde, su figura.

Caso de uso	Tabla	Figura
Dar de alta	3.2	3.7
Dar de baja	3.3	
Asignar vehículo a carga	3.4	
Asignar chófer a carga	3.5	
Ceder carga a subcontratado	3.6	
Realizar planificación	3.7	3.8
Visualizar planificación	3.8	3.9
Insertar pedido	3.9	3.12
Seleccionar tipo de carga	3.10	
Crear tipo de carga	3.11	
Generar carta de porte	3.12	3.2
Visualizar cargas asignadas	3.13	
Consultar detalles de carga	3.14	
Reportar incidencia	3.15	
Visualizar incidencias	3.16	

Tabla 3.1: Resumen de los casos de uso especificados.

Dar de alta:

Tabla 3.2: Caso de uso Dar de alta

Actor	Encargado
Precondiciones	El actor debe estar autenticado en el sistema. El actor debe estar en la página de gestión de usuarios.
Flujo	1. El encargado accede a la sección de gestión de usuarios 2. El sistema muestra un campo donde insertar el email del usuario. 3. El encargado inserta el email y hace clic en “Enviar Invitación”. 4. El sistema registra al usuario invitado y permite enviar un correo con las credenciales y un enlace para cambiar la contraseña temporal.
Flujo alternativo	4 alt. El encargado cancela la operación pulsando Cancelar. 5 alt. El usuario con ese correo ya existe y se cancela la operación.
Postcondiciones	El nuevo usuario se inserta en la base de datos.

En la Figura 3.7 se puede observar la interfaz donde se lleva a cabo el flujo del caso de uso descrito arriba.

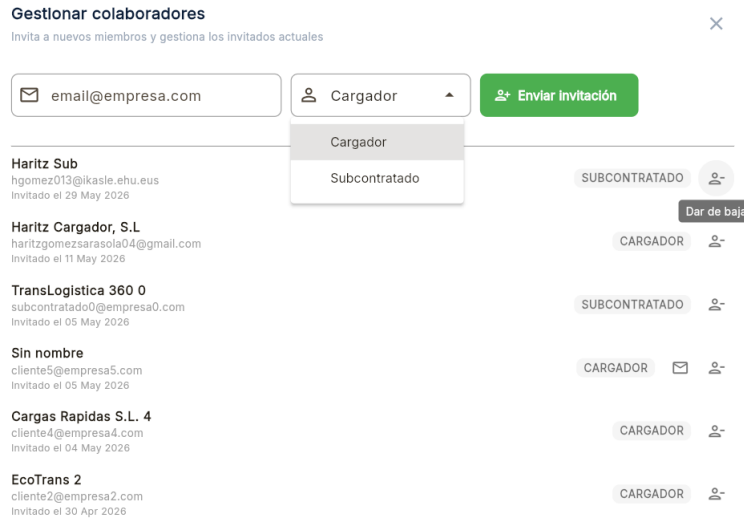


Figura 3.7: Interfaz de dar de alta a usuario

Dar de baja:

Tabla 3.3: Caso de uso Dar de baja

Actor	Encargado
Precondiciones	El actor debe estar autenticado en el sistema. El actor debe estar en la página de gestión de usuarios.
Flujo	1. El encargado selecciona el usuario al que desea dar de baja. 2. El sistema solicita confirmación al encargado para proceder con la baja. 3. El encargado confirma la operación pulsando en Confirmar. 4. El sistema elimina al usuario y muestra un mensaje de confirmación.
Flujo alternativo	3 alt. El encargado cancela la operación pulsando Cancelar.
Postcondiciones	Se hace soft-delete del usuario en la base de datos.

Asignar vehículo a carga:

Tabla 3.4: Caso de uso Asignar vehículo

Actor	Encargado y Subcontratado
Precondiciones	El actor debe estar autenticado en el sistema. El actor debe estar en la página de planificación o insertando un pedido.
Flujo	1. El actor selecciona una carga y pulsa en el menú para ver todos los vehículos disponibles. 2. El sistema muestra únicamente los vehículos disponibles para las fechas de esa carga. Si el actor es el Encargado, muestra la flota propia; si es Subcontratado, deberá introducir la matrícula del camión y del remolque a mano. 3. El actor selecciona el vehículo que se asignará a esa carga. 4. El actor guarda los cambios y se persiste esa asignación.
Postcondiciones	La carga tendrá un vehículo asignado.

Asignar chófer a carga:

Tabla 3.5: Caso de uso Asignar carga a chófer

Actor	Encargado y Subcontratado
Precondiciones	El actor debe estar autenticado en el sistema. El actor debe estar en la página de planificación o insertando un pedido.
Flujo	1. El usuario selecciona la carga a la que quiere asignarle un chófer. 2. El sistema muestra únicamente los chóferes disponibles. Si el actor es el Encargado, muestra los chóferes dados de alta; si es Subcontratado, deberá introducir el nombre y la identificación de su conductor. 3. El usuario selecciona al chófer responsable de transportar la carga. 4. El sistema vincula la carga al chófer y cambia el estado de la carga a "Asignado". 5. El sistema muestra un mensaje de confirmación y envía una notificación al dispositivo móvil del chófer.
Postcondiciones	La carga queda asociada a un chófer y el chófer podrá ver esa carga en su calendario.

Ceder carga a subcontratado:**Tabla 3.6:** Caso de uso Ceder carga a subcontratado

Actor	Encargado
Precondiciones	El actor debe estar autenticado en el sistema. El actor debe estar en la página de planificación.
Flujo	1. El usuario selecciona la carga que quiere ceder. 2. El sistema muestra las empresas subcontratadas a las que se les ha dado de alta. 3. El actor selecciona la empresa a la que quiere ceder la carga. 4. El actor selecciona la comisión con la que se va a quedar por ceder la carga y pulsa en el botón para cederla. 5. El sistema cambia el estado de la carga a “Cedido” y añade a la lista de cargas cedidas del subcontratado la nueva carga.
Postcondiciones	La carga pasa a estado “Cedido” y se liberan el chófer y el vehículo. No se permite planificar la carga y se queda bloqueada.

Realizar Planificación:**Tabla 3.7:** Caso de uso Realizar Planificación

Actor	Encargado
Precondiciones	El actor debe estar autenticado en el sistema.
Flujo principal	1. El encargado accede al panel de planificación. 2. El sistema muestra una lista con los pedidos pendientes por planificar y la disponibilidad actual de la flota y chóferes. 3. El encargado realiza la planificación semanal parcial o completa utilizando los casos de uso disponibles (Ver flujos de extensión). 4. El encargado selecciona la cantidad de horas durante las que el conductor y vehículo de esa carga estarán bloqueados hasta poder asignarse de nuevo, debido al tiempo de retorno. 5. El sistema guarda la planificación y actualiza el estado.
Flujos de extensión	3a. Asignar chófer a carga: El encargado vincula un chófer a una carga específica. 3b. Asignar vehículo a carga: El encargado vincula el vehículo que realizará la carga a la carga. 3c. Ceder cargas a subcontratado: El encargado deriva la responsabilidad a una empresa subcontratada.
Postcondiciones	La planificación queda guardada y se actualiza el estado de las cargas.

3. ANÁLISIS DE REQUISITOS

En la Figura 3.8 se puede observar la interfaz que corresponde con el caso de uso de realizar la planificación, que se ha descrito en la tabla de arriba. Los números corresponden con los números del flujo de ejecución.

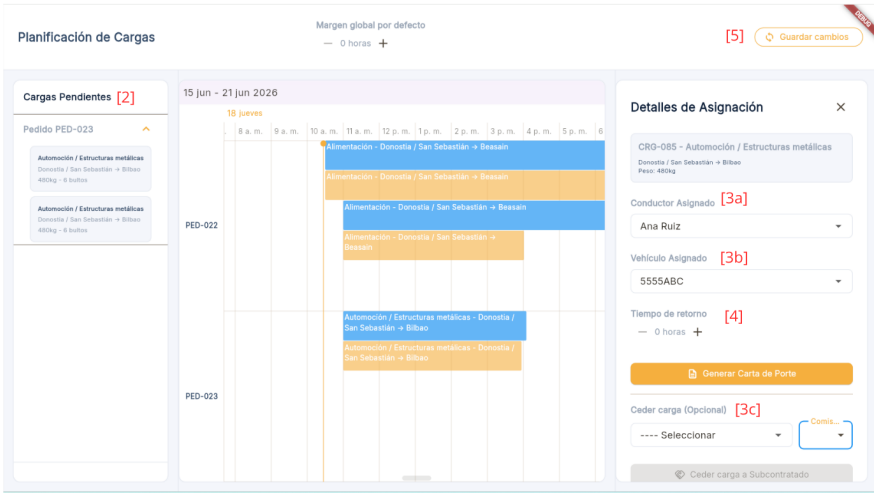


Figura 3.8: Interfaz de Realizar Planificación, numerada

Visualizar Planificación:

Tabla 3.8: Caso de uso Visualizar Planificación

Actor	Encargado
Precondiciones	El actor debe estar autenticado en el sistema.
Flujo principal	1. El encargado accede al panel de control. 2. El sistema carga el calendario configurado para la semana/mes en curso. 3. El sistema recupera de la base de datos todos los pedidos y cargas programadas para esa semana. 4. El sistema muestra las cargas sobre el calendario, indicando el estado de cada una y los recursos asignados.
Postcondiciones	El encargado visualiza la planificación de esa semana.

La Figura 3.9 muestra el calendario mensual donde se realiza el caso de uso descrito arriba, aunque también se puede cambiar a la vista semanal.

3.5. Especificación de los casos de uso

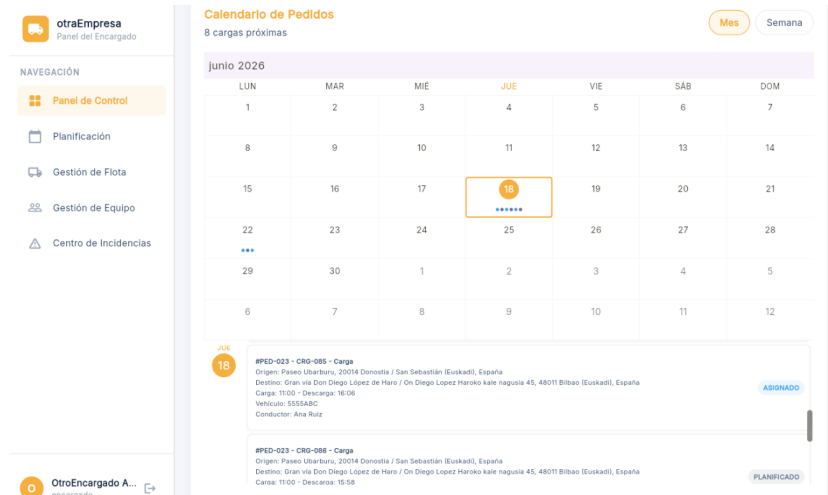


Figura 3.9: Interfaz de Visualizar Planificación

Insertar Pedido:

La inserción del pedido involucra el caso de uso Seleccionar tipo de carga (véase Tabla 3.10). En la Figura 3.12 se muestra una parte de la interfaz de inserción del pedido, que incluye tanto la selección del tipo de carga como la asignación de fechas individuales a las cargas creadas a partir del tipo de carga seleccionado.

Tabla 3.9: Caso de uso Insertar Pedido

Actor	Encargado o Cargador
Precondiciones	El actor debe estar autenticado. El actor debe estar en la sección de pedidos.
Flujo	1. El actor introduce los datos generales del pedido: cliente, cargador, fecha de inicio y fecha máxima. 2. El actor abre el <i>Dropdown</i> para seleccionar el tipo de carga. 3. El sistema invoca el Caso de Uso: Seleccionar tipo de carga (Tabla 3.10). 4. El actor selecciona la cantidad de cargas que quiere de ese tipo de carga y pulsa en “Siguiente”. 5. Cuando el actor termina de añadir cargas, puede definir fechas individuales para cada carga, dentro de las fechas del pedido. 6. El sistema calcula el resumen del pedido y confirma el registro.
Flujo de extensión	5a. Asignar recursos: Si el actor es el Encargado, puede asignar vehículo y chófer a cada carga. El sistema filtra los disponibles.
Flujo alternativo	4 alt. El actor cancela el pedido completo y el sistema elimina las cargas asociadas.

3. ANÁLISIS DE REQUISITOS

Postcondiciones	El pedido se inserta en la base de datos y se vincula a sus cargas.
-----------------	---

La Figura 3.12 muestra una parte de la interfaz utilizada para crear un nuevo pedido de transporte. Los números corresponden con los del flujo de ejecución de la tabla de arriba.

Nuevo Pedido

Selecccionar cargas

Cargador

Haritz Cargador, S.L

Fecha de carga

18/06/2026 13:01

Fecha límite descarga

25/06/2026 13:01

Crear nuevo tipo

Envío Componentes Metal

3

Añadir

Plantilla para el envío de racks metálicos pesados desde la fundición a la planta de ensamble. 45€/ud

Origen: Paseo Ubarburu, 20014 Donostia / San Sebastián (Euskadi), España

Destino: Gran vía Don Diego López de Haro / On Diego Lopez Haroko kale nagusia 45, 48011 Bilbao (Euskadi), España

Atrás

Siguiente

Tipo de carga

3

Añadir

Envío Componentes Metal (Unidad 1)

Carga

18/06 17:00

Descarga

18/06 20:00

Envío Componentes Metal (Unidad 2)

Carga

18/06 17:00

Descarga

18/06 20:00

Envío Componentes Metal (Unidad 3)

Carga

18/06 18:00

Descarga

18/06 21:00

Atrás

Siguiente

Figura 3.12: Flujo de inserción de cargas dentro del nuevo pedido.

Seleccionar tipo de carga:

Tabla 3.10: Caso de uso Seleccionar tipo de carga

Actor	Encargado o Cargador
Precondiciones	El actor debe estar autenticado. El actor ha iniciado el flujo de “Insertar Pedido”.
Flujo principal	1. El sistema despliega un listado de tipos de carga que se habrán prefijado anteriormente entre el cargador y la empresa de transporte. 2. El actor selecciona el tipo de carga deseado. 3. En caso de que sea un nuevo tipo de carga que aún no existe en el sistema entre la empresa cargadora y la empresa de transporte, se permite crear un nuevo tipo de carga.
Postcondiciones	El actor selecciona el tipo de carga o crea uno nuevo. Puede seguir con la inserción del nuevo pedido.

Una vez seleccionado el tipo de carga, el flujo continúa en el Caso de Uso Insertar Pedido descrito en la Tabla 3.9.

Crear tipo de carga:**Tabla 3.11:** Caso de uso Crear tipo de carga

Actor	Encargado o Cargador
Precondiciones	El actor debe estar autenticado. El actor ha iniciado el flujo de “Insertar Pedido”.
Flujo principal	1. El sistema muestra el formulario con los campos que debe tener un tipo de carga (véase el diagrama conceptual). 2. El actor rellena el formulario con los datos del tipo de carga. 3. Al insertar origen y destino, el sistema muestra sugerencias de direcciones reales para facilitar la inserción de datos. 4. El actor selecciona una dirección sugerida. 5. El sistema mapea esa dirección a la latitud y longitud correspondientes y guarda tanto la dirección como las coordenadas, para facilitar la búsqueda de la ruta al conductor al transportar la carga.
Postcondiciones	Se crea el tipo de carga que servirá como “plantilla” para crear pedidos de esa mercancía posteriormente.

Generar carta de porte:**Tabla 3.12:** Caso de uso Generar carta de porte

Actor	Encargado
Precondiciones	El actor debe estar autenticado en el sistema. El actor se encuentra en la página de planificación.
Flujo	1. El encargado selecciona la carga a la que va a generar la carta de porte. 2. El sistema muestra un botón para generar la carta de porte de esa carga. 3. El encargado pulsa en el botón de generar la carta de porte. 4. El sistema obtiene los datos del pedido, el cliente, el vehículo y el chófer asignado para generar la carta de porte y genera el PDF de la carta de porte. 5. El sistema envía una notificación al cargador y en su caso, al subcontratado.
Flujo alternativo	4 alt. La asignación no está completa por lo que no se permite generar la carta de porte.
Postcondiciones	Se genera la carta de porte.

Un ejemplo del documento PDF generado por este flujo puede consultarse en la [Figura 3.2](#).

Visualizar cargas asignadas:**Tabla 3.13:** Caso de uso Visualizar cargas asignadas

Actor	Chófer
Precondiciones	El actor debe estar autenticado en el sistema. El chófer debe tener cargas asignadas pendientes de finalizar.
Flujo	1. El chófer accede a su pantalla principal. 2. El sistema recupera la lista de cargas de ese chófer y las muestra en un calendario con los detalles.
Flujo alternativo	1 alt. El sistema no encuentra cargas asignadas a ese chófer e informa al chófer. 2 alt. El chófer solicita más información; el sistema invoca el Caso de Uso: Consultar detalles de carga.
Postcondiciones	El sistema mantiene la información de la carga seleccionada visible en la interfaz.

Consultar detalles de carga:**Tabla 3.14:** Caso de uso Consultar detalles de carga

Actor	Chófer
Precondiciones	El actor debe estar autenticado en el sistema. El chófer ha seleccionado una carga desde su planificación.
Flujo	1. El chófer pulsa sobre la tarjeta de la carga para expandir la información. 2. El sistema recupera de la base de datos la ficha técnica completa de la carga. 3. El sistema muestra los detalles en una nueva vista de la aplicación móvil.
Flujo alternativo	1 alt. El sistema no logra recuperar los datos y muestra un mensaje de error y permite reintentar la carga de datos.
Postcondiciones	Se muestran los detalles de la carga seleccionada.

Reportar incidencia:**Tabla 3.15:** Caso de uso Reportar incidencia

Actor	Chófer o Encargado
Precondiciones	El actor debe estar autenticado en el sistema.
Flujo	1. El actor selecciona la opción de Reportar incidencia. 2. El sistema solicita una descripción y de manera opcional el tipo de incidencia. 3. El actor introduce los detalles. Si es encargado también debe seleccionar los chóferes a los que afecta la incidencia.

	4. El sistema registra la incidencia con su descripción y el actor que la crea.
Flujo alternativo	
Postcondiciones	La incidencia se guarda en la base de datos.

Visualizar incidencias:**Tabla 3.16:** Caso de uso Visualizar incidencias

Actor	Chófer o Encargado
Precondiciones	El actor debe estar autenticado en el sistema.
Flujo	1. El actor solicita el listado de incidencias. 2. El sistema identifica el rol del usuario autenticado. 3. Si el actor es Encargado: El sistema recupera todas las incidencias globales del sistema. 4. Si el actor es chófer: El sistema filtra y recupera únicamente las incidencias creadas por él o asociadas a sus cargas.
Flujo alternativo	
Postcondiciones	Se visualizan las incidencias que le interesan a cada actor.

Selección de tecnologías

La aplicación implementada es una aplicación multiplataforma *full-stack*. El frontend se ha implementado utilizando el **framework multiplataforma Flutter**¹, mientras que para implementar el backend se ha utilizado Python, concretamente el **framework FastAPI**². Por último, se han utilizado varios servicios de la **plataforma de servicios en la nube Firebase**³.

4.1. Frontend - Flutter

Flutter es un framework basado en el lenguaje Dart, creado por Google en 2017, que permite crear aplicaciones multiplataforma a partir de una única base de código. Aunque es un framework fuertemente orientado al desarrollo móvil (*mobile-first*), hoy en día también permite la creación de aplicaciones web gracias a **Flutter Web**. Gracias a esta tecnología, se les permite a los usuarios que utilizan un ordenador (en esta aplicación, normalmente serán el encargado, los cargadores y los responsables de las empresas subcontratadas) acceder a la versión web a través del navegador, mientras que a los chóferes, que no tienen un ordenador disponible, se les da la posibilidad de acceder directamente a través de la aplicación móvil.

Más allá de su versatilidad, Flutter también destaca por su alto rendimiento, logrando un rendimiento similar al de aplicaciones nativas gracias a su compilación a lenguaje máquina. En entornos web, también se logra un gran rendimiento gracias a la adopción de **WebAssembly (Wasm)**, eliminando las limitaciones que había anteriormente al tener que traducir el código a JavaScript.

Comparándolo con otras tecnologías similares, React Native⁴ también es un framework multiplataforma y la mayor competencia de Flutter. Este framework está basado en el lenguaje

¹Sitio web oficial de Flutter: <https://flutter.dev/>. Consultado el 6 de agosto de 2026.

²Sitio web oficial de FastAPI: <https://fastapi.tiangolo.com/>. Consultado el 6 de agosto de 2026.

³Sitio web oficial de Firebase: <https://firebase.google.com/>. Consultado el 6 de agosto de 2026.

⁴Sitio web oficial de React Native: <https://reactnative.dev/>. Consultado el 6 de agosto de 2026.

JavaScript que se ha aprendido durante la carrera, por lo que la curva de aprendizaje sería más rápida. Sin embargo, el rendimiento y la consistencia visual entre plataformas en React Native no son tan buenos como en Flutter. Además de las ventajas técnicas, el aprendizaje de una nueva tecnología ha sido otra razón por haber elegido Flutter frente a React Native.

Otras características de Flutter:

- **Tipado estático:** Útil para detectar errores en tiempo de compilación en lugar de en ejecución.
- **Facilidad de diseños responsivos:** Flutter tiene su propio sistema de widgets que facilita la creación de interfaces responsivas. Esto es muy importante al tratarse de una aplicación pensada para navegadores y dispositivos móviles.
- **Hot Reload:** Permite ver los cambios de forma instantánea, sin tener que reiniciar la aplicación.
- **SEO limitado:** La desventaja de tener su propio sistema de widgets es que las aplicaciones son una “caja negra” para los *bots* de los motores de búsqueda. Aun así, no es una gran desventaja para este tipo de aplicaciones de gestión.

4.2. API - FastAPI

FastAPI es un framework moderno basado en el lenguaje de programación Python. Destaca por su velocidad y su facilidad de uso al crear APIs, lo que reduce mucho el tiempo de desarrollo. Gracias a su naturaleza asíncrona, permite gestionar múltiples peticiones sin bloquear el servidor, logrando así una gran eficiencia de recursos.

Comparándolo con otros frameworks basados en Python, FastAPI se encuentra en un buen equilibrio entre la simplicidad de Flask y la gran robustez de Django REST Framework (DRF). Además de ser considerablemente superior en rendimiento a ambos, FastAPI ofrece una implementación más robusta, moderna y segura que Flask. Por otro lado, la aplicación no necesita un framework tan pesado como Django, que añade una capa de complejidad y de recursos extra que no se aprovecharían completamente en una API que actúa como puente entre Flutter y la base de datos.

Características de FastAPI:

- **Documentación técnica automática:** Genera documentación automática interactiva basada en Swagger⁵. Esto permite a los nuevos desarrolladores visualizar y probar los *endpoints*.
- **Inyección de dependencias:** Los componentes se van uniendo a través de inyección de dependencias. Esto separa los componentes por responsabilidades y facilita el testing unitario de los componentes por separado.

⁵Sitio web oficial de Swagger: <https://swagger.io/>. Consultado el 6 de agosto de 2026.

- **Validación de datos:** Utiliza esquemas *Pydantic* para validar los datos de entrada, reduciendo los errores y el código repetitivo al definir una estructura en los datos.

4.3. Base de datos y servicios - Firebase

Firebase es una plataforma de servicios en la nube (Backend as a Service) de Google que proporciona un conjunto de herramientas para facilitar la implementación. Al ser una herramienta de Google, es muy común utilizarla con aplicaciones creadas en Flutter. En este proyecto se han utilizado varios de sus servicios dentro del plan gratuito:

- **Firestore:** Es la base de datos NoSQL de Firebase. Destaca por ser muy rápida, ya que realiza todas sus consultas a través de índices que se crean automáticamente, haciendo que el tiempo de respuesta dependa principalmente del conjunto de resultados. Además, al ser una base de datos gestionada en la nube de Google, evita tener que gestionar la infraestructura de una base de datos convencional. Ofrece un plan gratuito de 50.000 lecturas y 20.000 escrituras diarias, que lo hace ideal para probar prototipos.
- **Firebase Storage:** Es el servicio de almacenamiento de archivos de Firebase. Se utiliza para guardar las cartas de porte, ya que es un requisito legal guardarlas en la nube durante un año como mínimo.
- **Firebase Authentication:** Es el servicio de autenticación de usuarios, gestiona la identidad de los usuarios usando estándares de autenticación como OAuth 2.0. Permite implementar flujos de registro y acceso sin tener que programar la lógica de cifrado o almacenamiento de contraseñas en el servidor.

4.4. Tecnologías de testing

Tanto Flutter como FastAPI ofrecen herramientas para probar el software implementado mediante pruebas unitarias y de integración. En Flutter, el propio SDK ofrece la librería *flutter_test*, que permite ejecutar pruebas unitarias para verificar los componentes del frontend de manera aislada. Además, también se ha empleado la librería *integration_test* para realizar los tests de integración y probar los flujos entre distintas capas de la aplicación.

Para el backend, la librería de testing utilizada ha sido **pytest**⁶, que es el framework estándar de testing en Python. La metodología y las estrategias de testing utilizadas se detallarán en más profundidad en el capítulo de Testing (7).

Junto a las tecnologías de testing, se han utilizado tecnologías para la **Integración Continua** (CI). Se ha utilizado SonarCloud⁷ para el análisis estático de código (*code smells*, vulnerabilidades de seguridad, errores de mantenimiento, etc.) y poder ver el *coverage* de los tests. Por otro lado, se ha utilizado GitHub Actions para ejecutar los tests implementados en cada

⁶Sitio web oficial de pytest: <https://pytest.org/>. Consultado el 6 de agosto de 2026.

⁷Sitio web oficial de SonarCloud: <https://sonarcloud.io/>. Consultado el 6 de agosto de 2026.

cambio que se suba al repositorio. De esta manera, se garantiza que cualquier error se detecte antes de integrarlo en la rama principal.

4.5. Despliegue - Firebase Hosting y Google Cloud Run

Para el despliegue, se ha aprovechado que el proyecto se encuentra dentro del ecosistema de Google. El frontend se ha desplegado en **Firebase Hosting**⁸, el servicio de alojamiento de contenido estático de Firebase, que ofrece una integración directa con el resto de servicios de Firebase y distribuye el contenido a través de un CDN (Content Delivery Network), lo que ayuda a mejorar aspectos como el tiempo de carga. Para desplegar el backend, se requiere un entorno capaz de ejecutar un proceso de servidor. Para ello se ha utilizado **Google Cloud Run**⁹, un entorno *serverless* que ejecuta aplicaciones empaquetadas en contenedores a partir de una imagen Docker¹⁰. Para utilizar estas herramientas se requiere el plan pay-as-you-go de Firebase, pero cuenta con un límite gratuito muy generoso que no se va a superar dentro del proyecto. El proceso de despliegue del frontend y del backend se describe en la Sección 6.7.

⁸Sitio web oficial de Firebase Hosting: <https://firebase.google.com/products/hosting>. Consultado el 6 de agosto de 2026.

⁹Sitio web oficial de Google Cloud Run: <https://cloud.google.com/run>. Consultado el 6 de agosto de 2026.

¹⁰Sitio web oficial de Docker: <https://www.docker.com/>. Consultado el 6 de agosto de 2026.

CAPÍTULO 5

Diseño

Una vez cubiertas las tecnologías, en este capítulo se profundiza en el diseño y la arquitectura de la aplicación. Se analizan el modelado de datos, los diagramas de clases y finalmente, se muestran los diagramas de secuencia para entender el flujo de los principales casos de uso de la aplicación.

5.1. Diseño de la arquitectura general

La aplicación desarrollada debe ofrecer un servicio unificado para clientes web y móviles. La información debe estar centralizada para controlar la **integridad** de los datos, y al ser una **aplicación multi-tenant**, el aislamiento entre empresas y la distinción de roles juegan un papel muy importante. Por estas razones, una arquitectura **cliente-servidor** es la mejor opción, en la que el servidor centraliza todas estas responsabilidades técnicas. Además de esto, pensando en la escalabilidad futura de la aplicación, este tipo de aplicaciones suelen integrarse con sistemas externos, sobre todo sistemas empresariales o logísticos que ofrecen funcionalidades concretas que extienden la aplicación, por lo que es necesaria una **API** en el servidor que actúe como punto de entrada para los clientes y futuras posibles integraciones.

Esta arquitectura se diferencia de lo que suele verse en aplicaciones que usan Firebase, donde es común prescindir de un backend propio y que la lógica de negocio se gestione en el cliente, conectándose directamente a Firebase para interactuar con los datos o con la autenticación de usuarios. Sin embargo, esto suele ser más adecuado en aplicaciones pequeñas donde la complejidad del sistema y otros aspectos como la integridad de los datos o la seguridad son fáciles de gestionar sin un servidor propio. Aunque Firebase tiene mecanismos para gestionar la seguridad y la protección de los datos a través de sus reglas de seguridad en la nube, depender únicamente de estas reglas se vuelve cada vez más complejo a medida que la aplicación escala.

Con esta arquitectura, las responsabilidades quedan bien separadas, cumpliendo con el

Single Responsibility Principle (SRP) de SOLID: el cliente gestiona la presentación y el estado del frontend, mientras que el backend dedicado se ocupa de la lógica de negocio, la validación de los datos, de la gestión de la seguridad y del intercambio de datos entre el cliente y la base de datos a través de la API. Firebase, por tanto, se convierte en un componente de **infraestructura** (base de datos en la nube) y **servicios** (Firebase Auth, notificaciones push, etc.).

Además de las ventajas en el control de la seguridad y la integridad, la separación de responsabilidades permite una mayor facilidad para evolucionar las tecnologías del sistema. Como Firestore es una base de datos en la nube, su coste depende de la cantidad de operaciones y su coste no es fijo. En un entorno real de producción, podría darse la situación de que el coste de uso supere el presupuesto que se tiene para el proyecto. Al tener un backend propio, migrar el sistema a otra base de datos únicamente requeriría cambiar el código del servidor, manteniendo la comunicación intacta en todas las aplicaciones al estar consumiendo la misma API; así, el corte en el servicio es mínimo para cualquier cambio o refactorización. Para facilitar un futuro cambio de base de datos, se ha aplicado el **patrón de diseño Repository**, que desacopla el acceso a la base de datos de la lógica de aplicación utilizando interfaces, como se puede observar en la Figura 5.2. Con este patrón se cumple el **DIP: Dependency Inversion Principle** de SOLID, ya que la lógica de negocio depende de una abstracción de la capa de persistencia. Por el contrario, hacer este mismo cambio directamente en Flutter requeriría volver a compilar las aplicaciones y desplegar una nueva versión en las tiendas de aplicaciones (para las apps móviles) o en el servidor de hosting (para la aplicación web), cortando el servicio durante un tiempo mucho mayor.

La Figura 5.1 detalla el diagrama por capas de la aplicación. El cliente sigue una **arquitectura MVVM (Model-View-ViewModel)**, donde se separa la presentación (View), la gestión del estado (ViewModel) y el modelo (Model). Esta arquitectura es una variante del MVC (Modelo-Vista-Controlador) clásico, adaptado para frameworks declarativos, en la que el ViewModel expone su estado y la vista se suscribe al estado para actualizarse de manera reactiva (esto se detalla más en 5.1.1). El backend sigue una **arquitectura modular por capas** y está compuesto por controladores (routers), servicios y el acceso a la base de datos. La Figura 5.2 muestra la arquitectura por módulos, representada a través de un diagrama de componentes UML.

Cada componente del diagrama de componentes pasa por las capas de su subsistema del diagrama de capas. Cuando el usuario realiza una operación desde la interfaz, la petición se envía al backend mediante HTTP. El controlador recibe la solicitud, las dependencias verifican la autenticación y autorización, los esquemas Pydantic validan la estructura de los datos de entrada, la capa de servicios ejecuta la lógica de negocio y finalmente el módulo de acceso a la base de datos interactúa con Firestore. La respuesta sigue el recorrido inverso hasta llegar de nuevo al cliente.

5.1. Diseño de la arquitectura general

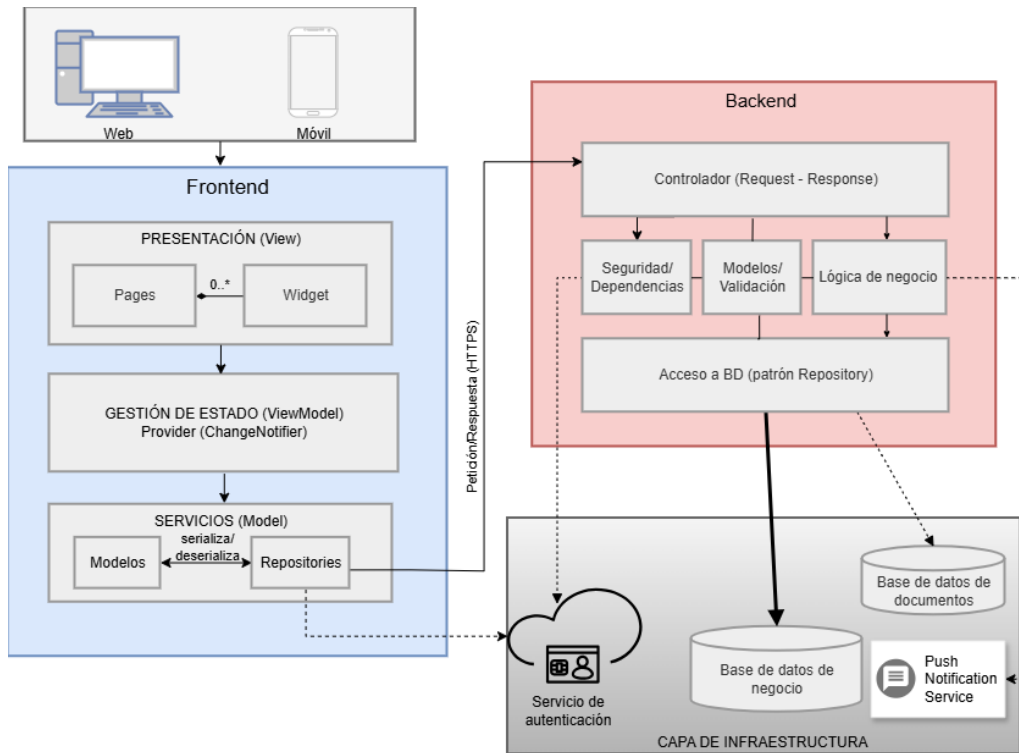


Figura 5.1: Diagrama de la arquitectura general por capas

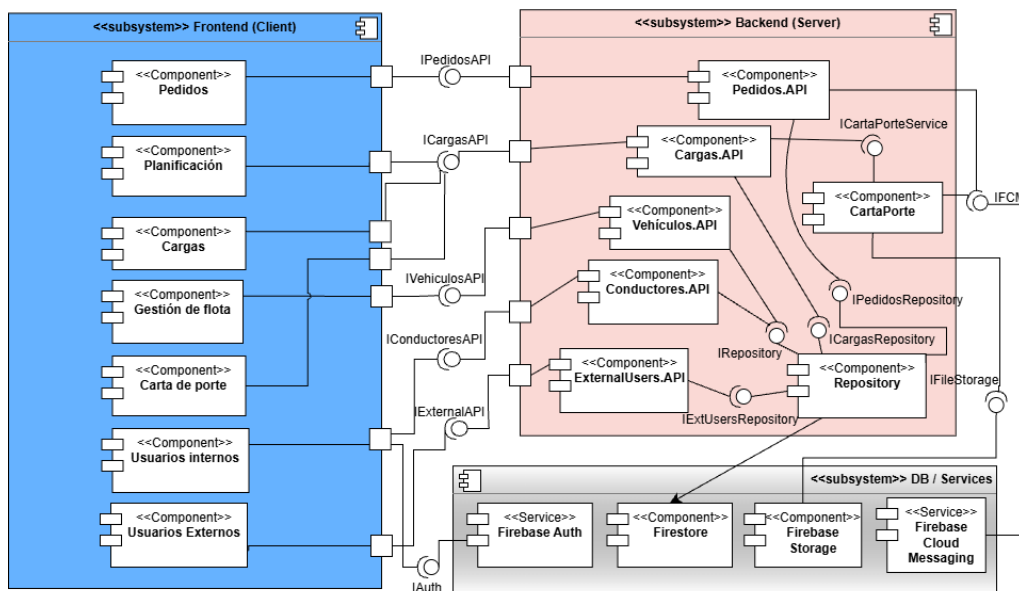


Figura 5.2: Diagrama de componentes UML del sistema desarrollado

5.1.1. Lado del cliente (Frontend)

Como se observa en la Figura 5.1, la parte de cliente incluye las capas de presentación y de gestión del estado. La capa de presentación solo se encarga de mostrar las interfaces al usuario, mientras que la capa de estado se ocupa de gestionar los datos temporales, la navegación y todos los datos necesarios para el correcto funcionamiento de la aplicación, pero que no requieren persistencia.

Para desarrollar esta segunda capa, se ha escogido el **patrón de diseño Provider**. El patrón Provider es un patrón de diseño popular en Flutter que aprovecha la estructura en árbol de las aplicaciones para desacoplar la lógica y el estado de la interfaz de la propia capa de presentación. Su funcionamiento es muy parecido al patrón *Observer* de otros lenguajes como Java. La clase principal es la clase Provider, cuya función es informar sobre los cambios que suceden en el estado de la aplicación a las interfaces que se suscriben a esa clase a través de una función *notifyListeners()*. Como se ve en la Figura 5.3, en la estructura en forma de árbol, las interfaces gráficas son las hojas del árbol y tienen acceso a las clases que se encuentran en niveles superiores a ellas. Además, es común que una misma interfaz reciba datos de varias clases proveedoras, separando las funcionalidades de la aplicación en distintos Providers que se ocupan de una única entidad. Si un *widget* solo necesita ejecutar una función de su proveedor, lee el provider mediante la función *context.read()*. Si, además de ejecutar funciones, el *widget* también necesita conocer el valor de alguna variable que se gestiona en ese proveedor, se suscribe a él mediante *context.watch()*.

Normalmente, los Providers que se utilizan en varias interfaces de la aplicación se crean como objetos globales en la raíz utilizando un *MultiProvider* que envuelve a los demás, ya que muchas interfaces requerirán sus datos y funciones. Los Providers que no tienen muchas interfaces suscritas se crean en el momento en que se crean las interfaces y se destruyen al mismo tiempo que el usuario navega fuera de la pantalla, para que no consuman recursos de manera innecesaria. A esto se le llama **Provider local**.

Gracias a este patrón de diseño, se consigue una correcta separación entre el manejo de estado y la capa de presentación. Además, los cambios solo requieren recrear las interfaces que están suscritas al proveedor y no la aplicación completa, mejorando el rendimiento global de la aplicación al evitar renderizados globales.

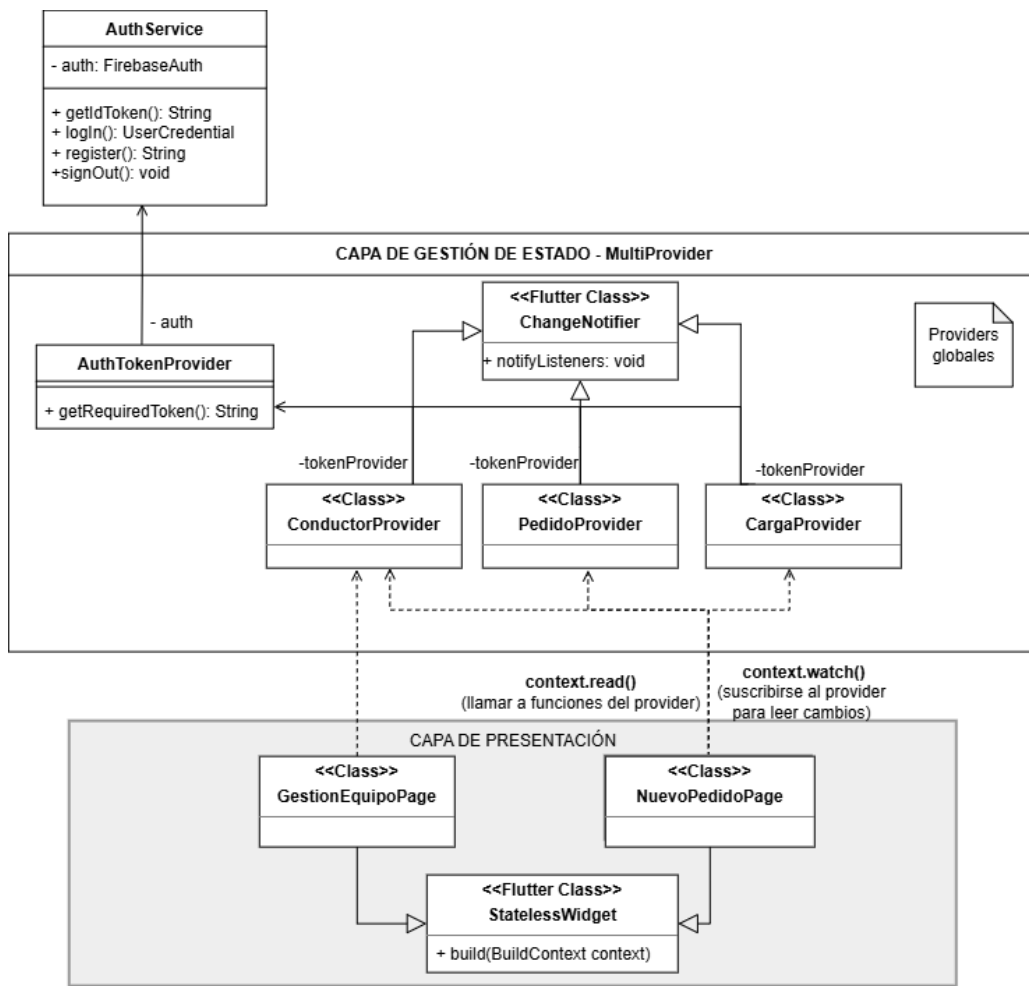


Figura 5.3: Diagrama de clases UML para la gestión de estado usando el patrón Provider

5.1.2. Lado del servidor (Backend)

El lado del servidor es el núcleo de la aplicación. Centraliza las reglas de negocio y el acceso a la base de datos. Tal y como se observa en la Figura 5.1, la estructura de esta capa se divide en varias partes encargadas de gestionar el flujo de las peticiones HTTP.

- **Controlador:** Es la parte donde se definen los *endpoints*. Son los componentes de la API encargados de recibir la petición (*Request*) del cliente, pasar la petición al siguiente módulo encargado de ejecutar la lógica y devolver la respuesta (*Response*) al cliente cuando se ha procesado la petición.
- **Servicios de seguridad y middleware:** Son los componentes que funcionan como requisitos para poder pasar el flujo desde el *router* al *service*. Son funciones que se utilizan principalmente para verificar la autenticación (validez del token JWT) y autorización

(privilegios suficientes) del usuario al llamar al *endpoint* y para verificar la integridad de datos de entrada. De esta manera, aseguran que el flujo esté en un entorno seguro y preconfigurado antes de pasar a la siguiente capa.

- **Servicios de aplicación:** Se encargan únicamente de la ejecución de los casos de uso y son independientes del resto del sistema. No conocen los detalles de la base de datos ni de ningún componente externo, lo que los hace muy fáciles de testear mediante testing unitario.
- **Acceso a BD:** Contiene las funciones que interactúan directamente con la base de datos.

Junto con estos componentes, la API utiliza un sistema de modelos (*schemas Pydantic*) encargado de definir la estructura de los datos que viajan en las peticiones. Cuando llega la petición, se encarga de **deserializar** el JSON de la petición tras validar que todos los campos obligatorios están presentes y en el formato correcto. En el flujo de salida, realiza la **serialización** del objeto, convirtiéndolo a JSON para que pueda ser transportado en la respuesta HTTP.

5.2. Diseño de la base de datos

El diseño de una base de datos NoSQL se diferencia bastante del diseño habitual de las bases de datos relacionales (SQL), esto se debe principalmente a las diferencias en la manera en la que se almacena la información físicamente. Mientras que en SQL se prioriza la normalización de las tablas y la obtención de información relacionada a través de operaciones de unión (*JOINS*), en las bases de datos NoSQL la normalización completa no es lo más adecuado y se opta por una arquitectura más flexible que se adapte directamente a la manera en que se leerán los datos.

Debido a la falta de *JOINS* en las NoSQL, el problema principal suele ser el conocido como **problema N+1**, donde para obtener datos relacionados de varias colecciones se necesitan realizar otras N lecturas, en vez de una simple lectura como ocurriría con un *JOIN*. Este problema afecta mucho el rendimiento de la aplicación, pero en Firestore tiene un peligro especial ya que el costo de esta base de datos depende del número de documentos devueltos, por lo que es muy importante diseñar la base de datos pensando en la optimización de las lecturas.

Para mitigar el problema, la técnica utilizada ha sido la **desnormalización**, esta técnica consiste en “duplicar” y anidar los campos que requieren una unión para que en una única lectura se tenga la información que se necesita.

En la Figura 5.4 se puede apreciar la relación entre los pedidos y las cargas. Mientras que en SQL los pedidos y las cargas irían en tablas separadas, aquí las cargas son una subcolección del pedido y además, las cargas tienen **el nombre** (desnormalización) **del conductor** asignado a la carga para poder mostrarlo con una lectura y prevenir las lecturas extras a la colección de usuarios para leer el nombre del conductor. Realizar esta desnormalización requeriría que

si el nombre del conductor cambia, en la carga que tiene ese conductor asignado el nombre también cambiase, pero como esto es un caso muy improbable se ha desactivado la opción de cambiar el nombre de los usuarios, por lo que este caso queda descartado.

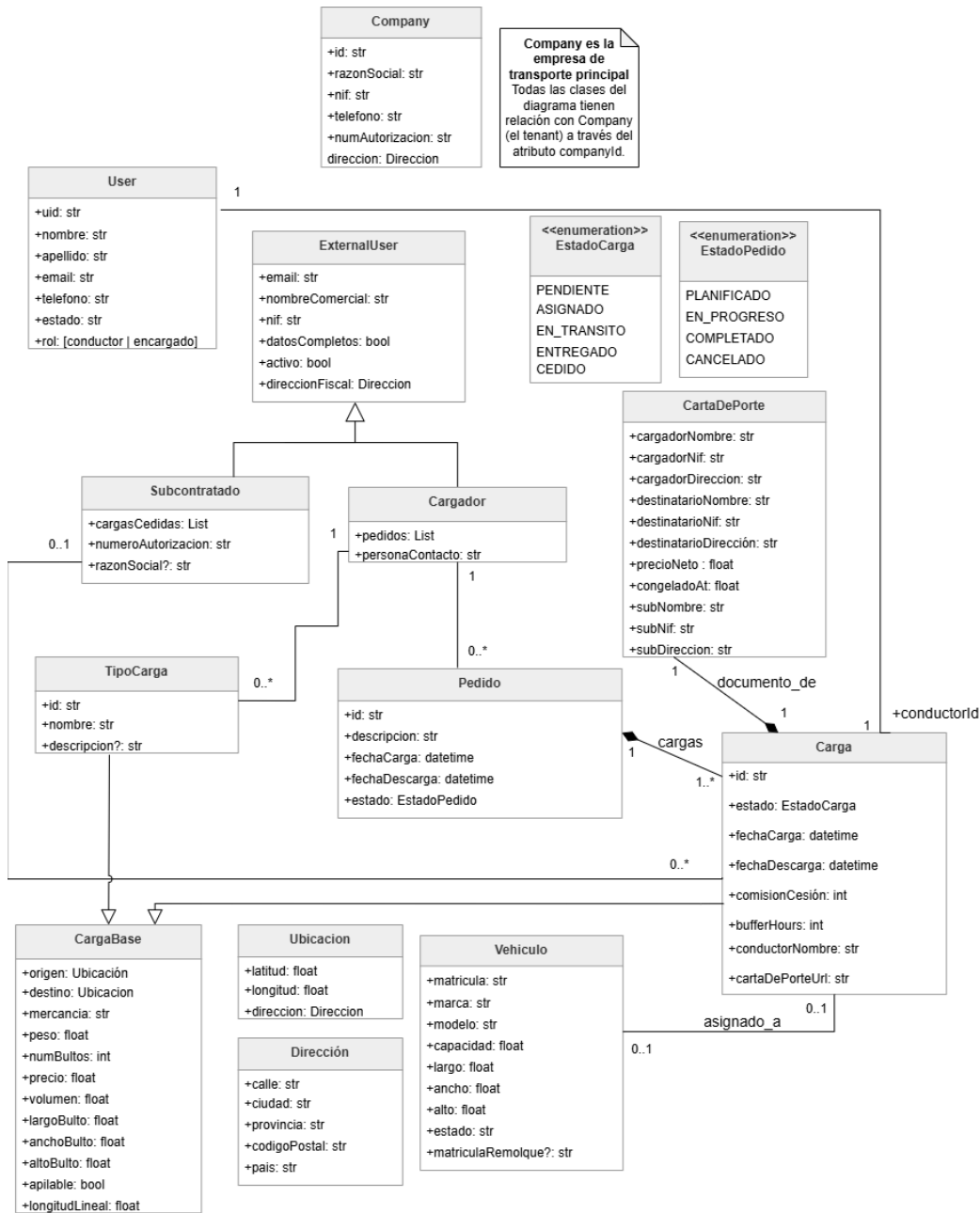


Figura 5.4: Modelo lógico de la Base de datos, mediante diagrama de clases UML

5.3. Diseño de la seguridad

Un buen nivel de seguridad es vital para cualquier aplicación que trate con datos de usuarios que pertenecen a diferentes empresas (**multi-tenant**). Cada usuario solo debe tener acceso a los datos de la empresa a la que pertenece y debe estar limitado a los datos dependiendo del nivel de privilegios que posee dentro del sistema. Para gestionar el acceso seguro, se diferencian la autenticación y la autorización.

5.3.1. Autenticación

Ya que se va a utilizar Firebase en el proyecto, la autenticación se delega en el servicio Firebase Authentication. Este servicio verifica la identidad del usuario que inicia sesión en la aplicación y, si el usuario existe en el sistema, se le concede un token JWT que se usará para realizar las llamadas a la API. De esta forma, no se tienen que guardar las credenciales directamente en la base de datos. Cuando un usuario se registra en el sistema, las credenciales guardadas en Firebase Auth y los datos necesarios para la lógica de negocio, que se guardan en Firestore, se vinculan utilizando el mismo **UID** (user ID). Así, el sistema puede recuperar los datos del usuario en futuros inicios de sesión a través de ese UID.

5.3.2. Autorización

Una vez que se consigue el token JWT en el paso de la autenticación, la autorización ocurre en el backend. Primero, para dar acceso únicamente a los datos de la propia empresa del usuario y dentro de los privilegios que le otorga su rol, se utilizan los **custom claims** en el token. Los *custom claims* son campos útiles para la autorización que se añaden directamente al *payload* del token; en este caso, se han incluido el *company_id* y el rol del usuario. Cuando se llama a la API, las dependencias del endpoint validan el *company_id* y el rol y deniegan el acceso a las peticiones si el usuario no tiene un rol con suficientes privilegios o no contiene el *company_id*.

Para que el backend interactúe con los servicios de Firebase, se utiliza el SDK *Firebase Admin*. Este SDK otorga privilegios de administrador para acceder al proyecto en la nube, por lo que las credenciales se obtienen de las variables de entorno del proyecto.

5.3.3. Otros aspectos de seguridad

Además de la autenticación y la autorización de los usuarios de la aplicación, el sistema también incluye otros aspectos de seguridad que lo protegen de usuarios externos malintencionados.

5.3.3.1. Políticas de origen cruzado (CORS)

Como la aplicación tiene soporte para clientes web, se han configurado las políticas CORS en el backend para las respuestas de la API solo sean accesibles desde orígenes autorizados. Esta configuración se implementa a través de un middleware de FastAPI.

5.3.3.2. Límite de peticiones (Rate Limiting)

Es un mecanismo que limita la cantidad de peticiones que puede hacer un usuario en un tiempo determinado, para evitar usos abusivos contra la API. Se implementa con un *RateLimitMiddleware*.

5.3.3.3. Cifrado de datos

En cuanto al cifrado de datos, todas las comunicaciones entre los clientes y el servidor se cifran a través de HTTPS para prevenir ataques de **Man In the Middle (MITM)**. Además de este sistema de cifrado, Google cifra todos los datos almacenados en Firestore utilizando el cifrado AES-256.

5.4. Diseño de la API

La comunicación entre cliente y servidor sigue una **arquitectura RESTful** (*Representational State Transfer*). Esta arquitectura define varias reglas en la manera en la que se tiene que dar la comunicación entre el cliente y el servidor. La regla principal es que la comunicación es sin estado (*Stateless*) y cada petición debe contener toda la información que requiere el servidor: cabeceras HTTP, parámetros y cuerpo.

Los endpoints de la API se definen en la capa de controladores o *Routers*. Cada *Router* define los endpoints para una única entidad del dominio.

Ruta	Descripción
/auth	Registro de los datos de usuarios,
/trans	CRUD de chóferes.
/vehiculos	CRUD de vehículos
/external_users	Invitación y gestión de invitados (cargador y subcontratado)
/pedidos	Creación y obtención de pedidos (propios o del cargador).
/cargas	Planificación, subcontratación, creación de tipos de carga, creación de incidencias en las cargas, etc.
/dashboard	Consultas agregadas para el panel de control.

FastAPI permite generar documentación técnica completa de manera automática sobre los endpoints implementados utilizando la especificación OpenAPI, esta documentación se puede consultar en el Anexo [E.2](#).

5.5. Diseño de las interfaces de usuario multiplataforma

5.5.1. Componentes principales

Para resolver las necesidades comunes de los distintos actores y facilitar la implementación, el diseño se ha estructurado en base a componentes reutilizables:

- **Patrón *Dashboard* (Panel de Control):** Para el encargado, se ha diseñado una estructura clásica de panel de control, compuesta por una barra de navegación lateral y el contenido principal, donde aparecen el calendario de visualización (reutilizado por el conductor en su vista) y el panel de incidencias. En la cabecera de esta vista se utilizan tarjetas resumen que muestran indicadores sobre el estado de las cargas y las incidencias (véase la Figura 5.5a).
- **Calendarios:** Para el módulo de planificación, se ha diseñado una interfaz basada en un calendario de tipo Gantt (véase la Figura 3.8), que permite al usuario interactuar con las cargas individualmente, arrastrando elementos para ajustar las fechas.
- **Modales en pasos (*Wizard*):** Para procesos largos, como la inserción de un nuevo pedido y sus cargas, se utilizan modales divididos en pasos lógicos (por ejemplo: inserción de datos del destinatario, selección de cargas y por último, asignaciones). Esto guía al usuario progresivamente y permite ir validando los datos de manera más gradual. La Figura 3.12 muestra un modal de este tipo, para el caso de uso de crear un nuevo pedido de transporte.
- **Tablas:** Para mostrar información de manera visual y clara, se ha utilizado el mismo componente de tabla con acciones de editar y eliminar filas. Es un componente muy reutilizado que se utiliza para mostrar los vehículos y conductores de la empresa al encargado, mostrar los pedidos creados al cargador y las cargas cedidas al subcontratado.

5.5.2. Diseño responsive

Flutter es un framework conocido por permitir crear interfaces atractivas y de una consistencia visual idéntica en todas las plataformas. Al ser una aplicación implementada para que se pueda acceder desde dispositivos de distinto tamaño, es indispensable que las interfaces sean *responsive*. Teniendo esto en cuenta, se han tomado las siguientes decisiones de diseño para que la implementación *responsive* sea más sencilla:

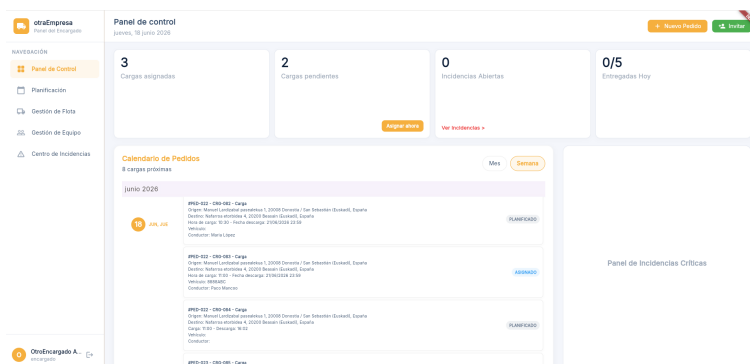
- Se ha decidido poner la barra de navegación en la parte izquierda, con el contenido de las páginas en la derecha. Esto permite que la barra se minimice cuando se esté utilizando el móvil, para mostrar todo el contenido en la pantalla. Esto se puede observar en la Figura 5.5.
- El uso de tablas en móviles no suele ser muy cómodo, por lo que en móviles se ha decidido utilizar listas verticales aprovechando el scroll vertical natural de estos dispositivos.

5.5. Diseño de las interfaces de usuario multiplataforma

Esta decisión es extrapolable a todas las interfaces, donde en móviles se ha decidido aprovechar la distribución vertical con scroll, mientras que en tamaño de escritorio se utiliza una distribución más horizontal, normalmente sin scroll. Esta decisión se puede observar en la Figura 5.6.

- Se ha decidido reutilizar las tablas y calendarios entre las pantallas de los distintos usuarios para mantener un diseño homogéneo en las interfaces de todos los usuarios.

Para ver la diferencia entre la aplicación en dispositivos móviles y ordenadores, se muestran dos ejemplos de las mismas interfaces finales del proyecto en tamaño *desktop* y *mobile*. Las interfaces móviles que se muestran son las de un dispositivo móvil real. En la Figura 5.5, se puede observar cómo la barra de navegación se cierra para el móvil y la distribución pasa a ser más vertical, reorganizando las tarjetas en una rejilla 2x2 y apilando el calendario y la lista de incidencias. Por otro lado, la Figura 5.6 muestra la conversión de tabla a lista al pasar de escritorio a móvil. Las listas optimizan la visualización en móviles al adaptarse al flujo vertical del *scroll*.



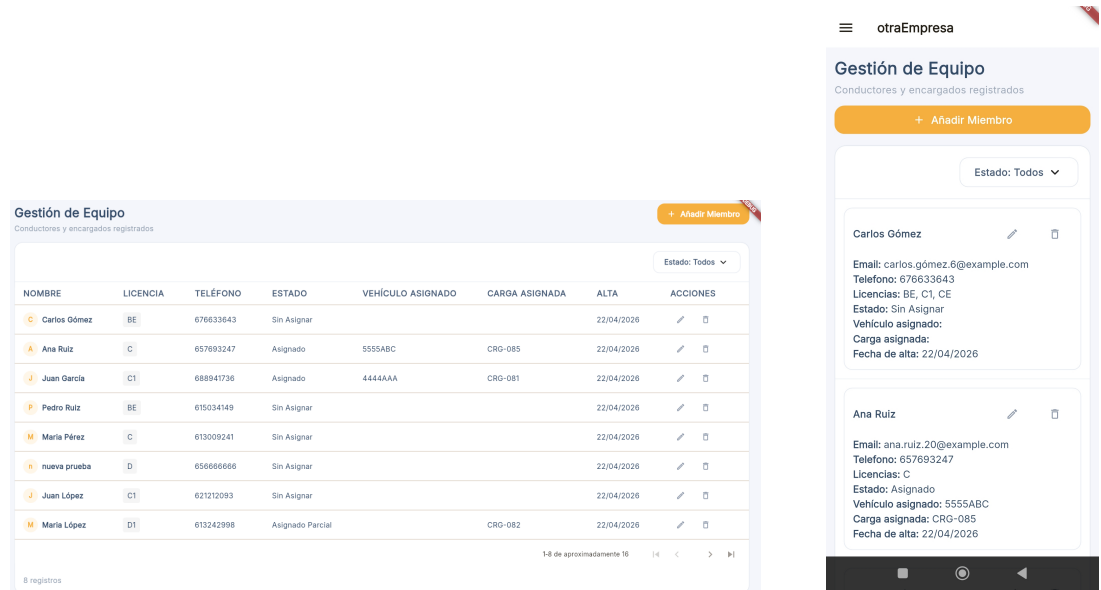
(a) Pantalla de escritorio.



(b) Pantalla de móvil.

Figura 5.5: Ejemplo de interfaz responsive

5. DISEÑO



(a) Pantalla de escritorio.

(b) Pantalla de móvil.

Figura 5.6: Ejemplo de conversión de tabla a lista

5.5.3. Principios de diseño

El diseño de las interfaces de usuario se realizó antes de empezar con las fases de desarrollo. Debido a que es un proyecto más orientado a la implementación y que hacer diseños para todas las interfaces manualmente es costoso, probablemente la tarea de diseño se saldría del tiempo planificado. Por esta razón, se decidió que lo más inteligente sería definir unos principios y reglas de diseño y utilizar la herramienta *Figma Make* para que realizase los primeros diseños a partir de esas reglas.

Como consta en la [Declaración del uso de IA generativa](#), esta herramienta utiliza inteligencia artificial para generar los diseños. A partir del primer resultado, los diseños se fueron refinando para que se pudiesen implementar todos los casos de uso definidos.

Los principios de diseño han sido:

- **Paleta de colores:** La aplicación solo tiene “modo claro”. Se ha escogido un tono blanco (#FFFFFF) para el fondo y un color gris azulado para elementos como títulos (#0E203A), subtítulos (#9AA6B6) y texto (#344B68). El color principal de la aplicación es un naranja claro (#F5AF3E). Para las incidencias, se ha escogido el color rojo (#EF4444) para resaltar la advertencia.
- **Tipografía:** Se ha seleccionado la tipografía Inter, debido a su gran legibilidad en pantallas.

- **Minimalismo:** Se ha querido mantener la interfaz lo más sencilla posible para facilitar la futura implementación y un uso sencillo de la aplicación. Los contrastes se han limitado a los clásicos como el rojo para advertencias y distintos colores para distinguir los estados de las cargas (pendiente, planificado, entregado, etc.).

5.6. Diagramas de secuencia

En esta sección se muestran los diagramas de secuencia de varios casos de uso relevantes de la aplicación, introducidos en el Capítulo 3.

Los diagramas de secuencia incluidos en este apartado son los siguientes:

- Inicio de sesión (Figura 5.7).
- Registro de usuario (Figura 5.8).
- Dar de alta a un usuario (Figura 5.9).
- Dar de baja a un usuario (Figura 5.10).
- Creación de pedido (Figura 5.11).
- Generación de carta de porte (Figura 5.12).

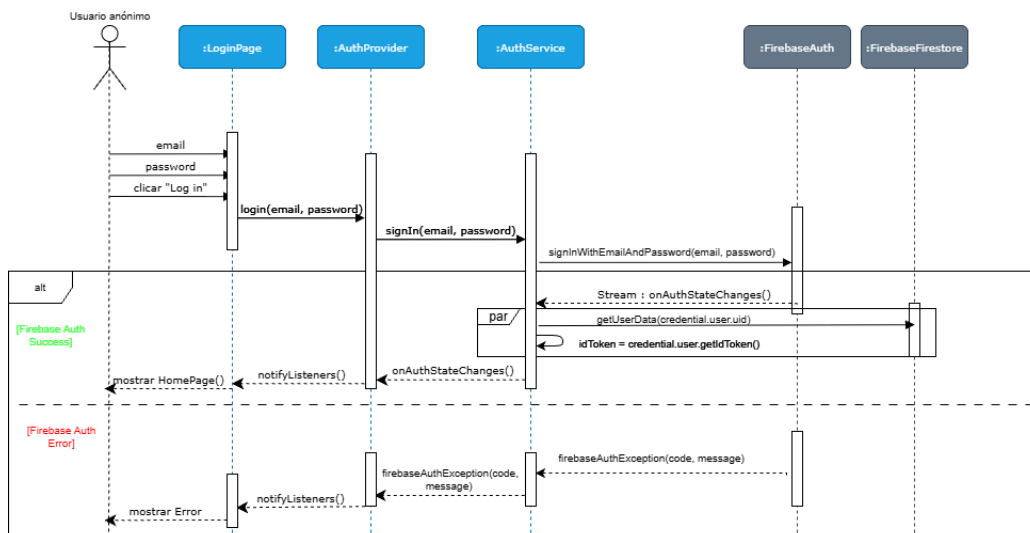


Figura 5.7: Diagrama de secuencia de inicio de sesión

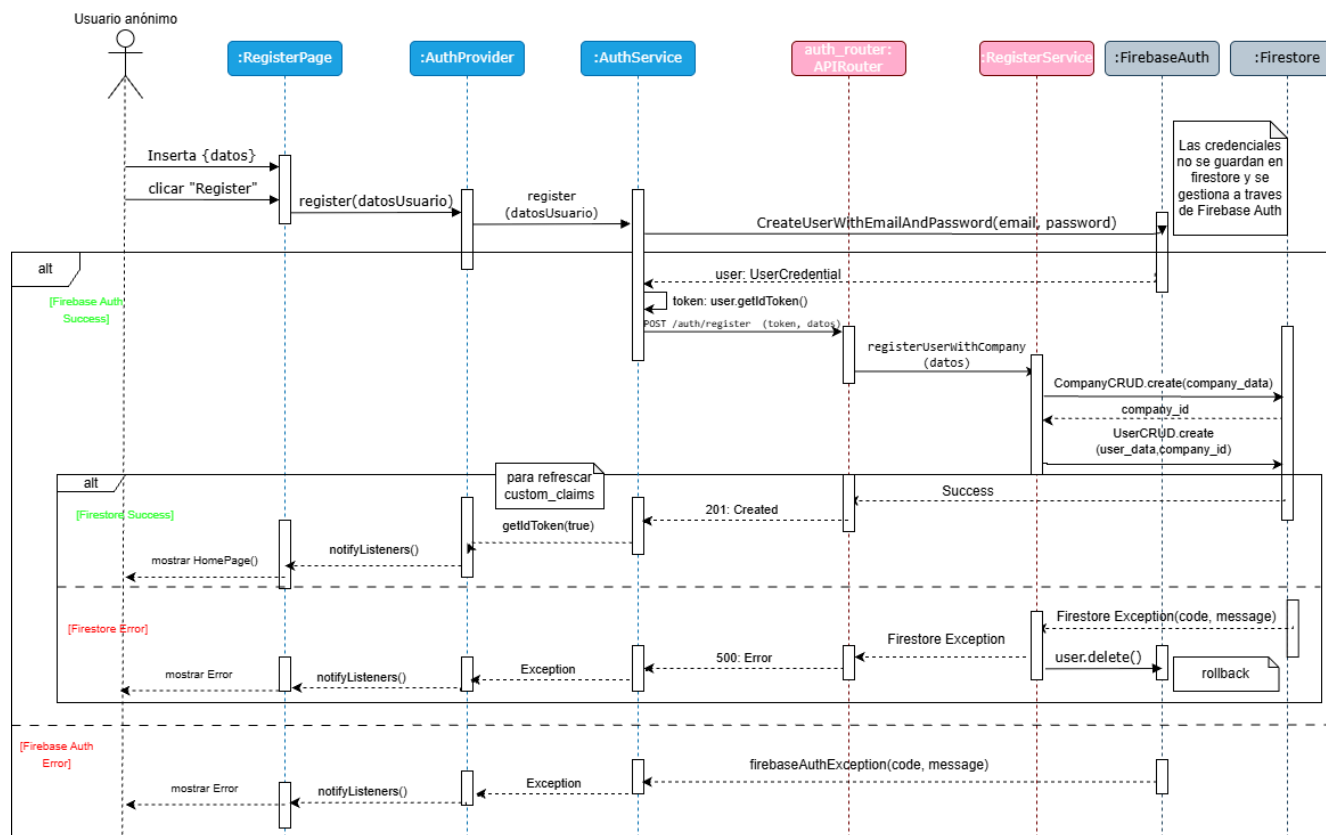


Figura 5.8: Diagrama de secuencia de registro de usuario

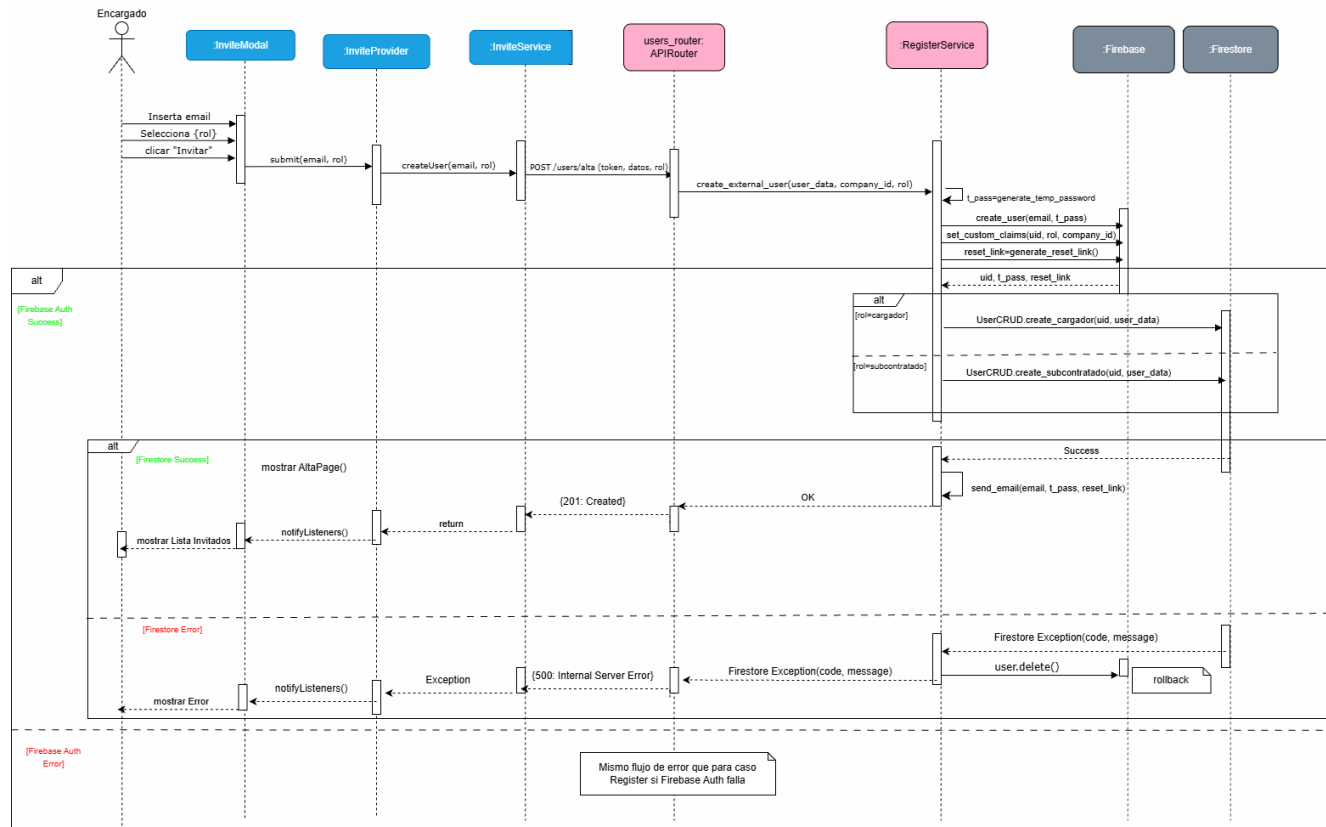


Figura 5.9: Diagrama de secuencia de dar de alta a un usuario

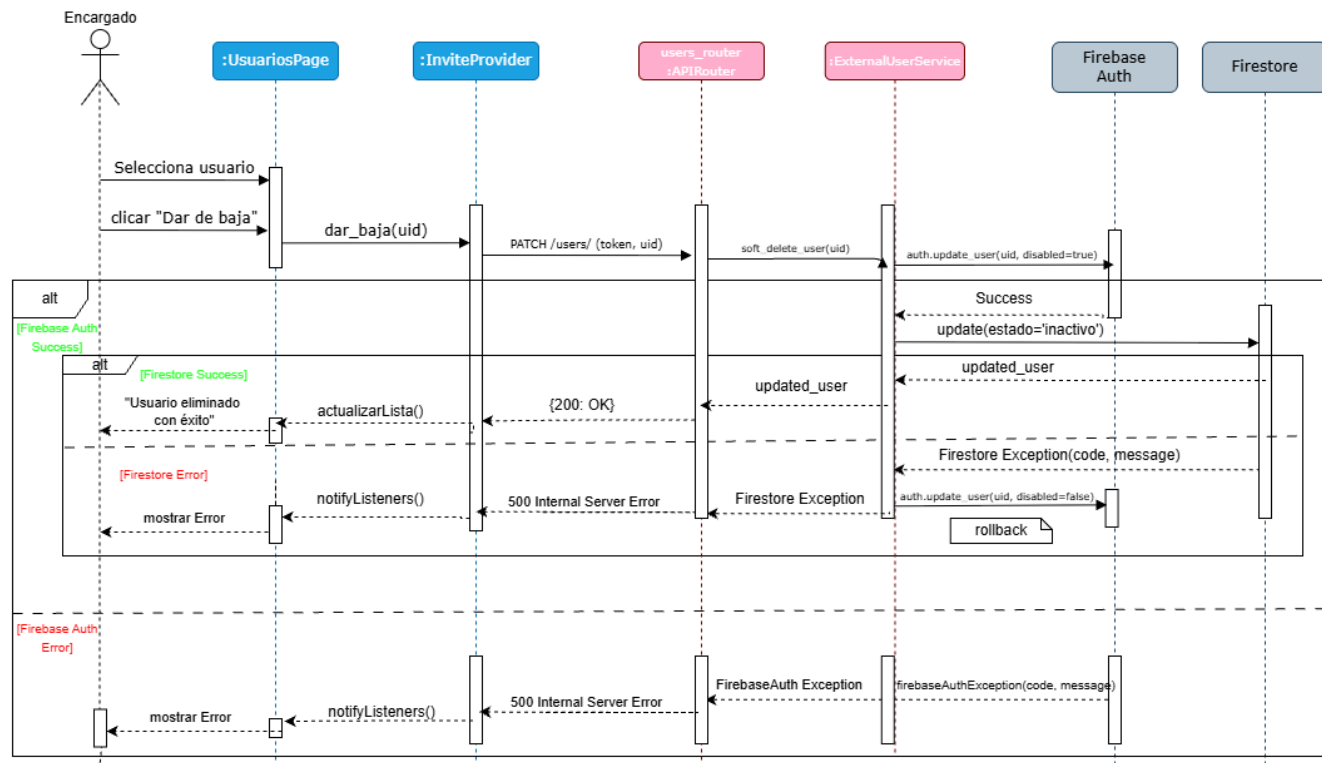


Figura 5.10: Diagrama de secuencia de dar de baja a un usuario

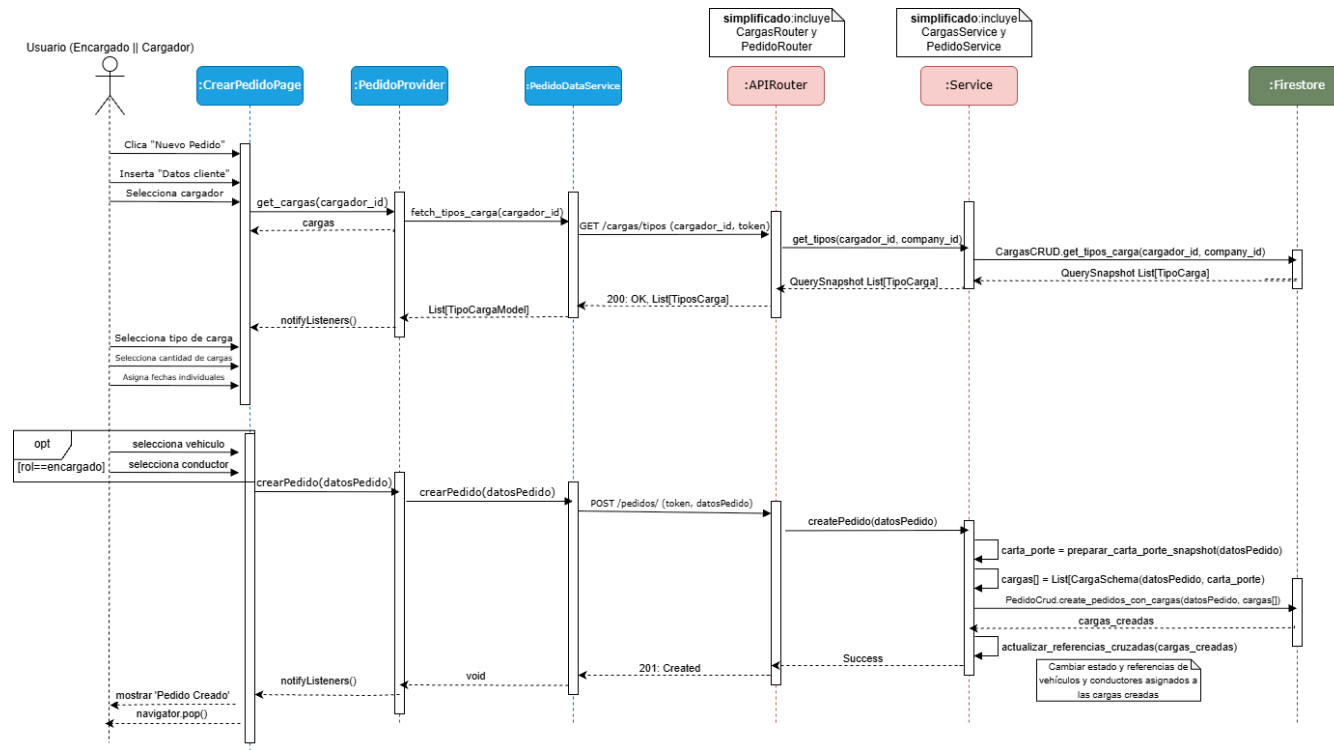


Figura 5.11: Diagrama de secuencia de creación de pedido

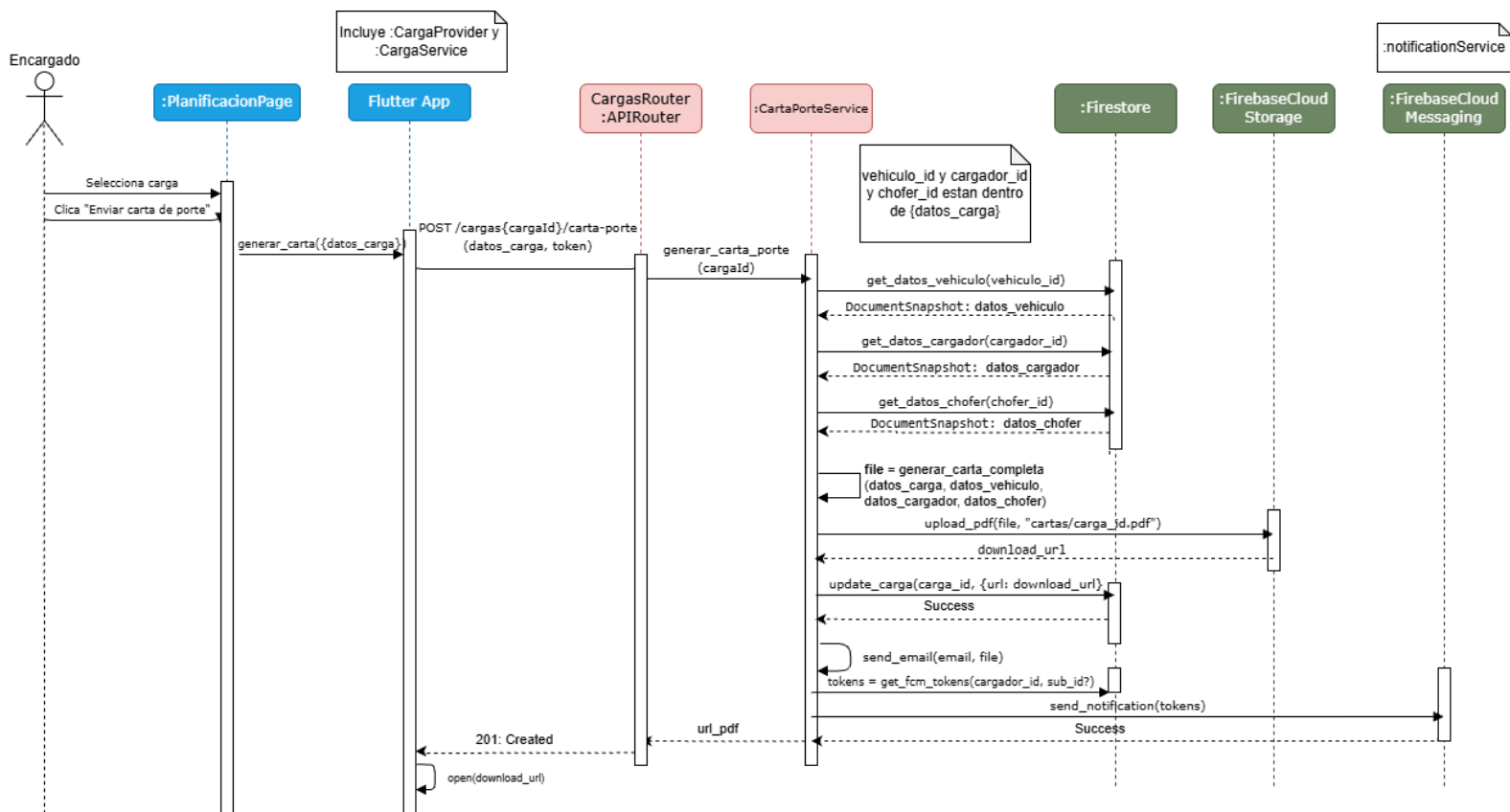


Figura 5.12: Diagrama de secuencia de generación de carta de porte

Implementación

En este capítulo se detalla el proceso de implementación de la aplicación. Se describe la configuración inicial del proyecto y de la capa de infraestructura, y se detallan las implementaciones de los módulos funcionales mencionados en el apartado de Requisitos de una manera más técnica y detallada. Toda la implementación se puede encontrar en el repositorio del proyecto¹.

6.1. Configuración inicial del proyecto

Antes de empezar con la implementación, se configuró el proyecto siguiendo buenas prácticas de desarrollo en aplicaciones Flutter.

6.1.1. Estructura del proyecto

Cuando se crea un proyecto en Flutter, se generan automáticamente los directorios con los archivos de configuración que permiten compilar la aplicación en todos los sistemas operativos de escritorio, de móvil y en la web. El código fuente propio de la aplicación se guarda en el directorio `/lib`. Existen varias formas de estructurar los directorios y archivos en Flutter dependiendo del tamaño del proyecto. Para mantener una buena escalabilidad del proyecto a futuro, se escogió una estructura que organiza las **carpetas por funcionalidades (feature-first)** dentro de `/lib`. Cada directorio agrupa todos los componentes (interfaces, *providers*, etc.) relacionados con esa funcionalidad y se mantiene una carpeta *core* donde se agrupan los componentes que son reutilizables por el resto de funcionalidades.

¹https://github.com/haritz99/TFG_Gestion_Transporte

Frontend

lib/	Código fuente del frontend
├── core/	Componentes compartidos
│ ├── models/	Modelos de datos
│ ├── widgets/	Componentes visuales reutilizables
│ └── ...	
├── features/	
│ ├── auth/	Autenticación de usuarios
│ │ ├── auth_provider.dart	Gestor de estado de autenticación
│ │ ├── auth_service.dart	Comunicación con Firebase Auth
│ │ └── ui/	Interfaces de autenticación (login/register)
│ ├── cargas/	Gestión de cargas
│ ├── dashboard/	Panel de control e indicadores
│ ├── external/	Gestión de usuarios externos
│ ├── plan/	Planificación de las cargas
│ └── ...	
└── app.dart	Punto de entrada de la aplicación
└── tests/	Pruebas

6.1.2. Flavors de Flutter

Una vez configurada la estructura de carpetas, el siguiente paso fue crear las versiones de desarrollo (*dev*), *staging* y producción del proyecto. Para hacer esto, Flutter tiene su **sistema de flavors**, donde cada *flavor* representa un entorno distinto con su propia configuración. Cada *flavor* hace que la aplicación apunte a un proyecto de Firebase distinto para mantener los datos y los usuarios de prueba aislados de los datos y usuarios de la aplicación desplegada.

Esta separación también sirve para que las versiones *dev* y *staging* apunten a *localhost* y la versión de producción apunte al servidor desplegado, en vez de cambiarlas a mano cuando se despliega la aplicación.

Los *flavors* son independientes del sistema operativo (el mismo flavor sirve para Android, iOS y web), pero cada uno requiere una configuración en cada plataforma. Para hacer estas configuraciones se utilizó la herramienta `flutter_flavorizr`, que genera los archivos necesarios de cada plataforma. En el lado del servidor, la API utiliza el *Firebase Admin SDK*, que permite interactuar con el proyecto Firebase correspondiente a cada *flavor* desde un entorno con privilegios de administrador.

6.2. Implementación de la capa de infraestructura y datos**6.2.1. Firebase Authentication**

Como se ha descrito en el apartado de Diseño, no es el backend el que se encarga de gestionar **la autenticación** del sistema, sino que se utiliza el servicio de autenticación de Firebase. Este servicio está pensado para utilizarse directamente desde el cliente, pasar por

el backend sería un paso extra innecesario en este caso. El servicio se encarga de generar el **token JWT** cuando un usuario se registra y vincularlo a las credenciales correspondientes. Cuando el usuario inicia sesión, el flujo se gestiona a través de una comunicación de tipo *Stream* (flujo continuo) con el servicio y cada vez que el estado de autenticación cambia para un usuario, el método `_onAuthStateChanges()` recupera en paralelo el token JWT y los datos de negocio del usuario. Al haber una comunicación entre el cliente y Firestore, se han definido las reglas de seguridad necesarias para que el usuario solo pueda leer su documento. En la siguiente función se puede observar el flujo de autenticación simplificado.

```
1 _authService.authStateChanges.listen((firebaseUser) async {
2   if (firebaseUser != null) {
3     final results = await Future.wait([
4       _authService.getUserData(firebaseUser.uid), // esto obtiene los datos del user
5       _authService.getIdToken(),                 // esto solo obtiene el token
6     ]);
7     _user = results[0] as UserModel?;
8     _idToken = results[1] as String?;
9   }
10  notifyListeners();
11 });
```

Código fuente 1: Suscripción al estado de autenticación

Como se definió en el apartado 5.3.2, la gestión de los roles y la separación de *tenants* ocurren con los `custom_claims`. Para implementar la inyección de los `custom_claims` del usuario en el token de autenticación, en el proceso de registro del usuario (tanto del encargado como de los usuarios invitados) se insertan el **company_id** y el **rol** directamente en el token JWT. De esta manera, para futuras consultas a la API ya existen esos datos en el *payload* del token JWT y no hay que hacer consultas extra a la base de datos.

```
1
2 company_id = self._company_crud.create(company_data)
3
4 firebase_auth.set_custom_user_claims(uid, {
5   'companyId': company_id,
6   'rol': register_data.rol,
7 })
```

Código fuente 2: Función que asigna los `custom_claims` al usuario encargado

Para prevenir que un usuario ilegítimo pueda asignarse un `company_id` que no le pertenece, esta función debe ejecutarse en el servidor, usando datos que el propio servidor genera (como ocurre en el ejemplo con `company_id`). Al registrar a un usuario externo, el rol lo seleccionará el encargado y el `company_id` será el del encargado que lo invita.

- **Validación de Esquema:** Antes de ejecutar el endpoint, RegisterRequest valida la estructura de los datos de registro, rechazando peticiones malformadas con un error HTTP 422.
- **Autenticación previa:** El *endpoint* de registro no crea la cuenta de Firebase Auth, el cliente invoca primero el SDK de Firebase Auth (`createUserWithEmailAndPassword`) para dar de alta al usuario y obtener un *token* válido, y es ese *token* el que se envía en el *header* de la petición a `/register`.
- **Delegación de lógica:** con la entrada validada y el uid verificado, el flujo pasa al componente de lógica de negocio, que genera en el servidor el `company_id` y crea la empresa, el documento de usuario y los *custom claims* vinculados a ese uid.

Una vez que los `custom_claims` quedan definidos en el registro, la autorización en las peticiones se resuelve comprobando esos claims mediante dependencias de FastAPI. Para realizar esta validación, se utiliza una función que encapsula una función interna y la devuelve configurada dependiendo del rol requerido. Esta función resultante es la que se pasa después como dependencia del endpoint con `Depends()`.

```
1 def _require_role(required_role: str):
2     async def role_checker(current_user: dict = Depends(get_current_user)) -> dict:
3         roles = normalize_roles(current_user.get("rol"))
4         if required_role not in roles:
5             raise HTTPException(status_code=403)
6         return current_user
7     return role_checker
8
9 get_current_encargado = _require_role("encargado")
10 get_current_conductor = _require_role("conductor")
```

Código fuente 3: Implementación de autorización por rol

6.2.2. Firestore

Mientras que en el capítulo de diseño se describió la estrategia de desnormalización para prevenir el problema de N+1 lecturas en Firestore, en esta sección se detallará cómo se ha llevado a cabo la implementación del backend para prevenir el problema más común al desnormalizar bases de datos: la **inconsistencia de los datos**. Ya se ha comentado en la sección de diseño de la base de datos que para prevenir este problema no se permite editar el nombre del conductor, por lo que las cargas **siempre tendrán el nombre correcto** del conductor asignado. Para asegurar que los documentos JSON de Firestore coincidan exactamente con la arquitectura del sistema, se han implementado clases de Dart en Flutter y de Pydantic en Python para todas las entidades del diagrama, los modelos de Pydantic serializan y deserializan los datos automáticamente y en Dart se utilizan los métodos **fromJson** y **toJson**. Los modelos

tienen tipado fuerte, por lo que todas las operaciones de escritura pasan por el filtro de tipos y no habrá errores estructurales.

Hay casos en los que hay que actualizar los datos de manera atómica, el caso más común es cuando se actualizan las fechas o asignaciones de las cargas de un pedido en la planificación. Esta actualización se hace utilizando **Batches** de Firestore, evitando que un pedido quede con algunas de sus cargas actualizadas y otras no, lo que generaría un estado intermedio inconsistente difícil de detectar. A continuación se puede observar un ejemplo de una actualización atómica de varias cargas.

```
1 def bulk_update_cargas(self, cargas: List[CargaSchema]) -> List[CargaSchema]:
2     batch = self._crud.get_batch()
3     for carga in cargas:
4         ref = self._crud.get_carga_ref(carga.id)
5         batch.update(ref, carga.model_dump(exclude={'id'}))
6     batch.commit()
7     return cargas
```

Código fuente 4: Actualización atómica utilizando Batches de Firestore

6.3. Implementación del frontend

6.3.1. Implementación del sistema MultiProvider

Para implementar el sistema de Providers, se utiliza la clase de Flutter MultiProvider en la raíz de la aplicación para agrupar a todos los Providers en el mismo nivel. Cada proveedor solo gestiona el estado global de una entidad en toda la aplicación, pero hay casos donde un proveedor puede depender de algún valor de otro proveedor. Esto en la aplicación ocurre con el proveedor que gestiona el token de autenticación (AuthTokenProvider). Todos los proveedores necesitan el token para hacer llamadas válidas al backend y si el token de autenticación cambia (porque ha habido un inicio/cierre de sesión o porque Firebase lo ha cambiado), los proveedores deben tener el nuevo token sin tener que actualizarlo individualmente en cada proveedor. Para gestionar esto de forma limpia, se usa un **ChangeNotifierProvider** que inyecta el AuthTokenProvider en cada proveedor.

6. IMPLEMENTACIÓN

```
1      @override
2      Widget build(BuildContext context) {
3          return MultiProvider(
4              providers: [
5                  // Creación del servicio de autenticación que se inyecta en AuthTokenProvider
6                  Provider<AuthService>(create: (_) => AuthService()),
7                  Provider<AuthTokenProvider>(
8                      create: (context) => AuthTokenProvider(context.read<AuthService>()),
9                  ),
10                 // Se crean los Providers de negocio con el AuthTokenProvider inyectado
11                 ChangeNotifierProvider<CargaProvider>(
12                     create: (context) => CargaProvider(tokenProvider:
13                         ↪ context.read<AuthTokenProvider>()),
14                 ),
15                 ....
16             ]
17         )
18     }
```

Código fuente 5: Implementación de MultiProvider

El MultiProvider completo se puede encontrar en el Anexo C. Como se ha comentado en el capítulo de Diseño (5.1.1), este sistema permite a las interfaces suscribirse al estado de la aplicación para poder mostrar la información de manera reactiva. Sin embargo, para entender cómo esta información se muestra de manera correcta en todos los dispositivos que abarca el proyecto multiplataforma, hay que entender cómo se implementan las interfaces de manera responsiva en Flutter.

6.3.2. Implementación responsiva de interfaces de usuario

La aplicación debe adaptarse principalmente a dos rangos de pantalla distintos: móvil y escritorio. Para conseguir esto, la agrupación de los componentes tiene que poder variar en función del tamaño disponible (véanse Figuras 5.5 y 5.6). Para ello se utiliza la librería *responsive_framework*, que ayuda a mantener una estructura general responsiva definiendo rangos de tamaños de pantalla para cada dispositivo. La transición de la estructura se controla a través de un condicional que detecta cuándo el ancho del dispositivo cruza el umbral (breakpoint) definido para la interfaz móvil. Además, Flutter cuenta con componentes que ayudan a que las interfaces sean responsivas de manera más automática. Los más utilizados en la aplicación han sido los siguientes:

- **LayoutBuilder:** Expone las restricciones de tamaño que impone el widget padre a sus hijos. Sirve para decidir cómo agrupar los elementos cuando lo que interesa es el tamaño máximo del padre, sin importar el tamaño completo de la pantalla. En la aplicación se utiliza para calcular el tamaño real de las páginas, quitando la barra de navegación, y así poder decidir cuándo pasar de una agrupación horizontal a una vertical.

- **Expanded:** Hace que los componentes ocupen todo el ancho disponible de su elemento padre. Si hay varios elementos de este tipo dentro del mismo padre, se reparten el ancho y se puede restringir qué porcentaje ocupa cada hijo utilizando el parámetro *flex*.
- **Flexible:** Es parecido a Expanded, pero permite que el elemento ocupe su espacio natural si es más pequeño, sin forzarlo a que ocupe todo el ancho.
- **Wrap:** Es como un Row, que agrupa los elementos en el eje horizontal, pero cuando un elemento no cabe por completo, lo pasa a la siguiente línea automáticamente. Ayuda a que los elementos nunca se desborden por la derecha cuando el tamaño pasa a ser de dispositivo móvil.

En el siguiente fragmento se puede observar el sistema de *breakpoints* y un ejemplo de cómo se crean interfaces responsivas en Flutter.

```

1  final isMobile = ResponsiveBreakpoints.of(context).isEqualTo(MOBILE);
2
3  if (isMobile) {
4    return Scaffold(drawer: Drawer(child: AppSidebar()), body: child);
5  }
6
7  return Scaffold(
8    body: Row(children: [AppSidebar(), Expanded(child: child)]),
9  );

```

Código fuente 6: Ejemplo de uso de *responsive breakpoints* en Flutter

6.4. Implementación del módulo de gestión de tráfico

En esta sección se detallarán las implementaciones de funcionalidades relevantes del módulo principal de la aplicación. En este módulo, el usuario central es el encargado de tráfico, aunque los usuarios Cargador y Subcontratado también colaboran de forma directa o indirecta en varias funcionalidades. En la Tabla 6.1 se muestra un resumen de lo implementado por submódulos.

Funcionalidad	Frontend	Backend
Creación del pedido	Formulario tipo <i>wizard</i> en varios pasos que recopila datos del pedido, tipo de carga y permite una primera asignación de conductor y vehículo	Valida las reglas de negocio, congela el snapshot de carta de porte y persiste el pedido en Firestore

6. IMPLEMENTACIÓN

Funcionalidad	Frontend	Backend
Realizar planificación	Calendario interactivo que calcula disponibilidad de conductores y vehículos en base a colisiones horarias y permite ceder cargas a subcontratados	Persiste la asignación de fecha, conductor y vehículo. Si se subcontrata, gestiona el precio y actualiza los datos de la carta de porte, añadiendo el nuevo usuario.
Generación de carta de porte	Descarga y visualización del PDF adaptada a web y móvil (clase abstracta: pdf_handler.dart)	Renderiza la plantilla HTML con los datos de la carga, genera el PDF y lo sube a Firebase Storage

Tabla 6.1: Resumen de funcionalidades implementadas en el módulo de gestión de tráfico

Creación del pedido

La creación del pedido es el punto de entrada para la entidad de Carga. La Tabla 6.2 recoge los archivos de la implementación de este caso de uso. También indica la capa a la que pertenece cada archivo, las líneas de código y su funcionalidad en la aplicación.

Capa	Archivo	LOC	Descripción
UI	form_builder.dart	256	Agrupar los widgets de la interfaz descritos a continuación
UI	datos_cliente.dart	68	Inserción de datos del cliente
UI	nuevo_pedido.dart	535	Selección de cargador, tipo de carga y cantidad
UI	nuevo_tipo_carga.dart	242	Creación de un nuevo tipo de carga que aún no existía
UI	seleccionar_cargas.dart	147	Primera asignación de chófer y vehículo
Provider	pedido_provider.dart	217	Guarda datos temporales del formulario
Provider	carga_provider.dart	-	Tipo de carga acordado (285-301), cálculo de disponibilidad para seleccionar_cargas.dart (111-145) y (147-180)
Provider	invite_provider.dart	54-66	Obtiene cargadores dados de alta para nuevo_pedido.dart
PedidosService (backend)	pedido_service.py	152	Verificación de reglas de negocio, preparación de datos de carta de porte
Repository	pedidos_crud.py	26-63	Inserción del pedido en Firestore

Tabla 6.2: Detalle de archivos - Creación del pedido

En este caso de uso se necesita conocer el estado de muchas entidades del sistema, por lo que el **MultiProvider** toma una gran importancia para prevenir el *prop-drilling* de padre a hijo. **PedidoProvider** se encarga de ir guardando los datos temporales del pedido mientras se avanza en las páginas del formulario, mientras que **CargaProvider** se encarga de obtener los datos del tipo de carga que tienen acordado entre el cargador y el encargado.

En caso de que el usuario sea el encargado, **InviteProvider** obtiene los cargadores que están dados de alta en la empresa del encargado de manera asíncrona. Si el usuario que crea el pedido es el propio cargador, no se permite cambiar la empresa cargadora. Si el actor es el encargado, **VehiculoProvider** y **TransportistaProvider** recuperan además los vehículos y conductores disponibles para permitir una primera asignación antes de confirmar el pedido, así ya se puede realizar la planificación de la semana sin tener que ir a la página de planificación.

Si el tipo de carga que se quiere transportar aún no existe en el sistema, el usuario crea uno nuevo definiendo los detalles de la carga, junto con el origen y el destino. Para obtener direcciones reales y facilitar su introducción, el formulario utiliza **Photon**, un servicio de geocodificación basado en los datos de **OpenStreetMap**. A medida que el usuario introduce la dirección, el sistema de autocompletado muestra sugerencias de calles (con número) reales. Cuando se selecciona una dirección, se almacenan tanto la dirección legible, para mostrarla posteriormente en la carta de porte, como las coordenadas geográficas, que se usarán para abrir la ruta de navegación entre el origen y el destino.

Con todo esto, **PedidoProvider** guarda los detalles del propio pedido, más las fechas individuales y la primera asignación de las cargas, esto se guarda en un **DTO CargasSeleccionadas**. Al crear el pedido, estas **CargasSeleccionadas** se anidan dentro del propio pedido y se envían al backend, donde se realizan las verificaciones de las reglas de negocio y se guardan en **Firestore**. Antes de persistir las cargas, el backend ejecuta `preparar_carta_porte_snapshot`, que congela los datos del cargador y del subcontratado vigentes en ese momento. Esto es necesario para poder generar la carta de porte después con los datos de todos los usuarios de la cadena de transporte y de la propia carga.

Realizar planificación

Realizar la planificación es el caso de uso principal del módulo de gestión de tráfico. En la Tabla 6.3 se detallan los archivos de implementación de este caso de uso.

Capa	Archivo	LOC	Descripción
UI	<code>plan_page.dart</code>	108	Página principal que agrupa los widgets de planificación
UI	<code>plan_header.dart</code>	90	Cabecera de la interfaz de planificación
UI	<code>lista_cargas_panel.dart</code>	185	Lista de cargas pendientes por planificar
UI	<code>calendario_cargas.dart</code>	196	Calendario estilo Gantt para planificación

6. IMPLEMENTACIÓN

Capa	Archivo	LOC	Descripción
UI	panel_asignacion vehiculo.dart	350	Panel para cambiar la asignación o subcontratar la carga seleccionada
Provider	carga_provider.dart	-	Guardar cambios en lote (400-417), cálculo de disponibilidad (111-145) y (147-180)
Provider	invite_provider.dart	54-66	Obtiene los subcontratados dados de alta para cederles la carga
Backend	cargas_service.py	-	Verificación de reglas de negocio, guardado en lote (función bulk_update), llamada al servicio de notificación (144-155) para notificar asignación

Tabla 6.3: Detalle de archivos - Realizar Planificación

Como las entidades de chófer y vehículo no guardan su propio atributo de estado y además el estado depende de la fecha de las cargas, el estado de estas entidades se calcula de manera dinámica dependiendo de la fecha asignada a la carga seleccionada. El provider de cargas contiene la lista completa de las cargas de la empresa y utilizando esta lista se filtran los conductores y vehículos disponibles para esas cargas, teniendo en cuenta la fecha de cada una para evitar colisiones horarias que harían la planificación imposible de realizar. La función para calcular la disponibilidad es idéntica en forma para los conductores y vehículos y tiene los siguientes pasos:

- Primero se itera sobre todas las cargas y se descartan aquellas que no pueden generar conflictos: se descartan las cargas cedidas o las que están pendientes (sin arrastrar al calendario) todavía. También se descartan las que no tienen un conductor o vehículo asignado.
- Para las cargas que pueden generar un conflicto, el sistema llama a la función auxiliar `_hayColisionHoraria`. Aquí se verifica si el intervalo de la carga objetivo (inicio, fin y su buffer) se solapa en algún punto con el intervalo temporal y el buffer de la carga ya existente en el sistema.
- Con el paso anterior se obtienen los IDs de los conductores/vehículos ocupados para esa fecha y se almacenan en un Set. Por último, se filtran los conductores/vehículos cuyos IDs no están presentes en el Set. De esta manera, solo se obtienen los recursos disponibles teniendo en cuenta colisiones horarias, lo que asegura que la planificación realizada sea realizable y no haya estados imposibles.

Una vez que se han asignado las fechas, conductores y vehículos para las cargas planificadas, se actualizan en la base de datos con la función mostrada en el Código 4.

Generación de la carta de porte

La carta de porte incluye los datos de todos los usuarios de la cadena de transporte, desde el cargador hasta el cliente final. Después de crear la carta de porte, se tiene que guardar en la nube y enviarla tanto al cargador como a la empresa subcontratada (si la hay para esa carga). Para renderizar la carta de porte se ha utilizado **Jinja2** para crear una plantilla HTML + CSS que se **renderiza en el servidor** dependiendo de los datos de la carga y WeasyPrint para generar el PDF a partir de esa plantilla. Esto permite añadir o quitar nuevas secciones a la carta de porte editando la plantilla directamente, sin tocar el código Python. Ya que la carta de porte digital es algo relativamente nuevo en el sector logístico, esto podría ser útil si en un futuro se requirieran distintas versiones de carta de porte dependiendo de la empresa o de distintos requisitos legales.

El PDF generado no se devuelve en la respuesta HTTP al cliente, sino que se guarda en Firebase Storage en la ruta `cartas_porte/company_id/carta_carga_id.pdf` y se devuelve la URL de la carta. De esta manera, todos los miembros de la cadena que pertenezcan al tenant identificado con `company_id` pueden acceder a la carta de porte desde el cliente. La ruta del blob también se guarda en el documento de la carga en Firestore para permitir regenerar la URL en consultas posteriores. En el lado del cliente, debido a que acceder a un PDF desde la web no es igual que acceder desde el dispositivo móvil, se implementan dos clases que heredan la clase abstracta `pdf_handler.dart`. Esta clase contiene la función `open()`, que recibe la URL, descarga el contenido desde Storage y lo abre en una pestaña del navegador si la versión es web, o lo abre en el dispositivo directamente si se está usando un dispositivo móvil.

6.5. Implementación del módulo de notificaciones

El módulo de notificaciones utiliza **Firestore Cloud Messaging (FCM)** para garantizar que la información relevante llega de manera inmediata a los distintos usuarios. La clase `NotificacionService` centraliza el envío de notificaciones push. Los casos de uso que envían notificaciones requieren que se envíen a usuarios de distintos tipos, cada usuario debe tener un token FCM para saber a qué dispositivo enviar la notificación. El servicio resuelve dinámicamente en qué colección debe buscar el token FCM del destinatario en función de su rol. El token de cada dispositivo se registra al iniciar sesión y se persiste en el documento correspondiente al rol del usuario, de forma que la propia ubicación del documento determina implícitamente el tipo de usuario sin necesidad de un campo adicional. El sistema dispara notificaciones en dos puntos del flujo de negocio:

- **Generación de la carta de porte:** al completarse la generación del PDF, se notifica al cargador y, si la carga ha sido cedida, también al subcontratado, informando de que el documento ya está disponible en sus aplicaciones.
- **Asignación de carga a conductor o cesión a subcontratado:** al vincular una carga a un chófer propio o cederla a una empresa subcontratada, se notifica a la parte correspondiente.

6. IMPLEMENTACIÓN

A modo de ejemplo, en el siguiente fragmento se puede observar cómo el servicio de generación de la carta de porte llama a `NotificacionService` tras completar la generación para enviar la notificación al cargador de esa carga:

```
1 self._notificacion_service.notificar(  
2     user_id=carga.get("cargador_id"),  
3     roles=["cargador"],  
4     titulo="Carta de porte generada!",  
5     cuerpo=f"La carta de porte de la carga {carga_id} ha sido generada.",  
6     data={"cargaId": carga_id},  
7 )
```

Código fuente 7: Ejemplo de llamada al servicio de notificaciones

6.6. Implementación del módulo de operaciones

El módulo de operaciones contiene la implementación de la hoja de ruta del chófer y la funcionalidad de las incidencias entre los chóferes y el encargado de planificación.

Hoja de ruta del conductor

Capa	Archivo	LOC	Descripción
UI	conductor_page.dart	67	Página principal: Agrupa el calendario reutilizado del módulo de gestión de tráfico, junto con la información de ese día para el conductor. La información que se muestra se filtra a través de <code>companyId</code> y <code>conductorId</code> para mostrar a cada conductor lo que le corresponde. Desde aquí, el conductor confirma la recogida y la entrega de las cargas y puede abrir el mapa externo para ver la ruta.
UI	conductores_kpi_grid.dart	38	Grid de tarjetas KPI: muestra el número de viajes del día y una tarjeta con la próxima entrega pendiente.
Provider	conductor_provider.dart	68	Se suscribe a la colección de cargas de Firestore directamente para que si hay replanificaciones por parte del encargado de tráfico se muestren directamente en la UI del conductor a través de los WebSockets nativos de Firebase.

Tabla 6.4: Detalle de archivos - Módulo de operaciones

Incidencias

De la misma forma que ocurre en `conductor_provider.dart`, la lectura de incidencias es un flujo urgente que debe mostrarse de manera inmediata en la pantalla del planificador para que este realice replanificaciones rápidas. Firestore implementa WebSockets de manera nativa, por lo que la lectura de incidencias por parte del planificador ocurre directamente entre Flutter y Firebase, sin pasar por la API y añadiendo las reglas de seguridad en la base de datos directamente, como se puede observar en el Anexo B. Se ha intentado que el acoplamiento entre Flutter y Firebase sea lo más mínimo posible, por lo que solo las **lecturas que requieren tiempo real** se realizan de esta forma. Las escrituras de las incidencias sí pasan por FastAPI.

6.7. Despliegue

6.7.1. Infraestructura necesaria

Como última tarea de implementación, la aplicación web se ha desplegado en la nube de Google. El frontend (archivos estáticos) y el backend se despliegan por separado. De esta forma, una actualización en el frontend no requiere redespargar el backend, y viceversa. Desplegar las aplicaciones móviles en las tiendas de Google y Apple queda fuera del alcance debido al coste económico y temporal que esto supondría. La Figura 6.1 muestra el diagrama de despliegue del sistema, indicando en qué nodo se ejecuta cada componente.

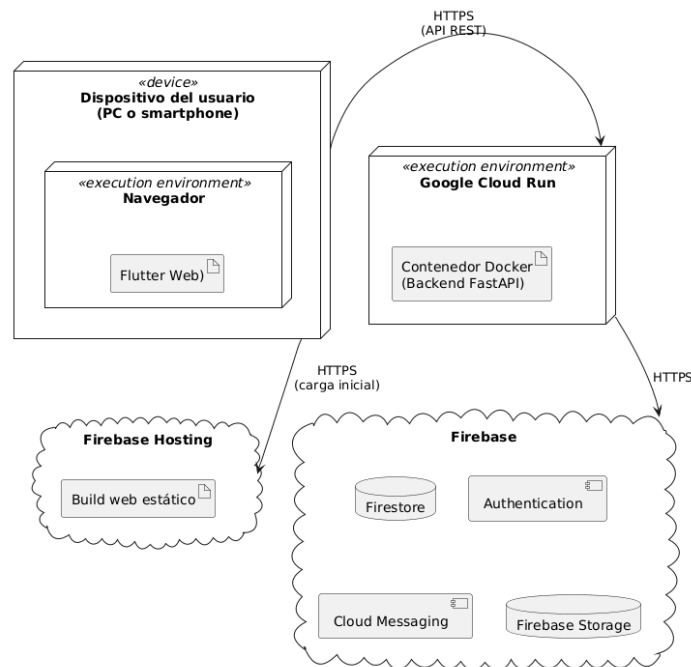


Figura 6.1: Diagrama de despliegue del sistema

Esta es la infraestructura necesaria para el despliegue:

- **Hosting de archivos estáticos:** Se ha elegido Firebase Hosting por su integración con el resto del proyecto y su CDN, que acelera el tiempo de carga.
- **Plataforma para el backend:** Se ha elegido Google Cloud Run, también por la integración que tiene con el resto del proyecto al ser de Google y por su capa gratuita.
- **Firebase:** Cubre la infraestructura funcional de la aplicación, que ya se ha descrito en la Sección 5.1.

6.7.2. Proceso de despliegue

El despliegue del frontend consiste en generar una compilación de producción de la aplicación Flutter Web utilizando WebAssembly, que aumenta la velocidad frente a la compilación regular de Flutter en JavaScript, y subirla a Firebase Hosting. Al contrario que el backend, el frontend no requiere *Dockerización* ya que solo contiene archivos estáticos. Las claves de configuración necesarias para el frontend se inyectan creando un archivo `dart_defines.json` y llamándolo con el flag `-dart-define` en el comando de despliegue del frontend. A continuación se muestra un ejemplo del archivo:

```
1 {  
2   "API_BASE_URL": "https://ruta-backend.a.run.app", // URL del backend en Cloud Run  
3   "SYNCFUSION_KEY": "clave_de_licencia", // licencia del calendario Syncfusion  
4   "VAPID_PUBLIC_KEY": "clave_publica_vapid" // clave pública para FCM  
5 }
```

Código fuente 8: Ejemplo de `dart_defines.json` para configurar dependencias del frontend

Con el `API_BASE_URL` del ejemplo anterior, el frontend ya sabe a qué ruta debe hacer las peticiones. Para el despliegue del backend, primero hay que *dockerizar* la aplicación FastAPI en un contenedor Docker utilizando un *Dockerfile* para instalar las dependencias de las librerías que se utilizan en el proyecto. Una vez que la imagen de Docker está lista, se sube a Google Cloud Run, que gestiona los recursos del servidor automáticamente en base a la demanda que tenga. Los comandos para realizar el despliegue completo se pueden encontrar en el Anexo E.

En el proceso de despliegue es donde toman importancia los *flavors* de Flutter que se han descrito al inicio de este capítulo (6.1.2). En un proyecto real, una vez que el desarrollo se ha hecho en el *flavor dev*, el despliegue debería hacerse apuntando al servidor desplegado y a un proyecto de Firebase limpio de producción (*flavor prod*). En este caso, para simplificar el proceso y evitar riesgos de costes económicos inesperados (si se supera el límite gratuito dentro del plan *pay-as-you-go*), se ha simplificado el proceso utilizando el *flavor dev* como entorno de despliegue y así tener un único proyecto suscrito al plan que puede generar costes. Estos cobros inesperados son improbables al no haber usuarios utilizando la aplicación, pero podrían ocurrir si hay bucles infinitos o ataques DDoS que aumenten la demanda de recursos.

CAPÍTULO 7

Pruebas

En este capítulo se detallan las pruebas realizadas y las herramientas utilizadas para validar el correcto funcionamiento de la aplicación. Las pruebas desarrolladas se dividen en **pruebas unitarias** para comprobar componentes aislados de la lógica de negocio y **pruebas de integración** para verificar una correcta comunicación entre distintas capas reales de la aplicación. Las tecnologías de testing utilizadas se han descrito anteriormente en [4.4](#).

7.1. Análisis estático de código

El **análisis estático** del código se ha utilizado principalmente para garantizar la calidad, seguridad y mantenibilidad del software desde las fases iniciales del desarrollo. Con la integración de la herramienta SonarCloud, se ha llevado a cabo una revisión automatizada del código, permitiendo la detección temprana de *bugs* y *code smells* que aumentan la deuda técnica. SonarCloud analiza aspectos como la seguridad, fiabilidad y mantenibilidad del código, además de mostrar la cobertura del código. En la Figura 7.1 se puede observar una imagen del análisis realizado por SonarCloud, donde la cobertura es de aproximadamente un 58 %. Este porcentaje incluye algunos archivos de interfaz de usuario y otros archivos de configuración de Flutter, que no se han probado.

7. PRUEBAS

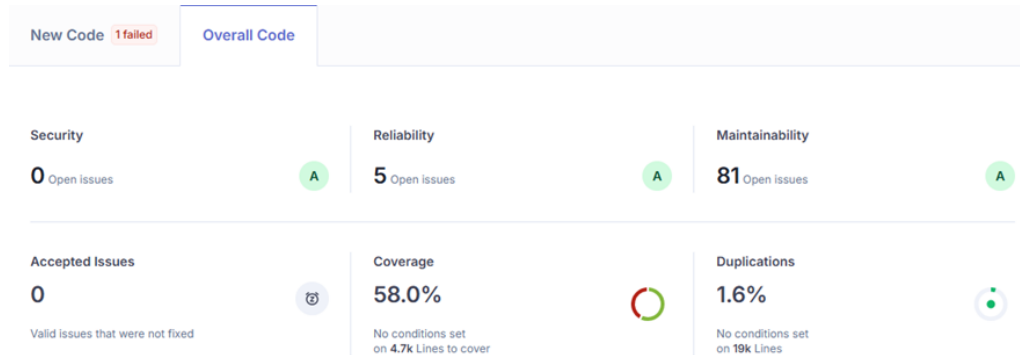


Figura 7.1: Resumen de SonarCloud

7.2. Pruebas implementadas

A continuación, se detalla la combinación de pruebas unitarias y de integración implementadas. Los tests unitarios prueban componentes del frontend y del backend por separado, mientras que los tests de integración se han centrado en probar la **integración del backend con la base de datos** y el flujo de autenticación. Las pruebas de integración requieren más configuraciones y tiempo al no utilizar *mocks* de las dependencias externas y necesitan preparar un entorno de ejecución lo más parecido posible al de producción, por lo que se han priorizado las pruebas unitarias para obtener una cobertura mayor y realizar los tests dentro del tiempo planificado, dejando los de integración solo para los flujos más importantes. En la Tabla 7.1, se puede observar un resumen de las pruebas implementadas.

Tabla 7.1: Resumen de las pruebas implementadas

Tipo de prueba	Objetivo	N.º de tests
Pruebas unitarias frontend	ViewModel del frontend	56
Pruebas unitarias backend	Servicios backend y lógica de negocio	48
Pruebas de integración	backend-Firestore y flujo de autenticación	7

7.2.1. Pruebas unitarias

Las siguientes tablas (Tabla 7.2 y Tabla 7.3) muestran un resumen de las pruebas unitarias implementadas. En cada tabla se muestra el componente probado, las pruebas realizadas para ese componente y su cobertura.

Tabla 7.2: Pruebas unitarias del frontend (Flutter/Dart)

Componente	Archivo	N.º tests	Cobertura
AuthProvider	auth_provider_test.dart	2	58,72 %
CargaProvider	carga_provider_test.dart	34	81,39 %
TransportistaProvider	transportista_provider_test.dart	6	75,00 %
TransportistaService	transportista_service_test.dart	7	85,51 %
IncidenciasProvider	incidencias_provider_test.dart	2	70,97 %
ConnectivityProvider	connectivity_provider_test.dart	5	100 %

Tabla 7.3: Pruebas unitarias del backend (Python/FastAPI)

Componente	Archivo	N.º tests	Cobertura
CargasService	test_cargas_service.py	15	66 %
CartaPorteService	test_carta_porte.py	4	72 %
CargaSchema	test_carta_porte_schema.py	1	86 %
TransService	test_contract_users.py	2	34 %
IncidenciasService	test_incidencias_and_notification.py	5	38 %
PedidosService	test_pedidos_service.py	12	96 %
RateLimitMiddleware	test_rate_limit.py	1	89 %
CargaSchema (validación)	test_schemas_cargas.py	5	86 %
TransService	test_trans_delete.py	3	34 %

Frontend

Las pruebas unitarias en el frontend se han centrado en los proveedores de estado, utilizando *mocks* de la librería *mockito* para simular los datos que vendrían del backend o de Firebase Auth. A continuación se muestra un resumen de los tests implementados más importantes por cada Provider.

Autenticación (AuthProvider):

- **Inicio de sesión lanza `TimeoutException` si no se hidrata la sesión:** Verifica que, tras una autenticación satisfactoria en Firebase Auth, la aplicación también es capaz de recuperar correctamente los datos del usuario desde la base de datos (hidratar la sesión), lanzando una excepción si no lo hace. Son dos procesos independientes, por lo que un fallo en la segunda etapa dejaría al usuario en un estado inconsistente (autenticado pero sin información de perfil).
- **El inicio de sesión se completa correctamente cuando hay token y usuario:** Este test verifica un inicio de sesión exitoso cuando se obtiene el token de Firebase Auth y se consiguen los datos de usuario de Firestore.

Cargas y planificación (CargaProvider):

- **El filtrado de conductores disponibles excluye los ocupados:** Verifica que la función encargada de filtrar los conductores disponibles excluye los que ya están asignados y

7. PRUEBAS

los que al acabar su viaje no tienen tiempo de llegar a la siguiente carga a tiempo.

- **El filtrado de vehículos disponibles excluye los ocupados:** Lo mismo que el test para conductores pero para vehículos.
- **Una carga cedida no se puede mover ni editar:** Prueba que las cargas cedidas, al ser responsabilidad de la empresa subcontratada, quedan congeladas y el porteador contractual (la empresa contratadora) no puede modificar las fechas ni asignar conductores/vehículos.
- **guardarCambios captura error de servicio y mantiene la lista de cargas:** Simula un error en el backend y comprueba que las cargas que se estaban editando siguen intactas en el frontend.

Otros tests secundarios:

También se han realizado tests más secundarios que comprueban operaciones CRUD y manejo de errores más simples en TransportistaProvider (CRUD de chóferes) y en las incidencias. Además, se han implementado pruebas sobre ConnectivityProvider, que verifican la detección de cambios en el estado de la red y el aviso a través de la interfaz cuando se pierde o recupera la conexión a internet. Aunque no es una funcionalidad crítica, resulta especialmente útil para los chóferes, que pueden perder la conexión con mayor facilidad en distintos puntos de sus rutas al utilizar el dispositivo móvil.

Backend

Los tests unitarios del backend se centran en los servicios de pedidos y cargas, que se utilizan para el caso de uso de planificación, y en comprobar la generación de cartas de porte. A continuación se describen los más importantes:

Gestión de pedidos (PedidosService):

- **Creación de pedido notifica carga asignada:** Verifica que, cuando un pedido se crea ya con cargas asignadas a un conductor, el servicio invoca correctamente al servicio de notificaciones con el userId, rol y payload esperados.
- **Creación de pedido rechaza carga fuera de ventana:** Comprueba que una carga cuya fecha de carga es anterior a la del pedido padre es rechazada (400), evitando inconsistencias temporales entre pedido y sus cargas.
- **Snapshot de carta de porte inmutable:** Genera el *snapshot* y comprueba que cambios posteriores en los datos del cargador no alteran el snapshot ya persistido, que debe reflejar los datos históricos de esa carga.

Gestión de cargas (CargasService):

- **Actualización en lote persiste cargas correctamente:** Verifica que `bulk_update_cargas` agrupa los pedidos referenciados, valida cada carga contra su pedido, y persiste los cambios en un único *batch write*.
- **Ceder carga a subcontratado notifica correctamente:** Verifica que al ceder una carga a una empresa subcontratada, el estado de la carga pasa a “Cedido” y se le envía la notificación de la cesión.
- **Cálculo de cargas asignadas y sin asignar:** Verifica que los contadores usados en el panel de control del encargado consultan correctamente Firestore filtrando por estado, devolviendo el recuento esperado.

Otros tests secundarios:

Además de los anteriores, también se han desarrollado pruebas específicas para la generación de la carta de porte, la eliminación de usuarios externos y el servicio de notificaciones.

7.2.2. Pruebas de integración

Las pruebas de integración se ejecutan contra el **Firestore Emulator**, sin ningún componente simulado entre el endpoint HTTP y la base de datos. Esto permite detectar fallos que los tests unitarios no pueden ver, como errores de persistencia real, consultas mal implementadas y problemas de aislamiento entre datos de distintas empresas. Se ejecutan en un job independiente de CI, ya que requieren el emulador levantado y una limpieza de colecciones entre cada test para evitar contaminación de estado. La Tabla 7.4 muestra un resumen de las pruebas de integración implementadas.

Tabla 7.4: Pruebas de integración

Componente / Flujo	Archivo	N.º tests
Login/logout con Firebase	<code>auth_integration_test.dart</code>	2
Flujo pedido → cargas → carta de porte	<code>integration_tests.py</code>	1
Snapshot de la carta de porte persistido en la carga	<code>integration_tests.py</code>	1
Aislamiento entre empresas (lectura)	<code>integration_tests.py</code>	1
Aislamiento entre empresas (modificación)	<code>integration_tests.py</code>	1
Carta de porte sobre carga inexistente	<code>integration_tests.py</code>	1
Total	2 archivos	7

- **Flujo de autenticación completo del frontend:** Este test utiliza una biblioteca que simula el comportamiento real de Firebase Auth para comprobar el cambio del estado de autenticación ante un `signIn/signOut`, verificando que el *provider* está correctamente suscrito y reacciona a los cambios de estado como debe.
- **Flujo completo pedido-carga-carta de porte:** Ejecuta la creación real de un pedido, verifica que la carga se crea en su subcolección, genera la carta de porte, y confirma que cada paso queda persistido en Firestore.

- **Snapshot de carta de porte persistido y su inmutabilidad:** Verifica que la carta generada contiene los datos correctos del destinatario, y que una actualización posterior de la carga no los altera. Al tener que actualizar la carga también se prueba el endpoint real que se llama en la planificación al actualizar las fechas y asignaciones de los viajes.
- **Aislamiento multi-tenant entre empresas:** Crea un pedido bajo una empresa y verifica que otra empresa autenticada no puede leerlo, modificarlo ni verlo en el listado general. Es el test más crítico de seguridad.
- **Atomicidad ante fallo de generación de la carta de porte:** Simula un fallo y comprueba que la carga no queda en un estado a medio persistir.

7.3. Integración Continua (CI)

La integración continua ayuda a mantener un control sobre lo que se ha implementado en fases anteriores del proyecto, permitiendo detectar cambios que rompen el comportamiento esperado del programa. Para realizarla, se ha utilizado **GitHub Actions** para orquestrar los distintos *jobs* donde se ejecuta cada grupo de tests. De esta manera, en cada push se ejecutan las pruebas y se hace el análisis de SonarCloud.

Las pruebas unitarias se separan en dos, distinguiendo las del frontend y las del backend. Las pruebas de integración requieren configurar e inicializar Firestore Emulator instalando Java 21, por lo que se ejecutan en un *job* separado. Estas pruebas producen los informes de cobertura de Flutter y Pytest, que SonarCloud combina automáticamente al realizar el análisis estático para reflejar el porcentaje del código cubierto. Si algún test o el control de calidad de SonarCloud falla, el push no se considera válido y habría que realizar los cambios oportunos. En la Figura 7.2 se muestra un ejemplo de la ejecución de los distintos *jobs* de GitHub Actions tras un *push*. El flujo completo de integración continua implementado puede consultarse en el Anexo D, donde se incluye el archivo `ci.yml` utilizado en el proyecto.

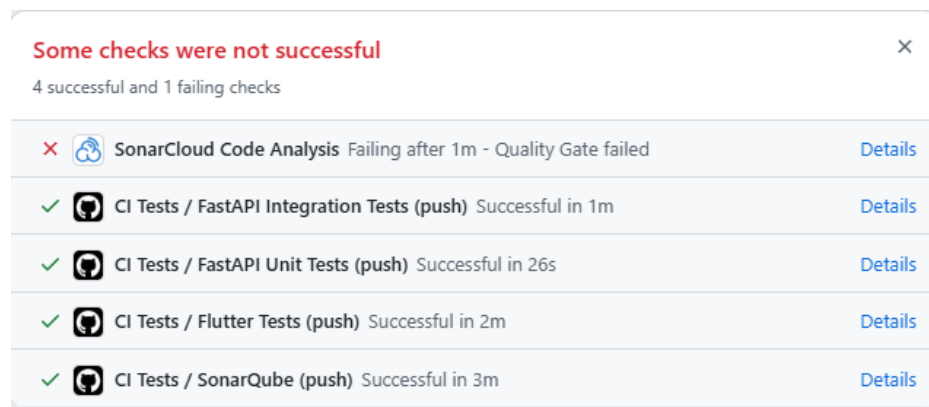


Figura 7.2: Ejecución automática de jobs en GitHub Actions

Seguimiento y control

En este capítulo se describe el seguimiento y control realizado a lo largo del proyecto. Se comentan las desviaciones temporales ocurridas y las medidas tomadas para resolverlas.

8.1. Evolución del alcance

Como el proyecto realizado pertenece a un dominio que no se conocía en profundidad al iniciar la fase de planificación, y aunque varias funcionalidades que el proyecto debía incluir ya fueron definidas por el tutor (que actuaba como cliente), fue complicado definir el alcance global y el tiempo que se iba a dedicar a las tareas. Aunque las funcionalidades principales definidas al inicio han sido implementadas sin muchos cambios, ha ocurrido algún cambio en el alcance global. El cambio principal ha sido que, debido al retraso que había en las dos primeras iteraciones, durante la segunda iteración se decidió reducir el alcance de la tercera iteración, manteniendo sus funcionalidades opcionales para poder terminar primero lo que faltaba de las iteraciones anteriores.

En la reunión de inicio se planteó que la interacción del usuario chófer con el sistema fuera a través de voz, además de poder interactuar con las interfaces de usuario. No obstante, las primeras semanas se decidió que era demasiado optimista dentro del alcance, por lo que se decidió dejarlo como funcionalidad opcional a realizar en la tercera iteración. Al finalizar la segunda iteración, se decidió que no era viable incluir esta funcionalidad debido al poco tiempo restante, por lo que no se ha implementado. Además de esto, también ha habido un pequeño cambio relacionado con la funcionalidad de entrega: las aplicaciones reales de transporte permiten a los conductores adjuntar pruebas (por ejemplo, fotografías) al marcar una carga como entregada, pero esta funcionalidad se ha descartado por falta de tiempo al ser algo más secundario.

Estas reducciones han provocado que la aplicación del conductor quede más limitada que la aplicación principal, la del encargado. Esto es entendible ya que el encargado es el

usuario principal, pero para que la aplicación sea equilibrada para ambos usuarios internos de la empresa de transporte, se plantea como línea de trabajo futuro ampliar las funcionalidades de los chóferes (véase 9.3).

8.2. Análisis de riesgos materializados

En la planificación se definieron los riesgos que se podrían materializar durante el proyecto y la manera de combatirlos. A continuación, se muestra de qué manera han afectado en el transcurso del proyecto:

- **R1 - Tecnologías desconocidas:** Se identificó Flutter como la principal causa de retraso dentro de este riesgo por no haberse utilizado nunca. Flutter y Dart se consideran tecnologías con una **curva de aprendizaje moderada** y el desconocimiento inicial del framework ha afectado algo más de lo que se planteó en un inicio. La primera iteración estaba pensada como toma de contacto con el framework, se fue adquiriendo conocimiento del framework a medida que se iba implementando el entorno inicial y los casos de uso simples, pero se produjo un retraso aproximado de dos semanas que se arrastró a las siguientes iteraciones.
- **R3 - Alcance demasiado ambicioso:** La planificación realizada al inicio del proyecto no ha sufrido grandes alteraciones en cuanto a las tareas a realizar, más allá de los ligeros cambios en el alcance comentados en 8.1. Sin embargo, el alcance quizás ha sido demasiado ambicioso y sí se han producido desviaciones en la estimación temporal de las tareas, como se muestra en 8.3. Estas desviaciones han resultado en la decisión de entregar el trabajo en la siguiente convocatoria a la planificada inicialmente.

8.3. Desviaciones temporales

La planificación realizada en marzo contemplaba la entrega del proyecto el 21 de junio. A inicios de junio, la aplicación ya se encontraba en las últimas fases de desarrollo, con alguna funcionalidad pendiente de realizar aún. Además de terminar con la implementación, las revisiones de los borradores de la memoria requirieron realizar cambios en el contenido y en los diagramas de la memoria bastante frecuentemente, a lo que se sumaba reflejar parte de la implementación realizada en la memoria y terminar con el testing final de la aplicación. Dado que en ese momento quedaban dos semanas para matricular el TFG (15 de junio), la primera semana de junio se decidió que no sería posible completar lo que quedaba para la convocatoria de junio, y se decidió posponer la entrega a la convocatoria de septiembre.

Mirando en retrospectiva, al haber iniciado el proyecto en marzo la planificación se tenía que cumplir a la perfección y sin desviaciones temporales para poder llegar a entregarlo el 21 de junio, ya que el margen entre finalizar la implementación y la entrega era de unas dos semanas. Al decidir que se iba a atrasar la entrega del proyecto, no se realizó una replanificación de las pocas tareas restantes, simplemente se extendió el margen para finalizarlas, fijando la finalización del proyecto de manera aproximada a mediados de julio.

A lo largo del proyecto se han ido guardando los tiempos dedicados a cada tarea. En la tabla 8.1 se puede observar la comparación entre el tiempo planificado y el tiempo dedicado a cada tarea.

Tarea	Planificado (h)	Real (h)
Investigación		
Investigación de tecnologías y sector de transporte	8	8
Análisis de Requisitos		
Requisitos funcionales	10	7
Requisitos no funcionales	5	4
Otros requisitos	2	0
Subtotal Análisis de requisitos	17	11
Diseño		
Diseño de arquitectura	5	7
Diseño de interfaces	5	3
Diseño de API	2	2
Diagramas UML principales	15	20
Diagramas UML secundarios	10	5
Diseño base de datos	3	6
Subtotal Diseño	40	43
Implementación		
Entorno inicial	15	17
Implementación	4	5
Implementación de lógica de negocio	55	60,5
Implementación de cartas de porte	10	10
Notificaciones	5	2
Implementación de interfaces de usuario	20	28
Implementación gestión de estado	15	25
Otras tareas (refactorizaciones, mejoras, correcciones inesperadas)	0	14
Despliegue	10	7
Subtotal Implementación	134	168,5
Testing		
Pipeline CI	5	3
Pruebas unitarias/integración	10	13
Subtotal Testing	15	16
Trabajo Académico		
Memoria	60	81
Preparación de la defensa	10	-
Subtotal Trabajo Académico	70	81
Gestión		
Planificación	10	12
Seguimiento y Control	5	5
Reuniones	5	10
Subtotal Gestión	20	27
TOTAL	304	354,5

Tabla 8.1: Comparación de horas planificadas y reales por tarea.

El recuento no incluye el tiempo que se empleará para preparar la presentación, pero con las 10 horas planificadas, el tiempo global subiría a unas 364,5 horas, lo que supone una desviación de unas 60 horas. El mayor incremento se ha dado en la **implementación (+34,5 h)**, especialmente debido a la carga del frontend y tareas transversales que abarcan más de una tarea concreta (mejoras, refactorizaciones, correcciones, etc.). También ha habido un incremento de **21 horas** en la **memoria**, debido a los cambios que se han tenido que ir realizando en la estructura, diagramas y contenido tras las distintas revisiones periódicas con el tutor. Además de las desviaciones temporales, es importante entender las desviaciones en plazo del proyecto para entender la decisión de atrasar el proyecto a la convocatoria de septiembre. Para mostrar estas desviaciones, la Tabla 8.2 muestra la fecha planificada del hito y su fecha de finalización aproximada.

Hito	Fecha planificada	Fecha real
Reunión de inicio	5/03/2026	5/03/2026
Inicio del proyecto	9/03/2026	9/03/2026
Fin Iteración 1	31/03/2026	Mediados de abril
Fin Iteración 2	24/04/2026	Mediados-finales de mayo
Fin Iteración 3	29/05/2026	Finales de junio
Entrega memoria y defensa	Convocatoria de junio	Convocatoria de septiembre

Tabla 8.2: Comparación de fechas planificadas frente a reales por hito.

Como puede observarse en los hitos de las iteraciones, el retraso de la primera iteración causó que la segunda empezase con unas dos semanas de retraso. En la segunda iteración ya había más control sobre el proyecto, pero esta iteración era la que más funcionalidades incluía, ya que se implementaban los casos de uso del encargado, la invitación de usuarios externos y sus casos de uso y la generación de cartas de porte. Esta iteración por sí misma ya habría tenido cierto retraso, que acumulado con el de la primera iteración hizo que el retraso global fuese de unas tres semanas.

En este punto, las funcionalidades que se mantuvieron como opcionales para la tercera iteración se descartaron. Aun así, se consideró que no implementar los casos de uso de los conductores dejaría la aplicación incompleta y el uso de Flutter para tener una aplicación móvil precisamente para este usuario perdería sentido. Por esa razón, como se ha comentado en 8.1, se ha simplificado el módulo de los conductores pero no se ha eliminado completamente.

Conclusiones

9.1. Objetivos cumplidos

Tras finalizar el proyecto, se puede considerar que el objetivo principal de desarrollar una primera versión de una aplicación para la planificación de empresas de transporte ha sido completado con éxito, aunque se haya realizado fuera del plazo planificado inicialmente. El dominio del transporte es un dominio complejo cuyos proyectos reales exceden lo que se puede conseguir realizar en aproximadamente 300 horas de trabajo. De todas formas, dentro del alcance definido y centrándose en el núcleo de la gestión de tráfico, se ha conseguido realizar un proyecto de cierta complejidad y con un buen nivel técnico, explorando nuevas tecnologías en alza que no se habían utilizado antes en la carrera. A continuación, se hace una revisión de los objetivos definidos en la sección de Planificación:

- **Diseñar una arquitectura adecuada y escalable:** Este objetivo se ha cumplido correctamente. Se ha definido una arquitectura en capas dividida en tres subsistemas (frontend, backend y Firebase), lo que ha permitido ir añadiendo funcionalidades de manera incremental sin grandes refactorizaciones.
- **Crear un sistema de gestión de cargas y pedidos:** Es el núcleo de la aplicación desarrollada, por lo que se ha cumplido correctamente.
- **Permitir a colaboradores y empresas externas interactuar con el sistema:** Los cargadores y empresas de transporte subcontratadas pueden colaborar en la aplicación, creando pedidos directamente en el sistema o visualizando los viajes cedidos, restringiendo correctamente sus permisos y ofreciéndoles únicamente las funcionalidades que necesitan.
- **Implementar un sistema de notificaciones:** Se envían notificaciones tanto a la aplicación web como a las aplicaciones móviles de los usuarios externos y conductores utilizando Firebase Cloud Messaging para notificar los eventos más importantes.

- **Generar cartas de porte digitales:** Se ha implementado la generación de cartas de porte siguiendo los requisitos legislativos vigentes. Los documentos contienen los campos necesarios y el QR para su trazabilidad.
- **Crear una interfaz de usuario sencilla de usar:** Es un objetivo difícil de medir por ser más subjetivo y no haber usuarios reales, pero el cliente del proyecto ha dado el visto bueno y se considera que la interfaz ofrece buenas ayudas de usabilidad. Además, las interfaces *responsive* hacen que sea fácil de utilizar en todos los dispositivos.
- **Alcanzar la calidad suficiente para el despliegue:** Estaba planificado desplegar únicamente la aplicación web. Este objetivo se ha cumplido y el proyecto está desplegado en la nube de Google.
- **Realizar pruebas de funcionamiento:** El objetivo se ha cumplido, ya que se han realizado pruebas unitarias en el frontend y backend y pruebas de integración. Todo esto se ha unido con un pipeline de CI y análisis estático en SonarCloud.

9.2. Competencias logradas

El proyecto ha sido muy enriquecedor a nivel personal, ya que me ha hecho profundizar en tecnologías desconocidas como Flutter y Firebase, y descubrir el mundo multiplataforma para realizar proyectos compatibles con dispositivos móviles y aplicaciones web de manera más rápida. Además de eso, como el proyecto pertenecía a un dominio desconocido al principio, también me ha permitido acercarme a cómo se desarrolla el software en el mundo laboral, donde muchas veces no se conoce el dominio del software y hay que realizar las capturas de requisitos con sumo cuidado.

Otra lección aprendida sería la importancia de las planificaciones bien pensadas y ejecutadas. Como la entrega se ha tenido que retrasar, tener un MVP en la segunda iteración habría sido algo que habría facilitado la entrega en el tiempo esperado, pudiendo dejar sin implementar la tercera iteración, cosa que no ha sido posible por tener las funcionalidades del conductor (un usuario importante para la coherencia global del proyecto) en la tercera iteración. Además de esto, tener un cliente simulado detrás del proyecto me ha hecho darme cuenta de que es normal que surjan retrabajos y cambios inesperados, por lo que es importante dejar márgenes para que estos no afecten a los objetivos principales.

9.3. Trabajo futuro

Dado que el desarrollo de un TMS (Transport Management System) completo excede ampliamente lo que se puede conseguir en un TFG, esta aplicación se puede expandir de muchas maneras y en diferentes direcciones. A continuación, se describen varias líneas de trabajo futuras para mejorar el proyecto e incorporar varias funcionalidades propias de los sistemas de gestión de transporte reales:

- **Geolocalización en tiempo real de la flota:** Actualmente, la comunicación entre los conductores y los encargados de tráfico se realiza a través de las incidencias. Aunque estas funcionan en tiempo real para facilitar las replanificaciones, la información que se transmite es limitada. Además, cada vez es más común que los cargadores requieran trazabilidad de la mercancía, por lo que estos sistemas son habituales en las aplicaciones comerciales. Como trabajo futuro, se podría implementar esta funcionalidad utilizando Firebase de la misma forma que se hace con las incidencias: el dispositivo del chófer reportaría sus coordenadas y el encargado visualizaría la posición actualizada de cada vehículo sobre un mapa integrado en su interfaz.
- **Ampliación de las funcionalidades del chófer:** Como se comenta en el capítulo de seguimiento y control, las funcionalidades del chófer han quedado reducidas tras las evoluciones en el alcance (véase 8.1). Como trabajo futuro, se propone retomar la interacción por voz planteada originalmente para facilitar el uso durante la conducción, así como permitir adjuntar evidencias fotográficas al confirmar una entrega, una funcionalidad habitual en las aplicaciones de transporte reales. Todo esto haría que la aplicación quedara más equilibrada para los dos usuarios internos.
- **Planificación avanzada:** El sistema de planificación actual permite hacer asignaciones de manera manual, pero no ofrece ayuda en la toma de esas decisiones ni realiza planificaciones óptimas. Como trabajo futuro, se pueden incorporar algoritmos de optimización de rutas como el *Vehicle Routing Problem* utilizando librerías como OR-Tools de Google, que es gratuita y open-source.

Declaración del uso de IA generativa

Herramienta/s utilizada/s:

GitHub Copilot (Plan Pro), Claude (Web), Gemini, Figma Make

Uso realizado:

- ☐ Generación de texto
- ☒ Reformulación
- ☒ Corrección ortográfica
- ☒ Sugerencia de estructura
- ☒ Apoyo metodológico
- ☒ Ayuda para codificar / programar
- ☒ Diseño de interfaz
- ☐ Otros

Descripción del uso realizado:

El uso de las herramientas de IA generativa se ha realizado en distintas fases del proyecto, acelerando el proceso general y ayudando en la toma de decisiones a lo largo del trabajo. Todas las consultas realizadas a estas herramientas han sido contrastadas, y en casos donde se dudaba de la decisión de la herramienta, se han consultado otras fuentes de Internet, como tutoriales, vídeos o blogs. Todo el código generado ha sido revisado personalmente línea a línea, adaptándolo a la arquitectura y las necesidades del proyecto y asegurándose de mantener la mejor calidad posible durante todos los ciclos de implementación. En cuanto a la memoria, el uso se ha limitado a sugerencias de estructura y correcciones, pero no se ha generado

y copiado texto directamente. A continuación, se detalla el uso realizado de las distintas herramientas:

- **GitHub Copilot:** Ha sido la herramienta principal de codificación. Se ha utilizado integrado en el IDE para acelerar la implementación y para contrastar las decisiones técnicas tomadas. De esta forma, ha servido como el asistente principal durante las fases de desarrollo y pruebas. Como se ha comentado, todo lo que se ha generado con el asistente ha sido contrastado y se han realizado iteraciones de mejora en los casos necesarios, tanto en la implementación como en las pruebas, priorizando siempre el criterio propio.
- **Claude y Gemini:** Han sido herramientas secundarias utilizadas para el apoyo metodológico, la resolución de dudas puntuales y la confirmación de decisiones. También se han utilizado para maquetar las tablas, estructurar la memoria y corregir erratas y faltas de ortografía.
- **Figma Make:** Es la herramienta de IA generativa de la aplicación de diseño Figma, genera diseños interactivos a partir de un *prompt* en lenguaje natural. Se ha utilizado para realizar el diseño visual de la aplicación a partir de las funcionalidades a incluir y los requisitos con los que debía cumplir la aplicación. Los primeros diseños generados por la herramienta se revisaron y se iteraron para que incluyeran lo necesario y fuese posible implementarlo. Esta herramienta también genera el código de la interfaz, pero en React, por lo que el diseño ha sido una referencia y el código de la interfaz se ha implementado independientemente.

Compromiso de autoría:

Confirmando que todo el contenido generado ha sido revisado, reelaborado y validado personalmente.

Interfaces de usuario secundarias

The figure displays two sequential steps of a user registration process. Both steps feature a progress bar at the top with two indicators: a yellow circle with a checkmark for 'Datos personales' and a grey circle with a number '2' for 'Registra tu empresa'.

(a) Registro de los datos del usuario. This screen contains the following fields and options:

- Nombre
- Apellido
- Email
- Teléfono
- Contraseña
- Confirmar contraseña
- ☐ También soy transportista
- Siguiente

(b) Registro de los datos de la empresa. This screen contains the following fields and options:

- Nombre Comercial *
- Razón Social
- NIF *
- Teléfono *
- Nº Autorización (LOTT) *
- Obligatorio
- Dirección
- Calle y número *
- Dirección completa
- Ciudad *
- Código Postal *
- Provincia *
- País *
- Registrar
- Atrás

(a) Registro de los datos del usuario.

(b) Registro de los datos de la empresa.

Figura A.1: Flujo de registro del usuario y de su empresa.

A. INTERFACES DE USUARIO SECUNDARIAS

Mercancía

Tipo de carga
Bultos

Tipo de embalaje

Origen
Buscar dirección
Escribe al menos 3 letras...

Destino
Buscar dirección

Peso (kg)

Nº bul... Largo ... Ancho... Alto (m)

☐ Apilable

Precio del transporte (€)

Peso máx. (kg)

Cancelar Crear

Figura A.2: Interfaz para registrar un nuevo tipo de carga.

Editar Miembro

Nombre Carlos Apellido Gómez

Email carlos.gómez.6@example.com

Teléfono 676633643

Licencias
☒ C1 ☐ C ☐ D1 ☐ D ☒ BE ☐ C1E
☒ CE ☐ D1E ☐ DE

Asignaciones
Matrícula Vehículo ID Carga

Cancelar Guardar

(a) Detalles de un conductor.

Editar vehículo

Matrícula 8666AAA

Marca Citroen Modelo C3

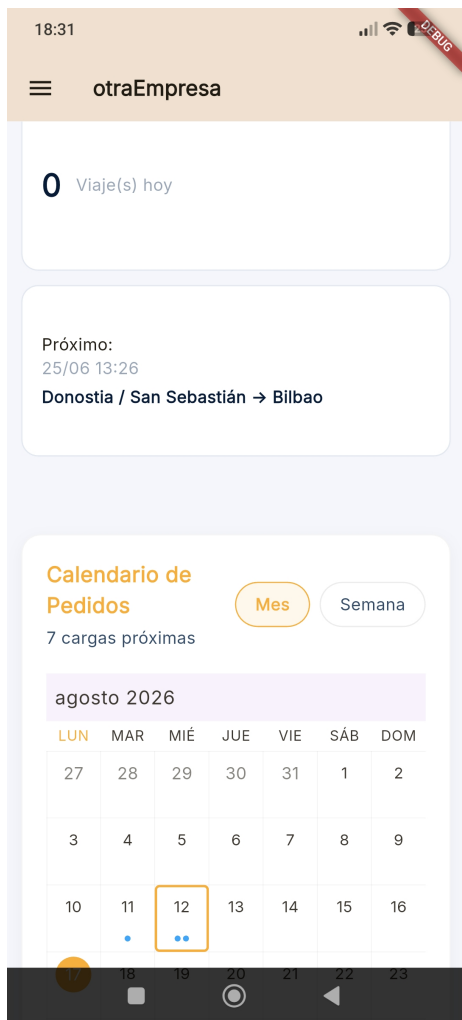
Capacidad (l) 3.0 Largo (m) 3.0 Ancho (m) 3.0 Alto (m) 3.0

Matrícula Remolque (opcional) 8666AAA

Actualizar Vehículo

(b) Detalles de un vehículo.

Figura A.3: Interfaces de consulta y gestión de conductores y vehículos.

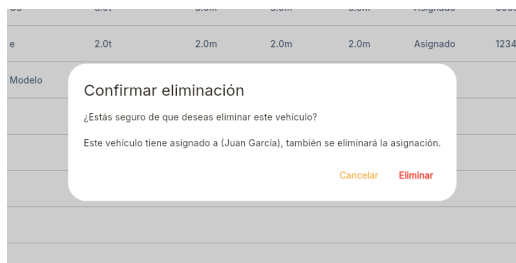


(a) Vista principal de la hoja de ruta.



(b) Opciones disponibles para una carga.

Figura A.4: Interfaces móviles del módulo de operaciones del conductor.



(a) Confirmación de eliminación de un vehículo.



(b) Confirmación de eliminación de un usuario externo.

Figura A.5: Diálogos de confirmación para operaciones de eliminación.

A. INTERFACES DE USUARIO SECUNDARIAS

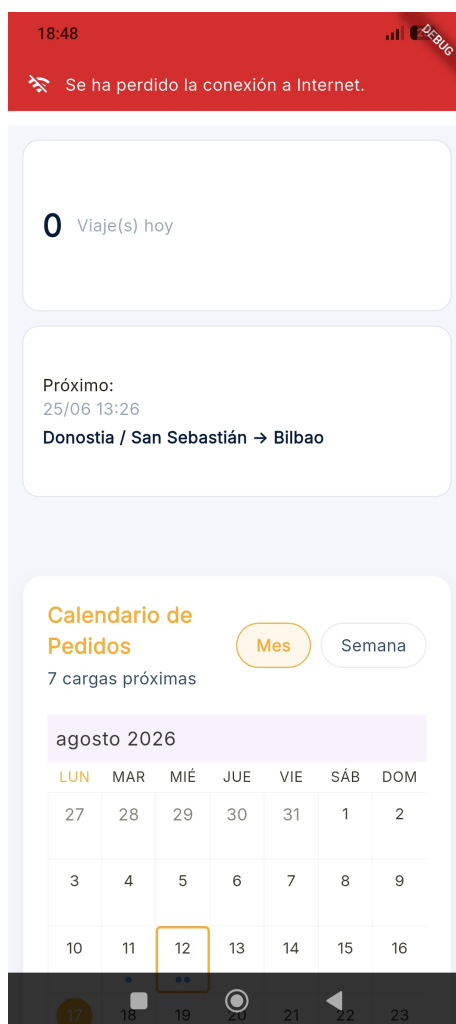


Figura A.6: Aviso mostrado cuando la aplicación pierde la conexión a Internet.

Reglas de seguridad de Firebase

Estas reglas de seguridad limitan el acceso desde el SDK del cliente Flutter a Firebase. Al tener un backend propio, la mayor parte de la seguridad se gestiona en él, por lo que las reglas simplemente limitan el acceso a los datos del usuario que inicia sesión para que solo pueda leer los datos de su perfil y realizar las lecturas estrictamente necesarias para el consumo de datos en tiempo real: la lectura de incidencias del encargado y de las asignaciones de cargas a los chóferes.

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4
5     function isSignedIn() {
6       return request.auth != null;
7     }
8
9     function isEncargado() {
10      return isSignedIn() && "encargado" in request.auth.token.roles;
11    }
12
13    function isTransportista() {
14      return isSignedIn() && "transportista" in request.auth.token.roles;
15    }
16
17    match /users/{userId} {
18      allow read: if isSignedIn() && (request.auth.uid == userId);
19      allow create, update, delete: if isSignedIn() && (request.auth.uid == userId);
20    }
21  }
```

B. REGLAS DE SEGURIDAD DE FIREBASE

```
22 match /clientes/{userId} {
23   allow read: if request.auth.uid == userId;
24 }
25
26 match /subcontratados/{userId} {
27   allow read: if request.auth.uid == userId;
28 }
29
30 match /{path=**}/cargas/{cargaId} {
31   allow read: if isTransportista() && request.auth.uid == resource.data.transportistaId;
32 }
33
34 match /incidencias/{incidenciaId} {
35   allow read: if isEncargado() && resource.data.companyId ==
    ↪ request.auth.token.companyId;
36 }
37
38 match /{document=**} {
39   allow read, write: if false;
40 }
41 }
42 }
```

MultiProvider completo

```
1 MultiProvider(  
2   providers: [  
3     Provider<ConnectivityService>(create: (_) => ConnectivityService()),  
4     ChangeNotifierProvider<ConnectivityProvider>(  
5       create: (context) => ConnectivityProvider(  
6         connectivityService: context.read<ConnectivityService>(),  
7       ),  
8     ),  
9     Provider<AuthService>(create: (_) => AuthService()),  
10    Provider<AuthTokenProvider>(  
11      create: (context) => AuthTokenProvider(context.read<AuthService>()),  
12    ),  
13    ChangeNotifierProvider<AuthProvider>(  
14      create: (context) => AuthProvider(authService: context.read<AuthService>()),  
15    ),  
16    ChangeNotifierProvider<CargaProvider>(  
17      create: (context) => CargaProvider(tokenProvider: context.read<AuthTokenProvider>()),  
18    ),  
19    ChangeNotifierProxyProvider2<AuthTokenProvider, CargaProvider, DashboardProvider>(  
20      create: (context) => DashboardProvider(  
21        tokenProvider: context.read<AuthTokenProvider>(),  
22        cargaProvider: context.read<CargaProvider>(),  
23      ),  
24      update: (context, tokenProvider, cargaProvider, dashboardProvider) {  
25        dashboardProvider!.refresh();  
26        return dashboardProvider;  
27      },  
28    ),  
29    ChangeNotifierProxyProvider<AuthProvider, IncidenciaProvider>(
```

C. MULTIPROVIDER COMPLETO

```
30     create: (context) => IncidenciaProvider(  
31         companyId: '',  
32         tokenProvider: context.read<AuthTokenProvider>(),  
33     ),  
34     update: (context, authProvider, previous) {  
35         final companyId = authProvider.user?.companyId ?? '';  
36         if (previous?.companyId == companyId) return previous!;  
37         return IncidenciaProvider(  
38             companyId: companyId,  
39             tokenProvider: context.read<AuthTokenProvider>(),  
40         );  
41     },  
42 ),  
43 ChangeNotifierProvider<PedidoProvider>(  
44     create: (context) => PedidoProvider(tokenProvider: context.read<AuthTokenProvider>()),  
45 ),  
46 ChangeNotifierProvider<VehiculoProvider>(  
47     create: (context) => VehiculoProvider(tokenProvider:  
48         ↪ context.read<AuthTokenProvider>()),  
49 ),  
50 ChangeNotifierProvider<TransportistaProvider>(  
51     create: (context) => TransportistaProvider(tokenProvider:  
52         ↪ context.read<AuthTokenProvider>()),  
53 ),  
54 ChangeNotifierProvider<InviteProvider>(  
55     create: (context) => InviteProvider(tokenProvider: context.read<AuthTokenProvider>()),  
56 ),  
57 ],  
58 )
```

Pipeline CI completo

```
1 name: CI Tests
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7     branches: [ "main" ]
8   workflow_dispatch:
9
10 jobs:
11   frontend-tests:
12     name: Flutter Tests
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v4
16
17       - name: Set up Flutter
18         uses: subosito/flutter-action@v2
19         with:
20           channel: 'stable'
21
22       - name: Install dependencies
23         run: flutter pub get
24         working-directory: ./frontend
25
26       - name: Generate Mocks
27         run: dart run build_runner build --delete-conflicting-outputs
28         working-directory: ./frontend
29
```

D. PIPELINE CI COMPLETO

```
30   - name: Create Mock dart_defines.json for Flutter
31     run: |
32       echo '{"API_BASE_URL": "http://localhost:8000", "SYNCFUSION_KEY": "mock",
33         ↪ "VAPID_PUBLIC_KEY": "mock"}' > dart_defines.json
34     working-directory: ./frontend
35
36   - name: Run Tests
37     run: flutter test --coverage
38     working-directory: ./frontend
39
40   - name: Upload Flutter coverage
41     uses: actions/upload-artifact@v4
42     with:
43       name: flutter-coverage
44       path: frontend/coverage/lcov.info
45
46 unit-tests:
47   name: FastAPI Unit Tests
48   env:
49     FIREBASE_CREDENTIALS_JSON: ${ secrets.FIREBASE_CREDENTIALS_JSON }
50   runs-on: ubuntu-latest
51   steps:
52     - uses: actions/checkout@v4
53
54     - name: Set up Python
55       uses: actions/setup-python@v5
56       with:
57         python-version: '3.13'
58         cache: 'pip' # caching pip dependencies
59
60     - name: Install dependencies
61       run: pip install -r backend/requirements.txt
62
63     - name: Create Firebase Config Mock
64       run: |
65         echo "{}" > gestion-transporte-dev-firebase-adminsdk-fbsvc-390f3bdf34.json
66       working-directory: ./backend
67
68     - name: Run Tests with pytest
69       run: pytest app/tests/ -m "not integration" --cov=app --cov-report=xml:coverage.xml
70       working-directory: ./backend
71       env:
72         PYTHONPATH: .
73         FIREBASE_CREDENTIALS_PATH:
74           ↪ gestion-transporte-dev-firebase-adminsdk-fbsvc-390f3bdf34.json
75
76     - name: Upload Python coverage
77       uses: actions/upload-artifact@v4
78       with:
79         name: python-coverage
80         path: backend/coverage.xml
81
82 integration-tests:
```

```

81  name: FastAPI Integration Tests
82  env:
83    FIREBASE_CREDENTIALS_JSON: ${ secrets.FIREBASE_CREDENTIALS_JSON }
84    GLOUD_PROJECT: test-project
85  runs-on: ubuntu-latest
86  steps:
87    - uses: actions/checkout@v4
88
89    - name: Set up Python
90      uses: actions/setup-python@v5
91      with:
92        python-version: '3.13'
93        cache: 'pip'
94
95    - name: Set up Node (para Firebase CLI)
96      uses: actions/setup-node@v4
97      with:
98        node-version: '20'
99
100   - name: Set up Java for Firestore Emulator
101     uses: actions/setup-java@v4
102     with:
103       distribution: 'temurin'
104       java-version: '21'
105
106   - name: Install Firebase CLI
107     run: npm install -g firebase-tools
108
109   - name: Install dependencies
110     run: pip install -r backend/requirements.txt
111
112   - name: Create Firebase Config Mock
113     run: echo "{}" > gestion-transporte-dev-firebase-adminsdk-fbsvc-390f3bdf34.json
114     working-directory: ./backend
115
116   - name: Run integration tests against Firestore Emulator
117     working-directory: ./backend
118     run: |
119       firebase emulators:exec --project test-project --only firestore \
120         "pytest app/tests/integration_tests.py -m integration --cov=app
121         ↪ --cov-report=xml:coverage-integration.xml"
122   env:
123     PYTHONPATH: .
124     FIREBASE_CREDENTIALS:
125       ↪ gestion-transporte-dev-firebase-adminsdk-fbsvc-390f3bdf34.json
126
127   - name: Upload integration coverage
128     uses: actions/upload-artifact@v4
129     with:
130       name: python-coverage-integration
131       path: backend/coverage-integration.xml
132
133  sonarqube:

```

D. PIPELINE CI COMPLETO

```
132  name: SonarQube
133  runs-on: ubuntu-latest
134  needs: [ frontend-tests, unit-tests, integration-tests ]
135  if: always()
136  steps:
137    - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
138      with:
139        fetch-depth: 0
140
141    - uses: subosito/flutter-action@v2
142      with:
143        channel: 'stable'
144
145    - name: Install Frontend Deps
146      run: flutter pub get
147      working-directory: ./frontend
148
149    - name: Generate Mocks
150      run: dart run build_runner build --delete-conflicting-outputs
151      working-directory: ./frontend
152
153    - name: Download Flutter coverage
154      uses: actions/download-artifact@v4
155      continue-on-error: true
156      with:
157        name: flutter-coverage
158        path: frontend/coverage
159
160    - name: Download Python coverage
161      uses: actions/download-artifact@v4
162      continue-on-error: true
163      with:
164        name: python-coverage
165        path: backend
166
167    - name: Download Python integration coverage
168      uses: actions/download-artifact@v4
169      continue-on-error: true
170      with:
171        name: python-coverage-integration
172        path: backend
173
174    - name: SonarQube Scan
175      uses: SonarSource/sonarqube-scan-action@efd88b7d7ccbaefd23d8f36f73b59db7a3d246602 #
176      ↪ v6.0.0
177      env:
178        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Documentación técnica

E.1. Configuración del entorno de desarrollo

E.1.1. Configuración del Backend (FastAPI)

1. Creación y activación del entorno virtual:

```
1 cd backend
2 python -m venv .venv
3
4 # En Windows (PowerShell):
5 .\.venv\Scripts\Activate.ps1
6
7 # En Linux / macOS:
8 source .venv/bin/activate
```

2. Instalación de dependencias:

```
1 pip install --upgrade pip
2 pip install -r requirements.txt
```

3. Variables de entorno locales: Modificar el .env.example de la carpeta backend/ con las credenciales necesarias (clave privada del SDK de Firebase Admin, URL del frontend, etc.):

```
1 FIREBASE_CREDENTIALS_PATH="<archivo de credenciales de Firebase>"
2 FRONTEND_URL="<URL del frontend (local)>"
```

```
3 EMAIL_USER="<Correo electrónico del que se envían los correos>"
4 EMAIL_PASSWORD="<Contraseña de aplicación (App Password)>"
```

4. Ejecución en servidor local de desarrollo:

```
1 uvicorn app.main:app --reload --port 8000
```

E.1.2. Configuración del Frontend (Flutter) y Flavors

Primero hay que crear tres proyectos en Firebase (Dev, Stg y Prod) y seguir las instrucciones de abajo para vincularlos al proyecto local. Para configurar los Flavors, también se puede seguir el tutorial de Code With Andrea [6].

1. Configuración del CLI de Firebase:

```
1 # Instalación global de Firebase CLI
2 npm install -g firebase-tools
3
4 # Instalación de FlutterFire CLI
5 dart pub global activate flutterfire_cli
6
7 # Autenticación en la cuenta de Google/Firebase
8 firebase login
```

2. Configuración de Flavors:

```
1 cd frontend
2
3 # Configuración de Flavors
4 # Configuración para el entorno de Desarrollo (dev)
5 flutterfire configure \
6   --project=<tu-proyecto-firebase-dev> \
7   --out=lib/firebase_options_dev.dart \
8   --ios-bundle-id=com.example.gestionTransporte.dev \
9   --android-package-name=com.example.gestion_transporte.dev
10
11 # Configuración para el entorno de Staging (stg)
12 flutterfire configure \
13   --project=<tu-proyecto-firebase-stg> \
14   --out=lib/firebase_options_stg.dart \
15   --ios-bundle-id=com.example.gestionTransporte.stg \
16   --android-package-name=com.example.gestion_transporte.stg
17
```

```

18 # Configuración para el entorno de Producción (prod)
19 flutterfire configure \
20   --project=<tu-proyecto-firebase-prod> \
21   --out=lib/firebase_options_prod.dart \
22   --ios-bundle-id=com.example.gestionTransporte \
23   --android-package-name=com.example.gestion_transporte

```

3. Fichero dart_defines.json:

```

1 {
2   "API_BASE_URL": "http://127.0.0.1:8000",
3   "SYNCFUSION_KEY": "TU_LICENCIA_LOCAL",
4   "VAPID_PUBLIC_KEY": "TU_CLAVE_VAPID_LOCAL"
5 }

```

4. Arrancar el frontend:

```

1 flutter pub get
2 flutter run -d chrome --flavor dev --web-port=5500
  ↪ --dart-define-from-file=dart_defines.json

```

E.2. Documentación de la API

El backend cuenta con documentación interactiva en Swagger generada de forma automática por FastAPI, para interactuar se necesita un token JWT válido.

<https://backend-tfg-transporte-s7ctmqnma-no.a.run.app/swagger>

E.3. Manual de despliegue

E.3.1. Despliegue del Backend (Google Cloud Run)

Primero, hay que asegurarse de que este Dockerfile está en la carpeta backend.

```

1 FROM python:3.12-slim
2
3 RUN apt-get update && apt-get install -y \
4   libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf-2.0-0 \
5   libffi-dev shared-mime-info \
6   && rm -rf /var/lib/apt/lists/*
7

```

```
8 WORKDIR /app
9 COPY requirements.txt .
10 RUN pip install --no-cache-dir -r requirements.txt
11 COPY . .
12
13 CMD exec uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8080}
```

Se despliega directamente desde el CLI de Google Cloud.

```
1 # 1. Autenticación y selección de proyecto
2 gcloud auth login
3 gcloud config set project <ID_DE_TU_PROYECTO_GCP>
4
5 # 2. Despliegue del servicio desde el código fuente
6 gcloud run deploy backend-tfg-transporte \
7   --source . \
8   --region europe-southwest1 \
9   --allow-unauthenticated \
10  --max-instances=2 \
11  --set-env-vars="FRONTEND_URL=https://<tu-app-firebase>.web.app"
```

E.3.2. Despliegue del Frontend (Firebase Hosting)

Primero se construye el proyecto en /build/web, si no se quiere compilación WebAssembly, quitar el flag `-wasm`.

```
1 cd frontend
2
3 # Limpieza de caché e instalación de dependencias
4 flutter clean
5 flutter pub get
6
7 # Compilación para la web con soporte WebAssembly
8 flutter build web --wasm --dart-define-from-file=dart_defines.json
```

Cuando los archivos están generados, se despliega utilizando el CLI de Firebase.

```
1 # No hace falta si ya se ha hecho en la fase anterior
2 firebase login
3
4 # Despliegue
5 firebase deploy --only hosting
```

Bibliografía

- [1] Jefatura del Estado. Ley 9/2025, de 3 de diciembre, de Movilidad Sostenible. Boletín Oficial del Estado, núm. 291, de 4 de diciembre de 2025, 2025. Disposición transitoria octava. Accedido: 13 de julio de 2026. Disponible en: <https://www.boe.es/eli/es/l/2025/12/03/9>. Ver página 1.
- [2] Ministerio de Transportes y Movilidad Sostenible. Resolución de 5 de junio de 2026, de la Dirección General de Transporte por Carretera y Ferrocarril, por la que se establecen las características que deben reunir los sistemas y los documentos electrónicos de control administrativo exigidos en los transportes por carretera. Boletín Oficial del Estado, núm. 143, de 12 de junio de 2026, 2026. Accedido: 13 de julio de 2026. Disponible en: <https://www.boe.es/eli/es/res/2026/06/05/2>. Ver páginas 4, 18.
- [3] ISO/IEC. ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. International Organization for Standardization, 2023. Disponible en: <https://iso25000.com/index.php/normas-iso-25000/iso-25010>. Ver página 17.
- [4] Jefatura del Estado. Ley 15/2009, de 11 de noviembre, del contrato de transporte terrestre de mercancías. Boletín Oficial del Estado, núm. 273, de 12 de noviembre de 2009, 2009. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2009-18004>. Ver página 18.
- [5] Ministerio de Fomento. Orden FOM/2861/2012, de 13 de diciembre, sobre el documento de control administrativo del transporte de mercancías por carretera. Boletín Oficial del Estado, núm. 310, de 26 de diciembre de 2012, 2012. Accedido: 16 de junio de 2026. Disponible en: <https://www.boe.es/eli/es/o/2012/12/13/fom2861>. Ver página 22.
- [6] Andrea Bizzotto. Flutter & Firebase: How to setup Multiple Flavors using FlutterFire CLI. Code With Andrea, 2022. Guía técnica sobre la configuración de entornos en Flutter con Firebase CLI. Accedido: 20 de julio de 2026. Disponible en: <https://codewithandrea.com/articles/flutter-firebase-multiple-flavors-flutterfire-cli/>. Ver página 106.